

基于T r a e的编程兴趣班

入门百练

C++ & Python 双语对照 · 四角色叙事对话

共16章 · 161题

覆盖语法基础→编程进阶→核心算法→算法进阶

小鲁 · 小华 · 小栋 · 小嘉

厦门大学 · 自强不息，止于至善

目 录

第一阶段：语法基础		
第1章	程序设计的第一个脚印 — 变量、输入输出与顺序结构	14题
第2章	选择的艺术 — 条件判断与分支结构	14题
第3章	循环的魔力 — for/while与嵌套循环	14题
第4章	数据的容器 — 数组与线性存储	12题
第5章	矩阵的舞蹈 — 多维数组与矩阵模式	12题
第二阶段：编程进阶		
第6章	字符的世界 — 字符串处理	12题
第7章	模块化的力量 — 函数、递归与库的威力	12题
第8章	指针与抽象 — 结构体、指针与STL容器	10题
第三阶段：核心算法		
第9章	分治之美 — 排序与二分	7题
第10章	预处理的智慧 — 前缀和、差分与双指针	7题
第11章	计算的边界 — 高精度、位运算与离散化	7题
第12章	结构的魔力 — 基础数据结构	8题
第13章	搜索的艺术 — 搜索与回溯	8题
第四阶段：算法进阶		
第14章	图的疆域 — 图论入门	8题
第15章	状态的艺术 — 动态规划	8题
第16章	终极试炼 — 数学、贪心与综合实战	8题

共计161题

程序设计的第一个脚印

第1章 · 变量、输入输出与顺序结构 · 14题 · C++ & Python 双语对照

九月，厦门大学芙蓉湖畔，凤凰花开得正红。小鲁背着新买的笔记本电脑走进海韵园实验室——今天是他第一节编程课。

"编程？那不是数学天才华罗庚才玩的东西吗？"小鲁心里打鼓。但当他看到屏幕上第一个程序的输出结果时，他愣住了："这么简单？我也会！"

本章14道题，从最简单的A+B开始，到多类型混合运算。你会认识小鲁、小华（数学天才）、小嘉（校主精神）和小栋（工程师），跟着他们的对话，一步步踏入编程的世界。**记住：编程不难，难的是开始。现在，打开TRAE，写下你的第一行代码吧。**

NQ001 A + B AcWing 1

小鲁第一次打开TRAE。AI助手弹出提示："你好，我是你的编程伙伴！想学编程，就从最简单的计算开始吧——输入两个整数，我来帮你算它们的和。"小鲁半信半疑地输入了"3 4"……屏幕上出现了"7"。"就这么简单？！"小鲁惊讶地喊道。小华从旁边探过头来："没错！程序本质上就是三个步骤——输入数据、计算处理、输出结果。你刚刚写的，就是世界上最著名的'Hello World'在算法竞赛中的版本。"

输入 一行，两个整数A和B，用空格隔开。

输出 一个整数，即A+B的结果。

范围 $0 \leq A, B \leq 10^8$

输入	输出
3 4	7

思路 这是所有程序的基本骨架：读入→计算→输出。C++用cin/cout，Python用input/print。两种语言体现了不同的设计哲学——C++需要声明main函数和变量类型，Python则更直白，直接写逻辑即可。

技巧 Python的[a,b=map\(int,input\(\).split\(\)\)](#)是处理空格分隔整数输入的标准写法。C++的cin>>a>>b自动跳过空白字符。记住这个模式，它将伴随你整个编程生涯。

C++

```
#include <iostream>
using namespace std;
int main() { int a, b; cin >> a >> b; cout << a + b << endl; return 0; }
```

Python

```
import sys
data = sys.stdin.read().split()
if data:
    a, b = map(int, data[:2])
    print(a + b)
```

NQ002 差 AcWing 608

"既然能做加法，那减法呢？乘法呢？"小鲁迫不及待地问。小华笑着在纸上写下： $A \times B - C \times D$ ，四个数，先各自相乘，再相减。"小鲁试着输入四个数……" $DIFERENCA = -26$ "出现了！"原来编程就是换个方式做数学题啊。"小华点点头："从某种意义上说，是的——编程就是把数学思维转化为机器能理解的指令。"

输入 一行，四个整数A、B、C、D，用空格隔开。

输出 输出" $DIFERENCA =$ "后跟计算结果。

范围 $-10^4 \leq A, B, C, D \leq 10^4$

输入

5 6 7 8

输出

$DIFERENCA = -26$

思路 按公式计算，注意运算顺序（先乘后减）。C++用int可满足范围。Python的int无范围限制。输出需要带前缀" $DIFERENCA =$ "——这种"前缀+结果"的格式是OJ题目常见要求。

技巧 Python的f-string格式化： $f"DIFERENCA = \{a*b-c*d\}"$ 。f-string是Python 3.6+最推荐的格式化方式，比%和format()更直观清晰。

C++

```
#include <stdio>
int main() { int a,b,c,d; scanf("%d%d%d%d",&a,&b,&c,&d);
printf("DIFERENCA = %d\n",a*b-c*d); return 0; }
```

Python

```
s = __import__('sys').stdin.read()
nums = []
for tok in s.replace('\r',' ').replace('\n',' ').split(' '):
    tok = tok.strip()
    if tok:
        try: nums.append(int(tok))
        except: pass
a,b,c,d = nums[0], nums[1], nums[2],
nums[3]
print(f"DIFERENCA = {a * b - c * d}")
```

NQ003 圆的面积 AcWing 604

小嘉路过时看到了小鲁在练习编程。"年轻人，你知道 π 吗？"小嘉问。"就是3.14159……"小鲁回答。"对，我在南洋经商时常常用到它——计算圆形的土地面积。来，写个程序，输入半径，算出圆的面积。"小鲁照做了，屏幕上出现了" $A=12.5664$ "。"看，编程不只是做题——它能解决真实世界的问题。"

输入 一个浮点数r，表示圆的半径。

输出 输出" $A=$ "后跟面积，保留4位小数。

范围 $0 < r \leq 10000$

输入

2.00

输出

A=12.5664

思路 圆面积= πr^2 。需要浮点数类型：C++的double，Python的float。C++用printf("%.4lf")控制小数位；Python用f"{area:.4f}"。这是第一次用到实数精度控制。

技巧 C++两种格式化输出：C风格printf("%.4lf",x)简洁精确；C++风格cout<

C++

```
#include <cstdio>
int main() { double r; scanf("%lf",&r); printf("A=%.4lf\n",3.14159*r*r); return 0; }
```

Python

```
r = float(input())
print(f"A={3.14159 * r * r:.4f}")
```

NQ004 平均数1 AcWing 606

小鲁拿到了两个科目的成绩：一门权重3.5，一门7.5。"哎，这加权平均怎么算来着？"小华提醒他：" $(A \times 3.5 + B \times 7.5) / (3.5 + 7.5)$ 。注意分母是权重之和=11，不是科目数2——很多人都会搞错。"原来如此！"小鲁恍然大悟，"就像奖学金评定，不同课程的重要程度不一样。"

输入

两行，每行一个浮点数（成绩A和B）。

输出

"MEDIA = "后跟加权平均分，保留5位小数。

范围

 $0 \leq A, B \leq 10.0$

输入

5.0
7.1

输出

MEDIA = 6.43182

思路 加权平均公式： $(A \times 3.5 + B \times 7.5) / (3.5 + 7.5) = (A \times 3.5 + B \times 7.5) / 11$ 。分母是权重之和，不是题目数——这是加权平均最常见的陷阱。保留5位小数需要精确的浮点格式化。

技巧 C++所有浮点字面量默认double。Python的/总是返回浮点数。注意： $3.5 + 7.5 = 11.0$ 不是2——这与普通平均完全不同。

C++

```
#include <cstdio>
int main() { double a,b; scanf("%lf%lf",&a,&b); printf("MEDIA = %.5lf\n",(a*3.5+b*7.5)/11); return 0; }
```

Python

```
import sys
data = sys.stdin.read().split()
if data:
    a = float(data[0])
    b = float(data[1])
    print(f"MEDIA = {(a * 3.5 + b * 7.5) / 11:.5f}")
```

NQ005 工资 AcWing 609

小鲁暑假打工了！他需要计算自己的工资——员工编号25，工作了100小时，时薪5.50元。"帮我写个程序算总工资！"小栋接过键盘："工资=时数×时薪。输入包含整数和浮点数混合类型。注意输出两行——第一行编号，第二行工资（保留两位小数）。"

输入 三行：编号（整数）、时数（整数）、时薪（浮点数）。

输出 "NUMBER = X"换行"SALARY = U\$ XX.XX"。

范围 $1 \leq \text{编号} \leq 100, 1 \leq \text{时数} \leq 200, 1 \leq \text{时薪} \leq 50$

输入

25
100
5.50

输出

NUMBER = 25
SALARY = U\$ 550.00

思路 工资=时数×时薪。混合使用int和double/float：C++的scanf可精确控制格式，Python逐行读取更清晰。输出两行各自需要格式化。

技巧 C++: scanf("%d%d%lf",&n,&h,&m)精确控制混合输入。Python: 逐行int(input())和float(input())更安全清晰。

C++

```
#include <cstdio>
int main() { int n,h; double m; scanf("%d%d%lf",&n,&h,&m); printf("NUMBER = %d\nSALARY = U$ %.2lf\n",n,h*m); return 0; }
```

Python

```
import sys
data = sys.stdin.read().split()
if data:
    n = int(data[0])
    h = int(data[1])
    m = float(data[2])
    print(f"NUMBER = {n}")
    print(f"SALARY = U$ {h * m:.2f}")
```

NQ006 油耗 AcWing 615

小鲁买了一辆二手车，想测试它的油耗。他记录了行驶500公里用了35升油。"每升能跑多少公里？" $500 \div 35 \approx 14.286$ 。小华说："这是除法运算——但注意，要用浮点数除法，否则C++会做整数除法截断。Python的/自动浮点，安全得多。"

输入 两行：距离（浮点数，km）、汽油量（浮点数，L）。

输出 每升公里数，保留3位小数，后跟" km/l"。

范围 $1 \leq \text{距离} \leq 10^6, 0 < \text{汽油量} \leq 10^5$

输入

输出

14.286 km/l

500
35.0

思路 燃油效率=距离÷汽油量。浮点除法，保留3位小数。C++中距离如果是整数需先转double。Python的/自动浮点——这是两种语言在处理除法上的核心差异。

技巧 C++整数除法陷阱：5/2=2不是2.5！必须写成5.0/2或(double)5/2。Python的5/2=2.5自动浮点——这也是C++初学者最易犯的错误之一。

C++

```
#include <stdio>
int main() { double x,y; scanf("%lf%lf",&x,&y); printf("%.3lf km/l\n",x/y); return 0; }
```

Python

```
import sys
data = sys.stdin.read().split()
if data:
    x = float(data[0])
    y = float(data[1])
    print(f"{x / y:.3f} km/l")
```

NQ007 两点间的距离 AcWing 616

小鲁用坐标纸画了两个点：P1(1,7)和P2(5,9)。“这两点之间距离是多少？”小栋说：“欧几里得距离= $\sqrt{(x_1-x_2)^2+(y_1-y_2)^2}$ ”。这需要用数学库的sqrt函数——编程第一次用到外部库。”

输入 一行，四个浮点数x1 y1 x2 y2。

输出 两点间距离，保留4位小数。

范围 $-10^9 \leq \text{坐标} \leq 10^9$

输入

1.0 7.0 5.0 9.0

输出

4.4721

思路 距离= $\sqrt{(x_1-x_2)^2+(y_1-y_2)^2}$ 。C++需要#include，Python需import math。注意使用double而非float（精度更高）。保留4位小数。

技巧 C++链接时需要-lm（某些编译器）。Python的import math后调用math.sqrt()。f"{dist:.4f}"格式化输出。这是数学库使用的第一课。

C++

```
#include <cstdio>
#include <cmath>
int main() { double x1,y1,x2,y2; scanf("%lf%lf%lf%lf",&x1,&y1,&x2,&y2); printf("%.4lf\n",sqrt((x1-x2)*(x1-x2)+(y1-y2)*(y1-y2))); return 0; }
```

Python

```
import sys
import math
data = sys.stdin.read().split()
if data:
    x1, y1, x2, y2 = map(float, data[:4])
    d = math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)
    print(f"{d:.4f}")
```

NQ008 钞票 AcWing 653

小鲁去银行取576元，ATM机吐出了：5张100、1张50、1张20、0张10、1张5、0张2、1张1。"为什么是这些面额？"小华解释道："这是贪心策略——从最大面额开始，能用多少用多少，余下的交给更小面额。你看，100元用5张还剩76，50元用1张还剩26……"小鲁瞪大了眼睛："这算法好聪明！"

输入

一个整数N。

输出

第一行N，然后7行按面额100到1输出张数。

范围

 $0 < N < 10^6$

输入

576

输出

576

```
5 nota(s) de R$ 100,00
1 nota(s) de R$ 50,00
1 nota(s) de R$ 20,00
0 nota(s) de R$ 10,00
1 nota(s) de R$ 5,00
0 nota(s) de R$ 2,00
1 nota(s) de R$ 1,00
```

思路 贪心策略：count=N/face_value; N%=face_value。C++写了7段重复代码，Python用列表+循环消除了重复——这是本课最重要的工程思维：用数据结构消除重复逻辑。

技巧 Python: for v in [100,50,20,10,5,2,1]: print(f"{n//v} nota(s) de R\$ {v},00"); n%=v。7行→3行，这就是"用数据结构替代重复代码"的力量。

C++

```
#include <cstdio>
int main() { int n,s[]={100,50,20,10,5,2,1}; scanf("%d",&n); printf("%d\n",n); for(int i=0;i<7;i++){printf("%d nota(s) de R$ %d,00\n",n/s[i],s[i]);n%=s[i];} return 0; }
```

Python

```
import sys
data = sys.stdin.read().split()
if data:
    n = int(data[0])
    print(n)
    for v in [100, 50, 20, 10, 5, 2, 1]:
        print(f"{n // v} nota(s) de R$ {v},00")
        n %= v
```

NQ009 时间转换 AcWing 654

"556秒等于多少小时多少分钟多少秒？"小鲁拿着计时器犯了难。"这简单——"小华说，"556÷3600=0小时，余556。556÷60=9分钟，余16。所以是0:9:16。Python有个divmod()函数可以一次得到商和余数，特别方便。"

输入

一个整数N。

输出

HH:MM:SS格式。

范围

 $0 \leq N \leq 10^6$

输入

556

输出

0:9:16

思路 时=N/3600，分=N%3600/60，秒=N%60。Python的divmod(a,b)返回(商,余数)元组：
h,r=divmod(n,3600); m,s=divmod(r,60)。

技巧 Python的divmod()优雅解包接收。C++需分别计算/和%。divmod是Python"电池已包含"哲学的体现——常见操作都内置了。

C++

```
#include <cstdio>
int main() { int n; scanf("%d",&n); printf("%d:%d:%d",n/3600,n%3600/60,n%60); return 0; }
```

Python

```
import sys
data = sys.stdin.read().split()
if data:
    n = int(data[0])
    h = n // 3600
    m = n % 3600 // 60
    s = n % 60
    print(f"{h}:{m}:{s}")
```

NQ010 简单乘积 AcWing 605

"A+B已经会了，A×B呢？"小鲁跃跃欲试。小华把键盘推过去："你自己来。把第一题的代码复制过来，把+改成*，再加个'PROD = '前缀。"小鲁三下五除二就搞定了——"原来修改已有代码比从零开始快这么多！"

输入 两行，每行一个整数。

输出 "PROD = "后跟乘积。

范围 $-10^4 \leq A, B \leq 10^4$

输入

3
9

输出

PROD = 27

思路 与NQ001结构完全相同，+改成*，加"PROD = "前缀即可。这是让学生独立完成"修改→编译→AC"完整流程的训练题。

技巧 修改已有代码是最实用的编程技能之一。用TRAE打开NQ001的代码，告诉AI"把加法改成乘法，加PROD前缀"——这就是AI协同编程的日常。

C++

```
#include <iostream>
using namespace std;
int main() { int a,b; cin>>a>>b; cout<<"PROD = "<<a*b<<endl; return 0; }
```

Python

```
import sys
data = sys.stdin.read().split()
a, b = int(data[0]), int(data[1])
print(f"PROD = {a * b}")
```

NQ011 简单计算 AcWing 611

小鲁去买东西：产品编号12，买1个，单价5.30元。"总共多少钱？"小栋说："数量×单价。注意输入混合了整数（编号和数量）和浮点数（单价）。int×float自动转换为float——数据类型会自动'提升'到更精确的类型。"

输入 第一行：code和quantity（整数）。第二行：price（浮点数）。

输出 "VALOR A PAGAR: R\$ XX.XX"。

范围 $1 \leq \text{code} \leq 100, 1 \leq \text{quantity} \leq 100$

输入

12 1
5.30

输出

VALOR A PAGAR: R\$ 5.30

思路 总价=数量×单价。int×float自动提升为float/double。C++中int×double→double。Python中int×float→float。类型提升规则确保精度不丢失。

技巧 类型自动提升：宽类型"吸收"窄类型。记住这个原则：运算中最宽的类型决定结果类型。

C++

```
#include <cstdio>
int main() { int c,q; double p; scanf("%d%d%lf",&c,&q,&p); printf("VALOR A PAGAR: R$ %.2lf\n",q*p); return 0; }
```

Python

```
import sys
data = sys.stdin.read().split()
c, q, p = int(data[0]), int(data[1]), float(data[2])
print(f"VALOR A PAGAR: R$ {q * p:.2f}")
```

NQ012 球的体积 AcWing 612

小鲁在物理实验课上测量了一个半径3cm的球。"这个球的体积是多少？"小华写下公式： $V=(4/3)\pi r^3$ 。"注意——C++中4/3等于1不是1.333！整数除法会截断。必须写成4.0/3.0。这是C++初学者最经典的陷阱。"

输入 一个整数 r 。

输出 "VOLUME = "后跟体积，保留3位小数。

范围 $1 \leq r \leq 1000$

输入

3

输出

VOLUME = 113.097

思路 $V=(4.0/3.0)\times\pi\times r^3$ 。C++中4/3=1(整数除法)！必须写成4.0/3.0。Python自动浮点无此问题。这是C++整数除法的经典陷阱。

技巧 C++的4/3=1是初学者最容易犯的错误。任何涉及除法的表达式，至少保证一个操作数是浮点数。Python没有这个烦恼。

C++

```
#include <cstdio>
int main() { int r; scanf("%d",&r); printf("VOLUME = %.3lf\n",4.0/3.0*3.14159*r*r*r); return 0; }
```

Python

```
import sys
data = sys.stdin.read().split()
if data:
    r = float(data[0])
    v = (4.0 / 3.0) * 3.14159 * r ** 3
    print(f"VOLUME = {v:.3f}")
```

NQ013 面积 AcWing 613

"一题算三种图形的面积——三角形 $A\times C/2$ 、圆 $\pi\times C^2$ 、梯形 $(A+B)\times C/2$ 。注意A、B、C在不同公式中担任不同的角色！"小华提醒道。"这题考察的是细心——公式本身都很简单，但把三个公式的正确参数搞对才是关键。"

输入 一行，三个浮点数A、B、C。

输出 三行：TRIANGULO/CIRCULO/TRAPEZIO，各保留3位小数。

范围 $0 < A, B, C \leq 100$

输入

3.0 4.0 5.2

输出

TRIANGULO: 7.800
CIRCULO: 84.949
TRAPEZIO: 18.200

思路 三个独立公式，每个一行计算一行输出。三角形= $A \times C / 2$ ，圆= $3.14159 \times C^2$ ，梯形= $(A+B) \times C / 2$ 。
PI=3.14159。代码组织能力训练。

技巧 一道题含三个计算是测试代码组织能力的好题。每个面积独立计算、独立输出，清晰直观。多使用const/常量避免魔法数字。

C++

```
#include <cstdio>
int main() { double a,b,c; scanf("%lf%lf%lf",&a,&b,&c); printf("TRIANGULO: %.3lf\nCIRCULO: %.3lf\nTRAPEZIO: %.3lf\nQUADRADO: %.3lf\nRETANGULO: %.3lf\n",a*c/2,3.14159*c*c,(a+b)*c/2,b*b,a*b); return 0; }
```

Python

```
import sys
data = sys.stdin.read().split()
if data:
    a, b, c = map(float, data[:3])
    print(f"TRIANGULO: {a * c / 2:.3f}")
    print(f"CIRCULO: {3.14159 * c * c:.3f}")
    print(f"TRAPEZIO: {(a + b) * c / 2:.3f}")
    print(f"QUADRADO: {b * b:.3f}")
    print(f"RETANGULO: {a * b:.3f}")
```

NQ014 最大值 AcWing 614

小鲁拿到三份考试成绩：7分、14分、106分。"最高分是多少?"`max(7,14,106)=106`。"小鲁用Python的`max(a,b,c)`一行搞定。"Python太方便了!"小华说："C++需要`max({a,b,c})`（C++11初始化列表）或嵌套`max`。Python的'电池已包含'设计哲学——常用的都给你准备好了。"

输入

一行，三个整数。

输出

一个整数，即最大值。

范围

 $-10^9 \leq a, b, c \leq 10^9$

输入

7 14 106

输出

106

思路 Python的`max(a,b,c)`接收多参数。C++的`max({a,b,c})`需C++11和，或嵌套`max(a,max(b,c))`。体现了Python"电池已包含"vs C++"按需引入"的设计差异。

技巧 Python内置`max/min/sum/abs`等函数无需import。C++需要`#include`。两种语言的哲学差异在这一题得到完美体现。

C++

```
#include <cstdio>
int main() { int a,b,c; scanf("%d%d%d",&a,&b,&c); int mx=
a; if(b>mx)mx=b; if(c>mx)mx=c; printf("%d eh o maior\n",m
x); return 0; }
```

Python

```
import sys
data = sys.stdin.read().split()
if data:
    a, b, c = map(int, data[:3])
    mx = max(a, b, c)
    print(f"{mx} eh o maior")
```

本章知识点总结

- 程序三要素：输入(Input)→处理(Process)→输出(Output)
- 变量与类型：C++声明类型(int/double)，Python动态类型
- 格式化输出：C++用printf，Python用f-string
- 常见陷阱：C++整数除法截断(4/3=1)，Python无此问题
- 厦大校训：自强不息——编程之路始于足下，每一步都是进步

— 第1章完 · 共14题 · 凤凰花开 · 自强不息 —

选择的艺术

第2章 · 条件判断与分支结构 · 14题 · C++ & Python 双语对照

第二周，小鲁已经能熟练地写A+B了。但小华说："真正的程序不会只是直线执行——你根据不同的情况做出不同的决定。如果成绩及格就通过，否则补考；如果是周一就升旗，否则早读……这就是**条件判断**。"

本章14道题，小鲁将从简单的if-else开始，逐步掌握多分支、嵌套条件、决策树和dict映射表。最重要的是学会**"用数据结构替代逻辑堆叠"**——当你看到一大串if-elif-else的时候，第一反应应该问自己："能不能用一张表来替代？"

C++ vs Python 条件判断速查

- 单分支: `if(cond){...}` → `if cond:...`
- 多分支: `if/else if/else` → `if/elif/else`
- 链式比较: `0<=x&&x<=25` → `0<=x<=25` (Python独有!)
- 映射表: 8个if-else → `dict.get(key,default)` (Python独有!)
- 三元表达式: `cond?a:b` → `a if cond else b`

NQ015 倍数 AcWing 665

小鲁在数学课上学了倍数。"如果A能被B整除，或者B能被A整除，它们就是倍数关系。"小华说："编程里用%取模来判断——`a%b==0`说明a是b的倍数。用`||`(C++)或`or`(Python)把两个条件连起来。"

输入 一行，两个整数A和B。

输出 互为倍数输出"`Sao Multiplos`"，否则"`Nao sao Multiplos`"。

范围 $-10^4 \leq A, B \leq 10^4$ ($A, B \neq 0$)

输入

6 24

输出

Sao Multiplos

思路 用取模判断倍数关系：`A%B==0`或`B%A==0`。两个条件用"或"连接。这是if-else最简单的形式——单一条件判断，两个分支。

技巧 Py的if语句不需要括号。C++的%运算符要求整数。`a%b==0`比`!(a%b)`更易读。

C++

```
#include <iostream>
using namespace std;
int main() {
    int a, b;
    cin >> a >> b;
    if (a % b == 0 || b % a == 0)
        cout << "Sao Multiplos" << endl;
    else
        cout << "Nao sao Multiplos" << endl;
    return 0;
}
```

Python

```
a, b = map(int, input().split())
if a % b == 0 or b % a == 0:
    print("Sao Multiplos")
else:
    print("Nao sao Multiplos")
```

NQ016 零食 AcWing 660

小鲁去小卖部买零食：1号零食4元，2号4.5元，3号5元，4号2元，5号1.5元。"这么多选项，怎么写？"小华说："C++可以用if-else链或switch。Python可以用dict映射——一行搞定全部！"

输入 一行，两个整数：零食编号X和数量Y。

输出 "Total: R\$ "后跟总价（保留2位）。

范围 $1 \leq X \leq 5, 1 \leq Y \leq 100$ 。单价：1=4.00, 2=4.50, 3=5.00, 4=2.00, 5=1.50

输入

3 2

输出

Total: R\$ 10.00

思路 多分支选择。C++用if-else链或switch，Python用if-elif或dict映射。本课重点：dict={1:4.0,2:4.5,3:5.0,4:2.0,5:1.5}，prices[x]*y一行出结果——O(1)查找，代码量减少50%。

技巧 Py的dict映射表：prices={1:4.0,2:4.5,3:5.0,4:2.0,5:1.5}。替代if-elif链。"用数据驱动替代逻辑堆叠"是进阶编程的核心理念。

C++

```
#include <cstdio>
int main() {
    int x, y;
    scanf("%d%d", &x, &y);
    double p;
    if (x == 1) p = 4.0;
    else if (x == 2) p = 4.5;
    else if (x == 3) p = 5.0;
    else if (x == 4) p = 2.0;
    else p = 1.5;
    printf("Total: R$ %.2lf\n", p * y);
    return 0;
}
```

Python

```
prices = {1: 4.0, 2: 4.5, 3: 5.0, 4: 2.0, 5: 1.5}
x, y = map(int, input().split())
print(f"Total: R$ {prices[x] * y:.2f}")
```

NQ017 区间 AcWing 659

"一个数落在哪个区间里?"小鲁看着题目:"0到25是闭区间, 25到50是左开右闭....."小华提醒他:"注意开闭区间的区别——[0,25]包含25, (25,50]不包含25但包含50。Python的链式比较 $0 \leq x \leq 25$ 比C++的 $0 \leq x \&\& x \leq 25$ 更直观。"

- 输入** 一个浮点数 x 。
- 输出** 区间名或"Fora de intervalo"。
- 范围** $-10^9 \leq x \leq 10^9$

输入

25.01

输出

Intervalo (25,50]

思路 从最小区间开始判断, 逐级向上。Python链式比较更直观, C++需分开写 $\&\&$ 。注意开闭括号[包含](不包含)的数学含义——这是区间判断最容易出错的地方。

技巧 Python链式比较: $0 \leq x \leq 25$ 。C++需要 $0 \leq x \&\& x \leq 25$ 。Python的链式比较是语法糖——编译器自动转换成两个比较的and。

C++

```
#include <iostream>
using namespace std;
int main() {
    double x;
    cin >> x;
    if (x >= 0 && x <= 25) cout << "Intervalo [0,25]" << endl;
    else if (x > 25 && x <= 50) cout << "Intervalo (25,50]" << endl;
    else if (x > 50 && x <= 75) cout << "Intervalo (50,75]" << endl;
    else if (x > 75 && x <= 100) cout << "Intervalo (75,100]" << endl;
    else cout << "Fora de intervalo" << endl;
    return 0;
}
```

Python

```
x = float(input())
if 0 <= x <= 25:
    print("Intervalo [0,25]")
elif 25 < x <= 50:
    print("Intervalo (25,50]")
elif 50 < x <= 75:
    print("Intervalo (50,75]")
elif 75 < x <= 100:
    print("Intervalo (75,100]")
else:
    print("Fora de intervalo")
```

NQ018 三角形 AcWing 664

"给你三根木棍, 长度分别是6.0、4.0和2.0——能拼成三角形吗?"小栋问。小鲁想了想:"三角形的条件是任意两边之和大于第三边。2+4=6, 等于第三边.....不能!"小华说:"必须同时满足 $A+B>C$ 、 $A+C>B$ 、 $B+C>A$ 三个条件, 用 $\&\&$ 连接。"

- 输入** 一行, 三个浮点数A、B、C。
- 输出** 能构成则"Perimetro = 周长", 否则"Area = 梯形面积"。

范围

0

输入

6.0 4.0 2.0

输出

Area = 10.0

思路 三角形判定： $A+B>C$ 且 $A+C>B$ 且 $B+C>A$ 。三个条件必须同时满足，用 $\&\&$ (C++)或 and (Python)连接。不能构成时输出梯形面积 $(A+B)*C/2$ 。

技巧 三个条件同时满足：if $a+b>c$ and $a+c>b$ and $b+c>a$ 。Python可以写得非常干净。注意浮点数比较的精度问题——对于大数或极端接近的情况需谨慎。

C++

```
#include <cstdio>
#include <iostream>
using namespace std;
int main() {
    double a, b, c;
    cin >> a >> b >> c;
    if (a + b > c && a + c > b && b + c > a)
        printf("Perimetro = %.1lf\n", a + b + c);
    else
        printf("Area = %.1lf\n", (a + b) * c / 2);
    return 0;
}
```

Python

```
a, b, c = map(float, input().split())
if a + b > c and a + c > b and b + c > a:
    print(f"Perimetro = {a + b + c:.1f}")
else:
    print(f"Area = {(a + b) * c / 2:.1f}")
```

NQ019 游戏时间 AcWing 667

小鲁玩游戏从下午4点玩到凌晨2点。"这跨天了，怎么算？"小华说：" $A=16, B=2$ 。如果A

输入

两个整数A和B ($0 \leq A, B \leq 23$)。

输出

"O JOGO DUROU X HORA(S)"。

范围

 $0 \leq A, B \leq 23$

输入

16 2

输出

0 JOGO DUROU 10 HORA(S)

思路 先判断是否跨天：A

技巧 可以统一为 $(B-A+24)\%24$ ，但结果为0时需要特殊处理（表示24小时）。这种"循环型"问题在时间/角度/周期问题中反复出现。

C++

```
#include <stdio>
int main() {
    int a, b;
    scanf("%d%d", &a, &b);
    int res;
    if (a < b) res = b - a;
    else res = b - a + 24;
    printf("0 JOGO DUROU %d HORA(S)\n", res);
    return 0;
}
```

Python

```
a, b = map(int, input().split())
if a < b:
    horas = b - a
else:
    horas = 24 - a + b
print(f"0 JOGO DUROU {horas} HORA(S)")
```

NQ020 加薪 AcWing 669

小鲁收到年终调薪通知——涨多少取决于当前工资：0–400涨15%，400.01–800涨12%，800.01–1200涨10%，1200.01–2000涨7%，2000以上涨4%。“这是分段函数！”小华说：“从低到高判断最自然—— $s \leq 400$? 15 : $s \leq 800$? 12 : $s \leq 1200$? 10 :...”

输入

一个浮点数表示当前工资。

输出

三行：新工资、涨薪金额、涨幅百分比。

范围

 $0 < \text{工资} \leq 10^6$

输入

400.00

输出

```
Novo salario: 460.00
Reajuste ganho: 60.00
Em percentual: 15 %
```

思路 从低到高的if-elif链。区间互斥，顺序很重要——先判断小值再判断大值可以不加&&上限条件（因为前面的判断已经排除了）。注意输出格式：保留2位小数+空格+%。

技巧 C++中%%输出%字样（printf需要转义）。Python直接写%。区间判断顺序从低到高可以利用else的“否则”语义简化条件。

C++

```
#include <cstdio>
int main() {
    double s;
    scanf("%lf", &s);
    int p;
    if (s <= 400) p = 15;
    else if (s <= 800) p = 12;
    else if (s <= 1200) p = 10;
    else if (s <= 2000) p = 7;
    else p = 4;
    double r = s * p / 100;
    printf("Novo salario: %.2lf\nReajuste ganho: %.2lf\nEm percentual: %d %%\n", s + r, r, p);
    return 0;
}
```

Python

```
s = float(input())
if s <= 400: p = 15
elif s <= 800: p = 12
elif s <= 1200: p = 10
elif s <= 2000: p = 7
else: p = 4
r = s * p / 100
print(f"Novo salario: {s + r:.2f}")
print(f"Reajuste ganho: {r:.2f}")
print(f"Em percentual: {p} %")
```

NQ021 动物 AcWing 670

"猜动物游戏！"小鲁兴奋地说。"输入三个特征——脊椎/非脊椎、哺乳/鸟/昆虫/环节动物、食性——判断是什么动物。"小华说："这是嵌套if的完美案例——像一棵决策树，每个内部节点是判断，叶子节点是答案。"

输入 三行字符串（vertebrado/invertebrado等）。

输出 动物名称（aguia/pomba/homem/vaca/pulga/lagarta/sanguessuga/minhoca）。

范围 输入保证合法。

输入

```
vertebrado
mamifero
onivoro
```

输出

```
homem
```

思路 三层嵌套if。先判断脊椎/非脊椎→再判断纲→再判断食性。本质是一棵深度为3的决策树。Python的嵌套if-elif结构清晰易读。也可用dict嵌套：tree={'vertebrado':{'ave':{'...}}}

技巧 Python的嵌套dict可以替代嵌套if——tree[a][b][c]一行定位答案。但写if逻辑更透明，两者都是合理的方案。"数据结构化思维"是编程进阶的关键能力。

C++

```
#include <iostream>
using namespace std;
int main() {
    string a, b, c;
    cin >> a >> b >> c;
    if (a == "vertebrado") {
        if (b == "ave") {
            if (c == "carnivoro") cout << "aguia" << endl;
            else cout << "pomba" << endl;
        } else {
            if (c == "onivoro") cout << "homem" << endl;
            else cout << "vaca" << endl;
        }
    } else {
        if (b == "inseto") {
            if (c == "hematofago") cout << "pulga" << endl;
            else cout << "lagarta" << endl;
        } else {
            if (c == "hematofago") cout << "sanguessuga" << endl;
            else cout << "minhoca" << endl;
        }
    }
    return 0;
}
```

Python

```
import sys
a, b, c = sys.stdin.read().split()
if a == "vertebrado":
    if b == "ave":
        print("aguia" if c == "carnivoro" else "pomba")
    else:
        print("homem" if c == "onivoro" else "vaca")
else:
    if b == "inseto":
        print("pulga" if c == "hematofago" else "lagarta")
    else:
        print("sanguessuga" if c == "hematofago" else "minhoca")
```

NQ022 选择练习1 AcWing 657

小鲁一口气写下五个条件：B>C、D>A、C+D>A+B、C>0、D>0、A是偶数。"要同时满足才能输出accepted——用&&(and)把它们串起来。"小华说："这是复杂布尔表达式的练习——五个条件，一个都不能少。"

输入

一行四个整数A、B、C、D。

输出

全部满足输出"Valores aceitos"，否则"Valores nao aceitos"。

范围

 $-10^9 \leq A, B, C, D \leq 10^9$

输入

5 6 7 8

输出

Valores nao aceitos

思路 5个条件用&&(and)连接。同时满足才输出accepted。训练复杂布尔表达式的使用。条件的排列顺序影响短路求值——先放最容易判断的。

技巧 Python把5个条件写一行：if b>c and d>a and c+d>a+b and c>0 and d>0 and a%2==0。可读性极好。先放C>0、D>0这种简单条件可以触发短路优化。

C++

```
#include <iostream>
using namespace std;
int main() {
    int a, b, c, d;
    cin >> a >> b >> c >> d;
    if (b > c && d > a && c + d > a + b && c > 0 && d > 0
        && a % 2 == 0)
        cout << "Valores aceitos" << endl;
    else
        cout << "Valores nao aceitos" << endl;
    return 0;
}
```

Python

```
a, b, c, d = map(int, input().split())
if b > c and d > a and c + d > a + b
and c > 0 and d > 0 and a % 2 == 0:
    print("Valores aceitos")
else:
    print("Valores nao aceitos")
```

NQ023 DDD AcWing 671

小鲁想给其他城市的同学打电话。"61是巴西利亚，71是萨尔瓦多，11是圣保罗……这么多城市怎么记？"小华说："C++可以写8个if-else。但Python用dict——{'61':'Brasilia','71':'Salvador',...}，一行get()全搞定。这就是'用数据驱动替代代码堆叠'的力量。"

输入

一个整数DDD区号。

输出

城市名或"DDD nao cadastrado"。

范围

DDD为正整数。

输入

11

输出

Sao Paulo

思路 Python的dict映射表一行替代8个if-else。ddd.get(ddd, 'DDD nao cadastrado')——查找存在就返回，不存在就返回默认值。这是本课最重要的工程思维：看到重复的if-else模式时，用数据结构替代。

技巧 ddd={'61':'Brasilia','71':'Salvador','11':'Sao Paulo','21':'Rio de Janeiro','32':'Juiz de Fora','19':'Campinas','27':'Vitoria','31':'Belo Horizonte'}。print(ddd.get(x,'DDD nao cadastrado'))。

C++

```
#include <iostream>
using namespace std;
int main() {
    int x;
    cin >> x;
    if (x == 61) cout << "Brasilia" << endl;
    else if (x == 71) cout << "Salvador" << endl;
    else if (x == 11) cout << "Sao Paulo" << endl;
    else if (x == 21) cout << "Rio de Janeiro" << endl;
    else if (x == 32) cout << "Juiz de Fora" << endl;
    else if (x == 19) cout << "Campinas" << endl;
    else if (x == 27) cout << "Vitoria" << endl;
    else if (x == 31) cout << "Belo Horizonte" << endl;
    else cout << "DDD nao cadastrado" << endl;
    return 0;
}
```

Python

```
ddd = {61:"Brasilia",71:"Salvador",11:"Sao Paulo",21:"Rio de Janeiro",32:"Juiz de Fora",19:"Campinas",27:"Vitoria",31:"Belo Horizonte"}
x = int(input())
print(ddd.get(x, "DDD nao cadastrado"))
```

NQ024 点的坐标 AcWing 662

小鲁在坐标系上画了一个点：P(4.5,-2.2)——这是第几象限？""第四象限！"小华说，"但要注意特殊情况：原点(0,0)、X轴(y=0)、Y轴(x=0)。判断顺序很重要——先判断原点，再判断轴，最后判断象限。如果顺序搞错——比如先判断x>0就归为Q1——可能把原点误判。"

输入

两个浮点数x和y。

输出

Q1/Q2/Q3/Q4/Origem/Eixo X/Eixo Y。

范围

 $-10^9 \leq x, y \leq 10^9$

输入

4.5 -2.2

输出

Q4

思路 判断顺序：原点→轴→象限。这是分类讨论的标准流程——先特殊后一般。 $x>0$ 且 $y>0 \rightarrow Q1$ ， $x<0$ 且 $y>0 \rightarrow Q2$ ， $x<0$ 且 $y<0 \rightarrow Q3$ ， $x>0$ 且 $y<0 \rightarrow Q4$ 。

技巧 判断顺序从特殊到一般：原点→轴→象限。Python中 $x==0$ 和 $y==0$ 用浮点比较在本题数据范围内安全。对于更精确的场景可使用 $\text{abs}(x)<1e-9$ 。

C++

```
#include <iostream>
using namespace std;
int main() {
    double x, y;
    cin >> x >> y;
    if (x > 0 && y > 0) cout << "Q1" << endl;
    else if (x < 0 && y > 0) cout << "Q2" << endl;
    else if (x < 0 && y < 0) cout << "Q3" << endl;
    else if (x > 0 && y < 0) cout << "Q4" << endl;
    else if (x == 0 && y == 0) cout << "Origem" << endl;
    else if (x == 0) cout << "Eixo Y" << endl;
    else cout << "Eixo X" << endl;
    return 0;
}
```

Python

```
x, y = map(float, input().split())
if x > 0 and y > 0: print("Q1")
elif x < 0 and y > 0: print("Q2")
elif x < 0 and y < 0: print("Q3")
elif x > 0 and y < 0: print("Q4")
elif x == 0 and y == 0: print("Origem")
elif x == 0: print("Eixo Y")
else: print("Eixo X")
```

NQ025 三角形类型 AcWing 666

"不仅要判定能不能构成三角形，还要分直/钝/锐/等边/等腰——这一道题等于前一道的10倍工作量！"小鲁被吓到了。小华笑道："别怕，先把三边降序排列 ($a \geq b \geq c$)，然后从一般到特殊逐个判断。排序降序是简化三角形分类的关键——排完序后一切都变简单了。"

输入

三个浮点数A、B、C。

输出

按顺序判断：不构成→直角→钝角→锐角→等边→等腰。可能输出多行。

范围

0

输入

7.0 5.0 7.0

输出

TRIANGULO ACUTANGULO
TRIANGULO ISOSCELES

思路 三步法：1.降序排列(swap直到 $a \geq b \geq c$)；2.判三角形： $a \geq b+c$?不能构成；3.角度类型： $a^2=b^2+c^2$ (直角)、 $a^2 > b^2+c^2$ (钝角)、 $a^2 < b^2+c^2$ (锐角)。

技巧 Python一行排序：`a,b,c=sorted([a,b,c],reverse=True)`。排序降序后所有判断条件都变得简单。注意用多个if而非if-elif——因为可能同时满足多个条件（如既是锐角又是等腰）。

C++

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    double a, b, c;
    cin >> a >> b >> c;
    if (b > a) swap(a, b);
    if (c > a) swap(a, c);
    if (c > b) swap(b, c);
    if (a >= b + c) cout << "NAO FORMA TRIANGULO" << endl;
    else {
        if (a*a == b*b + c*c) cout << "TRIANGULO RETANGULO" << endl;
        if (a*a > b*b + c*c) cout << "TRIANGULO OBTUSANGULO" << endl;
        if (a*a < b*b + c*c) cout << "TRIANGULO ACUTANGULO" << endl;
        if (a == b && b == c) cout << "TRIANGULO EQUILATERO" << endl;
        else if (a == b || a == c || b == c) cout << "TRIANGULO ISOSCELES" << endl;
    }
    return 0;
}
```

Python

```
a, b, c = sorted(map(float, input().split()), reverse=True)
if a >= b + c:
    print("NAO FORMA TRIANGULO")
else:
    if a*a == b*b + c*c: print("TRIANGULO RETANGULO")
    if a*a > b*b + c*c: print("TRIANGULO OBTUSANGULO")
    if a*a < b*b + c*c: print("TRIANGULO ACUTANGULO")
    if a == b == c: print("TRIANGULO EQUILATERO")
    elif a == b or b == c or a == c:
        print("TRIANGULO ISOSCELES")
```

NQ026 游戏时间2 AcWing 668

"上次只算了小时，这次要精确到分钟！"小鲁升级了游戏计时器。"起点7:08，终点9:10——持续多久？"统一换算成分钟：7*60+8=428，9*60+10=550，差122分钟=2小时2分钟。如果终点≤起点，加24*60处理跨天。"

输入 四个整数：A(开始时)、B(开始分)、C(结束时)、D(结束分)。

输出 "O JOGO DUROU X HORA(S) E Y MINUTO(S)"。

范围 0≤A,C≤23,0≤B,D≤59

输入

7 8 9 10

输出

0 JOGO DUROU 2 HORA(S) E 2 MINUTO(S)

思路 统一转换为分钟：start=A*60+B,end=C*60+D。若end≤start则加24*60。diff=end-start，再转回时=diff/60,分=diff%60。"统一量纲"是时间问题的通用技巧。

技巧 统一量纲(分钟)后比较和计算避免分别处理小时和分钟的复杂情况。Python的divmod(diff,60)可以直接分解时和分。

C++

```
#include <cstdio>
int main() {
    int a, b, c, d;
    scanf("%d%d%d%d", &a, &b, &c, &d);
    int start = a * 60 + b, end = c * 60 + d;
    if (end <= start) end += 24 * 60;
    int diff = end - start;
    printf("O JOGO DUROU %d HORA(S) E %d MINUTO(S)\n", diff / 60, diff % 60);
    return 0;
}
```

Python

```
a, b, c, d = map(int, input().split())
s = a * 60 + b
e = c * 60 + d
if e <= s: e += 24 * 60
d = e - s
print(f"O JOGO DUROU {d // 60} HORA(S) E {d % 60} MINUTO(S)")
```

NQ027 税 AcWing 672

小鲁第一次交税。"税率分段计算——0到2000免税，2000.01到3000收8%，3000.01到4500收18%，4500以上收28%。""注意是分段计税，不是全额×最高税率！"小华强调："每一段分别计算——超过2000的部分才需要交税。"

输入

一个浮点数表示月收入。

输出

免税输出"Isento"，否则"R\$ XX.XX"。

范围

0<收入≤10⁶

输入

3002.00

输出

R\$ 80.36

思路 分段计税：先判断是否≤2000(免税)。超出2000部分逐段按税率计算。Python用min()限制每段上限：
tax=min(x,1000)*0.08+min(max(x-1000,0),1500)*0.18+...

技巧 Python的min/max组合：min(x,1000)*0.08处理第一段(0-1000按8%)。理解"分段"原理比写出代码更重要——这是个人所得税的核心逻辑。

C++

```
#include <cstdio>
int main() {
    double x;
    scanf("%lf", &x);
    if (x <= 2000) { printf("Isento\n"); return 0; }
    x -= 2000;
    double t = 0;
    if (x > 0) { double v = x < 1000 ? x : 1000; t += v * 0.08; x -= v; }
    if (x > 0) { double v = x < 1500 ? x : 1500; t += v * 0.18; x -= v; }
    if (x > 0) t += x * 0.28;
    printf("R$ %.2lf\n", t);
    return 0;
}
```

Python

```
x = float(input())
if x <= 2000:
    print("Isento")
else:
    x -= 2000
    t = 0
    if x > 0:
        v = min(x, 1000)
        t += v * 0.08
        x -= v
    if x > 0:
        v = min(x, 1500)
        t += v * 0.18
        x -= v
    if x > 0:
        t += x * 0.28
    print(f"R$ {t:.2f}")
```

NQ028 简单排序 AcWing 663

"把三个数从小到大排列——这是排序算法的萌芽。"小华说："三步比较交换：先比较a和b，必要时交换；再比较a和c；最后比较b和c。三步之后 $a \leq b \leq c$ ——这就是冒泡排序思想的微型版。C++用swap(), Python用 $a, b = b, a$ ——交换变量最优雅的方式。"

输入

一行三个整数。

输出

升序排列的三个数。

范围

 $-10^9 \leq a, b, c \leq 10^9$

输入

7 21 -14

输出

-14 7 21

思路 三步交换排序：if $a > b$: swap(a,b); if $a > c$: swap(a,c); if $b > c$: swap(b,c)。这是排序算法的最小实现——理解交换操作的原理为后续学习排序算法打下基础。

技巧 Python的 $a, b = b, a$ 是最优雅的交流写法——基于元组解包。C++用std::swap或临时变量。Python一行sorted()也行但手写交换理解原理更重要。

C++

```
#include <iostream>
using namespace std;
int main() {
    int a, b, c;
    cin >> a >> b >> c;
    if (a > b) swap(a, b);
    if (a > c) swap(a, c);
    if (b > c) swap(b, c);
    cout << a << " " << b << " " << c << endl;
    return 0;
}
```

Python

```
a, b, c = map(int, input().split())
if a > b: a, b = b, a
if a > c: a, c = c, a
if b > c: b, c = c, b
print(a, b, c)
```

本章知识点总结

- **if-else/if-elif-else**：条件分支的基本语法，两语言核心差异在于Python的elif和缩进
- **Python链式比较**： $a \leq x \leq b$ 等价于 $a \leq x$ and $x \leq b$ ，语法糖但可读性极好
- **dict映射表**：替代长if-else链的利器。O(1)查找，代码减少50%+
- **决策树思维**：嵌套条件判断本质是决策树——从根到叶的路径
- **排序简化**：三角形类型题——先排序使 $a \geq b \geq c$ ，后续所有判断条件简化
- **分段计算**：税务题——min/max组合处理分段，理解"分段"而非"全额"

循环的魔力

第3章 · for/while与嵌套循环 · 14题 · C++ & Python 双语对照

第三周，小鲁发现了一个令他震惊的事实："一个for循环能替我做100行重复代码？！"小华笑了："这就是**循环**的威力——编程从手动执行进化到自动重复的时刻，就像从步行变成了骑车。能识别出重复模式并用循环解决，才是真正学会了编程思维。"

本章14道题，小鲁将在小华的指导下，从最简单的for循环开始，逐步掌握while循环、嵌套循环和条件循环。更重要的是，学会**"什么时候用循环，什么时候用公式"**——循环是工具，不是信仰。当数学公式能一行搞定1000次循环时，数学和编程的结合才真正开始。

本章角色

-  **小鲁**（初学者）· 从"手动写100行"的困惑中觉醒，学习循环的力量
-  **小华**（算法高手）· ACM集训队队长，能用最生动的比喻解释while vs for
-  **小栋**（工程师）· 喜欢用实际问题挑战小鲁，代表真实编程场景
-  **小嘉**（校主精神）· 偶尔现身，用"自强不息"激励大家在算法之路上坚持

C++ vs Python 循环速查

- **for循环**: `for(int i=0;i`
- **步长**: `i+=2` → `range(0,N,2)` (注意range的end不包含!)
- **while循环**: `while(cond){...}` → `while cond:...` (几乎一样)
- **不定长输入**: `while(cin>>x)` → `while True: x=input(); if not x: break`
- **嵌套循环**: 两层for在两种语言中语法不同、逻辑相同
- **Python独有**: 列表推导式一行替代循环+条件+赋值。生成器表达式处理大数据无内存压力

NQ029 偶数 AcWing 708

"我想打印出2到100之间的所有偶数，难道要写49行cout？！"小鲁被这重复劳动吓到了。小华指了指屏幕："看到这个模式了吗？2,4,6,8...100——每个数比前一个大2。这种重复操作正是循环的用武之地。一行for循环，49行的活一秒搞定——这就是为什么程序员永远不会被机器取代：因为我们知道什么时候让机器替我们干活。"

- 输入** 无。
- 输出** 每行一个偶数，从2到100。
- 范围** 无

输入
(无输入)

输出

```

2
4
6
...
100

```

思路 for循环的基础形态：设定起始值、终止条件、步长。C++: `for(int i=2;i<=100;i+=2)`, Python: `for i in range(2,101,2)`。注意range的end是不包含的，所以写101而非100。

技巧 Python的range(start, end, step)中end是exclusive。这是Python初学者最常见的off-by-one错误来源。`i+=2`比*i=i+2*更简洁——在所有类C语言中都通用。

C++

```

#include <iostream>
using namespace std;
int main() {
    for (int i = 2; i <= 100; i += 2)
        cout << i << endl;
    return 0;
}

```

Python

```

for i in range(2, 101, 2):
    print(i)

```

NQ030 奇数 AcWing 709

"偶数搞定了，那奇数呢？"小鲁问。小华写下了：`for i in range(1, X+1, 2)`。"你看，只需改两个数——起始值从2改成1，步长保持2。奇偶的区别只差一个起始位置。"小鲁恍然大悟："所以循环的威力就在于——一个模式能生成无穷多个结果。从1到X的所有奇数，只需要一行range！"

输入

一个正整数X。

输出

从1到X的所有奇数，每行一个。

范围

 $1 \leq X \leq 1000$

输入

8

输出

```

1
3
5
7

```

思路 奇数序列：步长固定为2，起始值决定是奇数(1)还是偶数(2)。range(1, X+1, 2)自动跳过偶数。注意上限是X（包含），所以写X+1。

技巧 Python的range不会包含end值——这是与很多其他语言不同的地方。记住这个特征：range(a,b)实际遍历[a, b-1]。

C++

```
#include <iostream>
using namespace std;
int main() {
    int x;
    cin >> x;
    for (int i = 1; i <= x; i += 2)
        cout << i << endl;
    return 0;
}
```

Python

```
x = int(input())
for i in range(1, x + 1, 2):
    print(i)
```

NQ031 正数 AcWing 712

小鲁在做班级的成绩统计："6个同学的成绩里，有几个是正数？"这需要循环+条件计数。"小华在屏幕上写了起来，"遍历每个数，如果大于0就让计数器加1。关键是理解'计数器'的概念：int cnt=0; 每满足条件就cnt++。Python更简洁——sum(1 for x in arr if x>0)。"

输入

6行，每行一个浮点数。

输出

"X positive numbers"，其中X为正数个数。

范围

 $-10^9 \leq \text{值} \leq 10^9$

输入

```
7
-5
6
-3.4
4.6
12
```

输出

4 positive numbers

思路 条件计数模式：遍历→判断→累加计数器。计数器初始化为0，每次满足条件+1。Python的sum()配合生成器表达式一行搞定。C++中浮点数与0比较注意精度。

技巧 Python生成器表达式sum(1 for x in arr if x>0)以O(n)时间和O(1)空间优雅地完成计数。C++的条件判断中整数和浮点数都适用>0。

C++

```
#include <iostream>
using namespace std;
int main() {
    int cnt = 0;
    for (int i = 0; i < 6; i++) {
        double x; cin >> x;
        if (x > 0) cnt++;
    }
    cout << cnt << " positive numbers" << endl;
    return 0;
}
```

Python

```
import sys
nums = list(map(float, sys.stdin.read().split()))
cnt = sum(1 for x in nums if x > 0)
print(f"{cnt} positive numbers")
```

NQ032 连续奇数的和1 AcWing 714

"设计一个有趣的数学游戏："小华说，"随便选两个整数X和Y，然后求X和Y之间所有奇数的和。比如2和15之间的奇数是3+5+7+9+11+13+15=63。"小鲁眼睛一亮："关键是要先确定X和Y谁大谁小，再从小的遍历到大的。""没错——编程第一步总是在处理边界。"

- 输入** 一行两个整数X和Y。
- 输出** 一个整数，X和Y之间所有奇数的和。
- 范围** $-10^6 \leq X, Y \leq 10^6$

输入	输出
2 15	63

思路 三步：1.确保起点 $\leq$ 终点（交换）；2.从起点到终点遍历每个数；3.如果是奇数则累加。Python的`sum(i for i in range(lo,hi+1) if i%2==1)`一行完成。C++需要手写for+if+累加。

技巧 Python的`range(lo,hi+1)[lo%2::2]`可以只生成奇数——利用切片+步长直接跳过偶数。但生成器表达式更可读。先确定边界再遍历是处理区间问题的通用习惯。

C++

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    int x, y, sum = 0;
    cin >> x >> y;
    if (x > y) swap(x, y);
    for (int i = x + 1; i < y; i++)
        if (i % 2) sum += i;
    cout << sum << endl;
    return 0;
}
```

Python

```
x, y = map(int, input().split())
if x > y: x, y = y, x
print(sum(i for i in range(x + 1, y)
if i % 2))
```

NQ033 最大数和它的位置 AcWing 716

小栋拿着一叠数据找到小鲁："帮我找这100个数里的最大值，还有第几个是它。"小鲁试了`max()`和`index()`："搞定了！"小华看了代码后说："不错，但你知道吗——你用了两遍遍历。更好的方法是遍历一次，同时记住最大值和位置。虽然对于100个数没区别，但如果是一亿个数，差一倍的时间就是差一个午饭。"

- 输入** 100行，每行一个整数。
- 输出** 第一行最大值，第二行位置（1-indexed）。
- 范围** $-10^9 \leq \text{值} \leq 10^9$

输入

2
113
45
34565
6
...

输出

34565
4

思路 一次遍历找最值：初设第一个为最大值位置1，然后从第二个开始，遇到更大的就更新值和位置。O(n)一次遍历完成。这比Python的max()+index()两次遍历更高效——算法优化的意识从小题培养。

技巧 Python的max()底层也是C实现的O(n)。但max()+index()会导致两次O(n)遍历。手写的O(n)单次遍历在数据量大时更优。这种"一次遍历完成多件事"的思维是算法设计的核心。

C++

```
#include <iostream>
using namespace std;
int main() {
    int n, x, mx, pos = 1;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> x;
        if (i == 1 || x > mx) { mx = x; pos = i; }
    }
    cout << mx << endl << pos << endl;
    return 0;
}
```

Python

```
import sys
data = list(map(int, sys.stdin.read().split()))
n = data[0]
vals = data[1:n+1]
mx = max(vals)
pos = vals.index(mx) + 1
print(mx)
print(pos)
```

NQ034 递增序列 AcWing 721

"有一个特殊的数据集——每个数都比前一个数大，形成一个递增序列。"小华在纸上画着："5,10,15,20,25...我们需要一种新的循环——不知道要循环多少次，因为不知道序列有多长。""那就是while循环！"小鲁抢答，"while True读入，遇到0就break——这就是'不定长输入'的处理模式。"

输入

多组，每组一个整数X。0结束。

输出

对每个非0的X，输出从1到X的所有整数。

范围

$0 \leq X \leq 10^6$

输入

5
10
3
0

输出

1 2 3 4 5
1 2 3 4 5 6 7 8 9 10
1 2 3

思路 while+break是不定长输入的标准处理模式：while(1){读入; if(==0) break; 处理}。Python的while True和C++的while(cin>>x,x)都支持。while和for的区别在于：while适合"不知道次数"的循环。

技巧 Python: while True: x=int(input()); if x==0: break; 输出序列。C++可以用逗号表达式while(cin>>x,x)——利用逗号运算符取最后一个值。但可读性不如先读入再break。

C++

```
#include <iostream>
using namespace std;
int main() {
    int x;
    while (cin >> x, x) {
        for (int i = 1; i <= x; i++) {
            if (i > 1) cout << " ";
            cout << i;
        }
        cout << endl;
    }
    return 0;
}
```

Python

```
while True:
    x = int(input())
    if x == 0: break
    print(' '.join(str(i) for i in range(1, x + 1)))
```

NQ035 连续整数相加 AcWing 720

小鲁遇到了变种题："读入A和N，求从A开始的连续N个整数的和。"比如A=3,N=2 => 3+4=7。"这跟NQ032有点像——但这次N告诉你加几个数。"小华提示道："在循环中用一个累加器sum，循环N次，每次加上当前值然后当前值加1。Python的range(a, a+n)直接生成这一串数。"

输入 多对整数A和N，一直读到N≤0。

输出 每对输出"Sum = X"。

范围 A,N ≤ 10⁶

输入

3 2
4 -1

输出

Sum = 7

思路 连续N个整数从A+A+1+...+A+N-1 = N×A+N(N-1)/2。但循环版本更易懂：sum=0; for i in range(a, a+n): sum+=i。也可以数学公式直接算——但理解循环累加的思维更重要。

技巧 Python: sum(range(a, a+n))一行完成。但手写循环累加才能理解"累加器模式"——这是所有求和、求积、统计类问题的基础。数学公式法体现了"先思考再编码"——但不要忘了循环是理解问题的第一步。

C++

```
#include <iostream>
using namespace std;
int main() {
    int a, n, sum = 0;
    cin >> a;
    while (cin >> n, n <= 0);
    for (int i = 0; i < n; i++) sum += a++;
    cout << sum << endl;
    return 0;
}
```

Python

```
a = int(input())
while True:
    for n in map(int, input().split()):
        if n > 0: break
    if n > 0: break
    print(sum(a + i for i in range(n)))
```

NQ036 约数 AcWing 724

小鲁在数学课上学到了约数：“6能被1,2,3,6整除——所以6有4个约数。”小华说：“编程找约数很简单：从1到N遍历每个数，如果 $N \% i == 0$ 就输出。不过有个优化技巧——约数是成对出现的，你只需要遍历到 $\sqrt{N}$ 。比如6的约数：i=1找到1和6，i=2找到2和3—— $\sqrt{6} \approx 2.45$ 所以搜索到2就够了。”

输入

一个整数N。

输出

按升序输出N的所有约数，每行一个。

范围

 $1 \leq N \leq 10^9$

输入

6

输出

1
2
3
6

思路 基础方法：从1到N遍历， $N \% i == 0$ 则输出。优化方法：只遍历到 $\sqrt{N}$ ，每个i找到两个约数i和 N/i 。注意完全平方数（如16）时 $i=N/i$ 只算一次。 $\sqrt{N}$ 优化将复杂度从 $O(N)$ 降到 $O(\sqrt{N})$ ——对于 $N=10^9$ ，从10亿降到3万。

技巧 C++的遍历条件 $i*i \leq N$ 比 $i \leq \sqrt{N}$ 更高效（避免浮点数）。Python用while $i*i \leq n$ 。注意输出顺序——如果要求升序，需要收集约数再排序输出。本课主要理解基本的枚举模式。

C++

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++)
        if (n % i == 0) cout << i << endl;
    return 0;
}
```

Python

```
n = int(input())
for i in range(1, n + 1):
    if n % i == 0:
        print(i)
```

NQ037 PUM AcWing 723

小鲁看到了一道来自巴西OJ的题目："输出N行，每行M个数——但每行最后一个数要替换成'PUM'。比如N=7,M=4....."小华笑道："这其实是考你对换行的精确控制——第M列不是数字而是'PUM'。关键是判断位置：如果 $(j+1)\%M==0$ 就输出PUM并换行，否则输出数字后跟空格。"

输入 一行两个整数N和M。

输出 N×M格式的序列，每行第M列输出"PUM"。

范围 $1 \leq N, M \leq 20$

输入	输出
7 4	1 2 3 PUM
	5 6 7 PUM
	9 10 11 PUM
	...

思路 计数器从1开始，每输出一个数+1。判断是否第M列：用 $\text{cnt}\%M==0$ （cnt从1开始）。是则输出PUM换行，否则输出数字+空格。这是格式化循环输出题——训练对循环索引和输出格式的精确控制。

技巧 Python: for i in range(n): 输出一行m-1个数字(空格分隔)+' PUM'。C++注意末尾空格的处理。理解%取模判断"是否该换行"是这类题的通用技巧。

C++

```
#include <iostream>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n * m; i++) {
        if (i % m == 0) cout << "PUM" << endl;
        else cout << i << " ";
    }
    return 0;
}
```

Python

```
n, m = map(int, input().split())
for i in range(1, n * m + 1):
    if i % m == 0:
        print("PUM")
    else:
        print(i, end=' ')
```

NQ038 六个奇数 AcWing 710

"给定一个整数X，输出从X开始的6个连续的奇数。"小栋出了一道题考小鲁。"但是我怎么知道X是奇数还是偶数？"小鲁问。小华提醒道："如果X是偶数，就从X+1开始——因为偶数+1就是奇数。然后用循环输出6个，每次步长2。关键是先处理奇偶性。"

输入 一个整数X。

输出 从X开始（如果X是偶数则从X+1开始）的6个连续奇数。

范围 $-10^9 \leq X \leq 10^9$

输入	输出
8	9
	11
	13
	15
	17
	19

思路 先归一化起点：if $x \% 2 == 0$: $start = x + 1$ else $start = x$ 。然后用for循环6次，每次输出 $start + 2 * i$ 。这是"预处理起点"的标准模式——先统一格式，再统一处理。

技巧 Python一行： $start = x + 1$ if $x \% 2 == 0$ else x ，然后for i in range(6): print(start + 2*i)。C++可以用 $x \% 2 == 0 ? x + 1 : x$ 做起点。这种"先定点再循环"的模式在循环题中反复出现。

C++

```
#include <iostream>
using namespace std;
int main() {
    int x;
    cin >> x;
    if (x % 2 == 0) x++;
    for (int i = 0; i < 6; i++, x += 2)
        cout << x << endl;
    return 0;
}
```

Python

```
x = int(input())
if x % 2 == 0:
    x += 1
for _ in range(6):
    print(x)
    x += 2
```

NQ039 乘法表 AcWing 711

"九九乘法表——这是我奶奶都会背的东西！"小鲁笑道。"但用代码生成它，需要你在一个循环里再套一个循环——这就是嵌套循环。"小华打开TRADE演示，"外层控制行，内层控制列。第*i*行第*j*列输出*i*j*。两层for循环，一共 $N \times N$ 次操作。注意格式对齐——每个数占4个字符宽度。"

输入 一个整数N（表示表格大小）。

输出 $N \times N$ 的乘法表，每个数占4宽对齐。

范围 $1 \leq N \leq 15$

输入	输出		
3	1	2	3
	2	4	6
	3	6	9

思路 嵌套循环的标准结构：for i in range(1,N+1): for j in range(1,N+1): print($i*j$)。内层循环完整执行N次后外层才进入下一次。注意格式化宽度——Python的f' $\{i*j:4d\}$ '。嵌套循环是程序设计的第一个心智跃迁。

技巧 Python的f'{val:4d}'表示4宽度右对齐。C++用setw(4)或printf("%4d")。嵌套循环复杂度 $O(N^2)$ ——当N翻倍时运算量翻4倍。这是理解"时间复杂度"的入门。

C++

```
#include <cstdio>
int main() {
    int n;
    scanf("%d", &n);
    for (int i = 1; i <= 10; i++)
        printf("%d x %d = %d\n", i, n, i * n);
    return 0;
}
```

Python

```
import sys
n = int(sys.stdin.read().strip())
for i in range(1, 11):
    print(f"{i} x {n} = {i * n}")
```

NQ040 实验 AcWing 718

小鲁的生物课在做动物实验：统计N组数据中青蛙、老鼠和兔子的总数及占比。"每组数据有两个数：数量和动物类型——C代表rabbit、R代表rat、S代表frog。""这不就是分类统计吗？"小华说，"用三个计数器分别统计，最后算百分比。Python可以用collections.Counter自动分类，但手写三个if分支更直观。"

输入 第一行N。接下来N行，每行"数量 类型(C/R/S)"。

输出 实验动物总数+各类数量+百分比。

范围 $1 \leq N \leq 100$

输入

3
5 C
7 R
8 S

输出

Total: 20 animais
Total de coelhos: 5
Total de ratos: 7
Total de sapos: 8
Percentual de coelhos: 25.00 %
Percentual de ratos: 35.00 %
Percentual de sapos: 40.00 %

思路 分类统计：three counters。每次根据类型字符判断加到哪个计数器。最后算百分比=各类/总数×100。注意保留两位小数。Python的if-elif或dict映射都比C++的if-else更简洁。

技巧 Python用dict映射：cnt={'C':0,'R':0,'S':0}，然后cnt[t]+=qty。比三个if更灵活。百分比输出用f"{cnt['C']/total*100:.2f} %"。注意%号的转义——f-string中用%%。

C++

```
#include <cstdio>
#include <iostream>
using namespace std;
int main() {
    int n, c = 0, r = 0, f = 0;
    cin >> n;
    for (int i = 0; i < n; i++) {
        int k; char t;
        cin >> k >> t;
        if (t == 'C') c += k;
        else if (t == 'R') r += k;
        else f += k;
    }
    int s = c + r + f;
    printf("Total: %d animals\n", s);
    printf("Total coneys: %d\n", c);
    printf("Total rats: %d\n", r);
    printf("Total frogs: %d\n", f);
    printf("Percentage of coneys: %.2lf %%\n", 100.0 * c / s);
    printf("Percentage of rats: %.2lf %%\n", 100.0 * r / s);
    printf("Percentage of frogs: %.2lf %%\n", 100.0 * f / s);
    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
n = int(data[0])
c = r = f = 0
idx = 1
for _ in range(n):
    k = int(data[idx]); t = data[idx + 1]
    idx += 2
    if t == 'C': c += k
    elif t == 'R': r += k
    else: f += k
s = c + r + f
print(f"Total: {s} animals")
print(f"Total coneys: {c}")
print(f"Total rats: {r}")
print(f"Total frogs: {f}")
print(f"Percentage of coneys: {c / s * 100:.2f} %")
print(f"Percentage of rats: {r / s * 100:.2f} %")
print(f"Percentage of frogs: {f / s * 100:.2f} %")
```

NQ041 余数 AcWing 715

小鲁在整理数据时发现了一个规律：“这组数里，除了2以外，其他被2整除的都不对……”小华纠正道：“你说的是‘除了2以外的所有整数，除以2的余数都是0或1’对吧？这道题就是数一数从1到10000有多少个数除以N余2。”小小的循环条件就能精准筛选。”

输入

一个整数N。

输出

从1到10000中除以N余2的所有数，每行一个。

范围

 $1 \leq N \leq 10000$

输入

13

输出

2

15

28

...

思路 条件循环：for i in range(1,10001): if i%N==2: print(i)。取模运算%是循环中最高频的条件运算符——余数判断可用于奇偶(i%2)、整除(i%N==0)、特定余数(i%N==k)等。

技巧 if i%N==2表示i除以N的余数等于2。比如N=13时， $2 \div 13 = 0$ 余2， $15 \div 13 = 1$ 余2， $28 \div 13 = 2$ 余2……余数是最基础的数论概念之一——理解它，后续的哈希、质数等问题才不会被拦在门外。

C++

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= 10000; i++)
        if (i % n == 2) cout << i << endl;
    return 0;
}
```

Python

```
n = int(input())
for i in range(1, 10001):
    if i % n == 2:
        print(i)
```

NQ042 区间2 AcWing 713

"上次我们数了正数有多少个——现在升级一下，统计位于[10,20]区间内的个数，以及区间外的个数。"小栋把题目升级了。小鲁自信满满："这简单！读入N个数，如果x在10到20之间（包含两端），in_counter++；否则out_counter++。一道题，两个计数器，一次遍历。"

输入

第一行N。接下来N个整数。

输出

"X in"换行"Y out"。

范围

$1 \leq N \leq 10000$

输入

4
14
123
10
-25

输出

2 in
2 out

思路 双计数器+区间判断：in_cnt统计 $10 \leq x \leq 20$ ，out_cnt统计x其他。一次遍历完成两个统计任务。Python用链式比较 $10 \leq x \leq 20$ 最直观。注意包含端点——10和20都算in。

技巧 Python链式比较：if $10 \leq x \leq 20$ 。比C++的 $x \geq 10 \ \&\& \ x \leq 20$ 更自然。两个计数器互斥且完备——in+out = N，可以只统一个数推另一个，但分别统计更安全（验证用的N个都处理了）。

C++

```
#include <iostream>
using namespace std;
int main() {
    int n, x, in = 0, out = 0;
    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> x;
        if (x >= 10 && x <= 20) in++;
        else out++;
    }
    cout << in << " in" << endl;
    cout << out << " out" << endl;
    return 0;
}
```

Python

```
n = int(input())
in_cnt = sum(1 for _ in range(n) if 1
0 <= int(input()) <= 20)
print(f"{in_cnt} in")
print(f"{n - in_cnt} out")
```

本章知识点总结

- **for循环**：确定次数的循环首选。Python的range(start,end,step)注意end不包含
- **while循环**：不确定次数的循环首选。while+break处理不定长输入
- **嵌套循环**：外层行内层列，复杂度 $O(N^2)$ 。理解两次for的执行顺序
- **条件循环**：循环中嵌入if判断——计数、筛选、分类，90%的算法代码都是这个模式
- **累加器模式**：sum/cnt初始化为0，循环中逐次更新。这是所有统计/求和的基础
- **先思考再编码**：约数的 $\sqrt{N}$ 优化、等差数列求和公式——数学知识直接减少循环次数

数据的容器

第4章 · 数组与线性存储 · 12题 · C++ & Python 双语对照

小鲁最近发现一个难题：他想统计全班50个同学的成绩，难不成要写50个变量？"这也太麻烦了吧！"他抱怨道。小华走了过来："你需要的是**数组**——编程中最基本的数据容器。"

本章12道题，小鲁将在小华（数学天才）和小栋（工程师）的帮助下，从基础的一维数组操作，逐步掌握二维矩阵、递推算法和算法优化。每一题都是一个故事——编程不应该是枯燥的，它应该像厦大芙蓉湖畔的凤凰花一样美丽。

本章角色

-  **小鲁**（初学者）· 代表正在学习编程的你，会犯错、会提问、会进步
-  **小华**（算法高手）· 数学天才，ACM竞赛队成员，总能用简单的话解释复杂算法
-  **小栋**（工程师）· 喜欢用编程解决实际问题，数据分析爱好者
-  **小嘉**（校主精神）· 厦大创始人，偶尔现身，带来数学智慧与校训力量

NQ043 数组替换 AcWing 737

小鲁正在写一个数据处理程序，但发现数据里混入了一些负数和零——这些"脏数据"会影响后面的计算。"得想个办法，把不合格的数据统一替换成1....."小鲁嘀咕着。小华凑过来看了一眼屏幕："这不就是数组遍历+条件判断吗？遍历每个元素，如果 ≤ 0 就改成1，一行代码的事。"

输入 10行，每行一个整数。

输出 10行，每行" $X[i] = Y$ "。

范围 $-10^9 \leq \text{值} \leq 10^9$

输入

```
0
-5
63
-8
0
1
2
-100
50
200
```

输出

```
X[0] = 1
X[1] = 1
X[2] = 63
...
```

思路 这是数组最基础的操作模式：读入→判断→输出。遍历每个元素时检查其值，如果 ≤ 0 就替换为1。C++中用for(int i=0;i<10;i++)访问，Python用enumerate()同时获取索引和值。

技巧 Python的enumerate(arr)返回(index, value)元组。C++中数组索引从0开始——这是初学者最常犯的off-by-one错误来源。记住：第0个位置是第一个元素。

C++

```
#include <iostream>
using namespace std;
int main() {
    int x;
    for (int i = 0; i < 10; i++) {
        cin >> x;
        if (x <= 0) x = 1;
        cout << "X[" << i << "] = " << x << endl;
    }
    return 0;
}
```

Python

```
for i in range(10):
    x = int(input())
    if x <= 0: x = 1
    print(f"X[{i}] = {x}")
```

NQ044 数组填充 AcWing 738

小华在教小鲁数组的递推生成："你看，这不就像我们小时候学过的'翻倍'游戏吗？第0个是V，第1个是V×2，第2个是(V×2)×2..."小鲁恍然大悟："原来数组可以这样自动生成！不用我一个一个敲数字了。"

输入 一个整数V。

输出 10行，每行"N[i] = X"。

范围 $-10^4 \leq V \leq 10^4$

输入

1

输出

```
N[0] = 1
N[1] = 2
N[2] = 4
N[3] = 8
...
```

思路 递推生成：每个新值由前一个值×2得出。这是"递推思想"的入门——每个新结果基于之前的结果计算，不需要从头算起。Python列表推导：`[v*(2**i) for i in range(10)]`一行搞定。

技巧 Python的一行列表推导：`arr = [v * (2**i) for i in range(10)]`。C++中i从0开始，v每次乘2。注意V为负数时序列呈负增长。

C++

```
#include <iostream>
using namespace std;
int main() {
    int v;
    cin >> v;
    for (int i = 0; i < 10; i++, v *= 2)
        cout << "N[" << i << "] = " << v << endl;
    return 0;
}
```

Python

```
v = int(input())
for i in range(10):
    print(f"N[{i}] = {v}")
    v *= 2
```

NQ045 数组选择 AcWing 739

小鲁看着屏幕上的100个数据犯了愁："这么多数据，我怎么能快速找出哪些小于等于10的？"小华笑道："这不就是'筛选'吗？Python一行代码就能搞定——列表推导或者边读边判断。记住，你不需要把所有数据都存下来，读一个判断一个就行。"

- 输入** 100行（或一行多个），每行一个浮点数。
- 输出** 对每个 ≤ 10 的数，输出"A[i] = X"（保留1位小数）。
- 范围** $-10^6 \leq \text{值} \leq 10^6$

输入

```
0
-5
63
8.5
100
...
```

输出

```
A[0] = 0.0
A[1] = -5.0
A[3] = 8.5
...
```

思路 流式处理：不需要把100个数都存下来，读一个判断一个输出一个。这种"边读边处理"的思维在处理海量数据时至关重要——内存是有限的，流式处理可以处理任意大小的数据。

技巧 Python格式化f"A[{i}] = {x:.1f}"。C++用printf("A[%d] = %.1f\n", i, x)。条件用 $\leq$ ，注意浮点精度。

C++

```
#include <cstdio>
int main() {
    for (int i = 0; i < 100; i++) {
        double x;
        scanf("%lf", &x);
        if (x <= 10) printf("A[%d] = %.1f\n", i, x);
    }
    return 0;
}
```

Python

```
for i in range(100):
    x = float(input())
    if x <= 10:
        print(f"A[{i}] = {x:.1f}")
```

NQ046 数组中的行 AcWing 743

小鲁在整理12个月的销售数据，每个月的销售额存在不同的行里。"我想单独看某个月的总销售额....."小华点点头："这就是二维数组的行操作。外层循环控制行，内层循环控制列，当行号匹配时累加。"

输入 第一行L(0~11)。第二行T('S'或'M')。接下来144个浮点数 (12×12矩阵)。

输出 结果保留1位小数。

范围 矩阵元素 -10^6 到 10^6

输入

2
S
(144个浮点数...)

输出

(第2行之和)

思路 双层循环是二维数组的标准遍历方式。外层for i in range(12)控制行，内层for j in range(12)控制列。当i==L时累加。这种行列遍历模式贯穿所有多维数组题目。

技巧 Python嵌套列表：[[0]*12 for _ in range(12)]。注意不能用[[0]*12]*12——那是浅拷贝，所有行指向同一个列表！

C++

```
#include <cstdio>
int main() {
    int l; char t;
    scanf("%d %c", &l, &t);
    double sum = 0, x;
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++) {
            scanf("%lf", &x);
            if (i == l) sum += x;
        }
    printf("%.1lf\n", t == 'S' ? sum : sum / 12);
    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
l = int(data[0])
t = data[1]
s = 0.0
idx = 2
for i in range(12):
    for j in range(12):
        x = float(data[idx])
        idx += 1
        if i == l:
            s += x
print(f"{s if t == 'S' else s / 12:.1f}")
```

NQ047 数组变换 AcWing 740

小鲁想把20个数据"倒过来看"——第一个放最后，最后一个放最前。"这不就是翻转吗？"小华说，"Python用arr[::-1]一行搞定，C++要手写交换循环。但理解算法比会用库更重要——你知道翻转的原理是什么吗？"

输入 20个整数。

输出 20行，每行"N[i] = X"。

范围 $-10^9 \leq \text{值} \leq 10^9$

输入

```
0
1
2
...
19
```

输出

```
N[0] = 19
N[1] = 18
...
```

思路 对称交换算法：arr[i]与arr[19-i]交换，只需循环10次（i从0到9）。每步交换两个对称位置的元素。这是理解“手写翻转”和“调用库函数”差异的好例子。

技巧 Python翻转用arr[::-1]（切片反转）或arr.reverse()（原地反转）。C++用std::reverse()或手写swap。切片创建新列表消耗额外内存。

C++

```
#include <iostream>
using namespace std;
int main() {
    int arr[20];
    for (int i = 0; i < 20; i++) cin >> arr[i];
    for (int i = 0; i < 10; i++) swap(arr[i], arr[19 - i]);
    for (int i = 0; i < 20; i++)
        cout << "N[" << i << "] = " << arr[i] << endl;
    return 0;
}
```

Python

```
arr = [int(input()) for _ in range(20)]
arr.reverse()
for i, v in enumerate(arr):
    print(f"N[{i}] = {v}")
```

NQ048 斐波那契数列 AcWing 741

小鲁在图书上看到了斐波那契数列：0,1,1,2,3,5,8,13... "这个数列有什么神奇的吗？"小华兴奋地说："太多了！兔子的繁殖、花瓣的排列、黄金分割.....在编程里，它是最经典的递推题——每个数都是前两个数的和，这就是动态规划(DP)的萌芽！""不过要注意，第60项已经超过20亿了，C++要用long long。"

输入

一个整数N。

输出

一行，空格隔开的N个整数。

范围

 $1 \leq N \leq 60$

输入

5

输出

0 1 1 2 3

思路 递推标准模板：a=0,b=1;不断输出a，然后a,b=b,a+b。Python的a,b=b,a+b是最优雅的递推写法。C++必须用long long（64位），N=60时F(60)≈ 1.5×10^{12} 远超出int范围。

技巧 C++千万用long long！int在N>46时就会溢出。Python自动大整数。a,b=b,a+b是Python递推的“指纹”——看到这行代码就知道在做递推。

C++

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    long long a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        if (i) cout << " ";
        cout << a;
        long long t = a + b; a = b; b = t;
    }
    cout << endl;
    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
n = int(data[0]) if data else 0
a, b = 0, 1
for _ in range(n):
    sys.stdout.write(str(a) + (' ' if
_ < n-1 else '\n'))
    a, b = b, a + b
```

NQ049 最小数和它的位置 AcWing 742

小栋找小鲁帮忙："我在做数据分析，找一组测量数据中的最小值以及它出现的位置。"小鲁打开TRAE，写下了min()和index()。小华看了说："不错，但你知道吗——你这实际上遍历了两遍数据。更好的方法是只遍历一次，同时记录最小值和位置。"

输入

第一行N。第二行N个整数。

输出

第一行"Menor valor: X"。第二行"Posicao: Y"（从0开始）。

范围

$1 \leq N \leq 1000$

输入

10
1 2 3 4 -5 6 7 8 9 10

输出

Menor valor: -5
Posicao: 4

思路 一次遍历同时找最小值和位置：初始设第一个值为最小值，后续每遇到更小的就更新。Python用min()+index()简洁但需要两次遍历。手写版本只需一次O(n)——理解算法效率的差异。

技巧 Python的min(arr)和arr.index(mn)组合虽简洁但效率不是最优。手写版本通过单次遍历实现O(n)时间+O(1)空间，是算法优化的入门案例。

C++

```
#include <iostream>
using namespace std;
int main() {
    int n, x, mn, pos = 0;
    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> x;
        if (i == 0 || x < mn) { mn = x; pos = i; }
    }
    cout << "Menor valor: " << mn << endl;
    cout << "Posicao: " << pos << endl;
    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
n = int(data[0])
a = list(map(int, data[1:1+n]))
mn = min(a)
pos = a.index(mn)
print(f"Menor valor: {mn}")
print(f"Posicao: {pos}")
```

NQ050 数组中的列 AcWing 744

"刚才你帮我算了某行的总和，现在我想算某列的总和——比如第3列，代表3月份各产品的销售额。"小栋说。小鲁看了一下代码："噢，这不就是把 $i=L$ 改成 $j=C$ 吗？其他都一样。"小华赞许地点头："这就是编程的对称思维——理解了关系表达式，行和列的操作本质上是对称的。"

输入 第一行列号C。第二行操作类型T。接下来144个浮点数。

输出 结果保留1位小数。

范围 同NQ046

输入

2
S
(144个浮点数...)

输出

(第2列之和)

思路 与行操作对称：判断条件从 $i=L$ 改为 $j=C$ 。这种对称性是理解多维数组遍历的关键——能灵活变换遍历条件才是真正的掌握。

技巧 对比NQ046：代码几乎一样，只改变了一个字母。行列对称是编程美学的体现。

C++

```
#include <cstdio>
int main() {
    int c; char t;
    scanf("%d %c", &c, &t);
    double sum = 0, x;
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++) {
            scanf("%lf", &x);
            if (j == c) sum += x;
        }
    printf("%.1lf\n", t == 'S' ? sum : sum / 12);
    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
c = int(data[0])
t = data[1]
s = 0.0
idx = 2
for i in range(12):
    for j in range(12):
        x = float(data[idx])
        idx += 1
        if j == c:
            s += x
print(f"{s if t == 'S' else s / 12:.1f}")
```

NQ051 简单斐波那契 AcWing 717

"我突然想——能不能只算第N项，不输出前面的？"小鲁问。"当然可以！"小华说，"只需要把输出改为只输出最后一个值，中间结果不用存储。这就是O(1)空间复杂度——不管N多大，你只需要两个变量a和b。"

输入 一个整数N。

输出 第N项的值。

范围 $0 \leq N \leq 60$

输入

4

输出

3

思路 Py:a,b=0,1;for _ in range(n):a,b=b,a+b;print(a)。循环n次后a就是第n项。注意循环次数和输出的对应关系——这是递推的"根"模式。

技巧 O(1)空间的意思是：无论N=10还是N=60，内存用量恒定（只需2个变量）。这展示了递推相比递归的最大优势——不需要存储所有历史结果。

C++

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    long long a = 0, b = 1;
    if (n == 0) { cout << 0 << endl; return 0; }
    for (int i = 1; i < n; i++) {
        long long t = a + b; a = b; b = t;
    }
    cout << b << endl;
    return 0;
}
```

Python

```
n = int(input())
a, b = 0, 1
for _ in range(n):
    a, b = b, a + b
print(a)
```


NQ052 数字序列和它的和 AcWing 722

小鲁在练习while循环的用法。"如果我想一直读数据直到遇到0为止怎么办?"小华说:"while True:读入→判断→break。这是处理'不定长输入'的标准模式。结合for循环生成序列和累加,你就能处理各种输入场景了。"

输入 多行, 每行两个整数M和N。以M≤0或N≤0结束。

输出 对每对M,N, 输出所有整数和"Sum=X"。

范围 M,N ≤ 100

输入

5 10
2 3
0 0

输出

5 6 7 8 9 10 Sum=45
2 3 Sum=5

思路 while+break是不定长输入的标准处理模式。对每对M,N, 先交换确保M≤N, 然后用for循环从M输出到N并累加。Python的range()+sum()组合最优雅。

技巧 Python: print(' '.join(map(str, range(m, n+1))), f'Sum={sum(range(m, n+1))}'). 一行搞定!

C++

```
#include <iostream>
using namespace std;
int main() {
    int m, n;
    while (cin >> m >> n, m > 0 && n > 0) {
        if (m > n) swap(m, n);
        int sum = 0;
        for (int i = m; i <= n; i++) {
            cout << i << " ";
            sum += i;
        }
        cout << "Sum=" << sum << endl;
    }
    return 0;
}
```

Python

```
while True:
    m, n = map(int, input().split())
    if m <= 0 or n <= 0: break
    if m > n: m, n = n, m
    nums = range(m, n + 1)
    print(' '.join(map(str, nums)),
          f"Sum={sum(nums)}")
```

NQ053 完全数 AcWing 725

小华在图书馆看到一本书上写着"完全数"。"6=1+2+3, 28=1+2+4+7+14.....这些数太美了!"他兴奋地对小鲁说:"你知道吗? 在 10^8 以内只有四个完全数: 6,28,496,8128。这是数学家欧几里得和欧拉证明的。所以说——有时候数学知识可以让你少写很多代码!"

输入 一个整数N。

输出 每行一个完全数。

范围 $1 \leq N \leq 10^8$

输入	输出
30	6 28

思路 10^8 以内只有4个完全数。直接判断它们是否 $\leq N$ 即可。如果用朴素算法（枚举因子求和），对于 $N=10^8$ 需要几十秒——而用数学知识， $O(1)$ 时间解决。数学+算法=效率的天花板。

技巧 这是“先思考，再编码”的典范。编程不是蛮力计算——有时候知道答案比计算答案更高效。厦门大学数学系有深厚的数论传统——学编程时别忘了你的数学课！

C++

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    int perfect[] = {6, 28, 496, 8128};
    for (int x : perfect)
        if (x <= n) cout << x << endl;
    return 0;
}
```

Python

```
n = int(input())
# All perfect numbers up to 10^8: 6,
# 28, 496, 8128, 33550336
perfect = [6, 28, 496, 8128, 33550336]
for p in perfect:
    if p <= n:
        print(p)
```

NQ054 质数 AcWing 726

小嘉路过实验室，看到小鲁和伙伴们正在讨论质数判定。“质数呀！”小嘉饶有兴趣地说，“我在南洋做生意的时候，数学帮我解决了不少问题。你们知道吗——判断一个数是不是质数，不需要检查到 $N-1$ ，只需要检查到 $\sqrt{N}$ 。因为如果 N 有大于 $\sqrt{N}$ 的因子，一定有一个小于 $\sqrt{N}$ 的对应因子。”

输入 一个整数 N 。

输出 每行一个质数。

范围 $2 \leq N \leq 10^6$

输入	输出
10	2 3 5 7

思路 $\sqrt{N}$ 优化：对2到 N 的每个数 i ，只需检查2到 $\sqrt{i}$ 的因子。 $j*j \leq i$ 作循环条件比 $j \leq \text{sqrt}(i)$ 更高效（避免浮点运算）。时间复杂度从 $O(n^2)$ 降到 $O(n\sqrt{n})$ 。算法优化的第一课——数学知识直接转化为效率提升。

技巧 Py用`math.isqrt(i)`获得整数平方根。 $j*j \leq i$ 替代 $j \leq \text{sqrt}(i)$ 避免浮点。小嘉说得对： $\sqrt{N}$ 的数学原理是优化算法的关键。厦大校训“自强不息”——算法优化永无止境。

C++

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    int n;
    cin >> n;
    for (int i = 2; i <= n; i++) {
        bool prime = true;
        for (int j = 2; j * j <= i; j++) {
            if (i % j == 0) { prime = false; break; }
        }
        if (prime) cout << i << endl;
    }
    return 0;
}
```

Python

```
import math
n = int(input())
for i in range(2, n + 1):
    prime = True
    for j in range(2, int(math.sqrt(i)) + 1):
        if i % j == 0:
            prime = False
            break
    if prime:
        print(i)
```

本章知识点总结

- 数组五大操作：遍历(iterate)、筛选(filter)、变换(transform)、翻转(reverse)、查找(search)
- C++ vs Python：定长栈数组 vs 动态堆列表。C++更快，Python更灵活
- 递推思想：斐波那契——每个新值由前两个已知值计算，是DP的萌芽
- 算法优化第一课：质数判定的 $\sqrt{N}$ 优化——用数学将复杂度从 $O(n^2)$ 降到 $O(n\sqrt{n})$
- 厦大校训：自强不息，止于至善。编程之路漫长，但每一步都在进步

矩阵的舞蹈

第5章 · 多维数组与矩阵模式 · 12题 · C++ & Python 双语对照

第四周结束，小鲁已经能熟练地操作一维数组了。但小华告诉他："一维数组只是第一步——真实世界的的数据很少是一条线。想想电子表格、数码照片、游戏棋盘……这些都是二维的。"小鲁若有所思："所以……我需要学会在行列之间跳舞？"

本章12道题分三个板块：**区域遍历**（NQ055-063）——学会用行列关系精确切割矩阵区域；**模式填充**（NQ059,064,065）——用数学公式一行替代几十行循环，体验"数学建模"的力量；**蛇形遍历**（NQ066）——方向数组的经典实战，贪吃蛇游戏引擎的底层算法。

本章角色

-  **小鲁**（初学者）· 正在经历从"逐题应付"到"寻找规律"的思维跃迁
-  **小华**（算法高手）· ACM竞赛队队长，用"对称性"一句话解决四道相似的题
-  **小栋**（工程师）· 把枯燥的矩阵遍历与游戏引擎、图像处理等实际应用联系起来
-  **小嘉**（校主精神）· 带着"同心方"从南洋归来，用数学和编程的交汇点启发学生

矩阵三角区域速查表（12×12矩阵）

- ↗ **右上三角**： $j > i$ （不含对角线，66个元素）
- ↖ **左上三角**： $j < 11-i$ （不含反对角线，66个元素）
- ↙ **左下三角**： $j < i$ （不含对角线，66个元素）
- ↘ **右下三角**： $i+j > 10$ （66个元素）
- ↑ **上方区域**： $i < j$ 且 $i+j < 11$ （30个元素）
- ↓ **下方区域**： $i > j$ 且 $i+j > 10$ （30个元素）
- ← **左方区域**： $j < i$ 且 $i+j < 11$ （30个元素）
- → **右方区域**： $j > i$ 且 $i+j > 10$ （30个元素）

核心心法：四个三角区域通过两种对角线分割（主对角线、反对角线），四个"口"字区域是两个三角的交集。掌握对称性=一道题的功夫解决四道题。

NQ055 数组的右上半部分 AcWing 745

小鲁拿着一支荧光笔，在一张12×12的格子纸上涂色：右上三角。" $j > i$ ——第0行从第1列到第11列，第1行从第2列到第11列……"小鲁一边涂一边念叨。小华走过来："你在画什么？""我在数右上三角有多少个元素！总共12×12减去对角线12，再除2——就是66个。"小华笑道："你已经发现了三角区域的核心：找到行和列的关系表达式，剩下的循环就水到渠成。"

输入 第一行操作类型(S/M)。然后144个浮点数（逐行读入）。

输出 结果保留1位小数。S输出和，M输出平均值。

范围

矩阵元素 -10^6 到 10^6

输入

S
(144个浮点数...)

输出

(sum,1 decimal)

思路 右上三角： $j > i$ 。双层循环遍历矩阵，当 $j > i$ 时累加。不包括对角线。66个元素。S求总和，M求平均时除以66（注意不是144）。区域遍历是矩阵题的灵魂——关键在于找到正确的行列关系表达式。

技巧 右上三角条件： $j > i$ 。共有元素数 $= (12 \times 12 - 12) / 2 = 66$ 。C++用double型累加，注意不要用int。Python的float是双精度。

C++

```
#include <cstdio>

int main()
{
    char t;
    scanf("%c", &t);
    double a[12][12];

    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            scanf("%lf", &a[i][j]);

    int c = 0;
    double s = 0;

    for (int i = 0; i < 12; i++)
        for (int j = i + 1; j < 12; j++)
        {
            c++;
            s += a[i][j];
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
t = data[0]
s = 0.0
idx = 1
for i in range(12):
    for j in range(12):
        x = float(data[idx])
        idx += 1
        if j > i:
            s += x
print(f"{s if t == 'S' else s / 66:.1f}")
```

NQ056 数组的左上半部分 AcWing 747

小鲁转向左上三角：“这次应该用 $j < 11 - i$ 吧？反对角线以上……”小华在纸上画了一笔：“看，反对角线满足 $i + j = 11$ 。左上三角就是 $j < 11 - i$ 。但还有另外一种理解——它和右上三角本质上是同一个形状，只是参考线不一样。一个用主对角线($j > i$)，一个用反对角线($j < 11 - i$)。两题放在一起对比，四种三角区域的条件就清晰了。”

输入

操作类型(S/M)+144浮点数。

输出

保留1位小数。

范围

同NQ055

输入

S
(144...)

输出

(sum,1 decimal)

思路 左上三角： $j < 11 - i$ （反对角线以上）。反对角线条件： $i + j = 11$ 。元素数量也是66。将四个三角区域的遍历条件对比理解，是掌握矩阵区域操作的关键—— $j > i$ （右上）、 $j < i$ （右下）。

技巧 反对角线条件： $i + j == n - 1$ （ $n = 12$ 时为11）。左上三角用 $<$ 。四个区域条件对称且互补，理解这一规律后所有矩阵三角题迎刃而解。

C++

```
#include <cstdio>

int main()
{
    double a[12][12];
    char t;

    scanf("%c", &t);
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            scanf("%lf", &a[i][j]);

    int c = 0;
    double s = 0;
    for (int i = 0; i < 12; i++)
        for (int j = 0; j <= 10 - i; j++)
        {
            c++;
            s += a[i][j];
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}
```

Python

```
t=input().strip();s=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if j<11-i:s+=x
    print(f'{s if t=="S" else s/66:.1f}')
```

NQ057 数组的上方区域 AcWing 749

"等等——'上方区域'又是什么？"小鲁挠着头。小栋在屏幕上画了一个"八"字形："你看，把 12×12 矩阵的上半部分，去掉左边和右边的两个三角——剩下的就是上方区域。它其实是两个条件的交集：既在主对角线之上(i

输入

操作类型(S/M)+144浮点数。

输出

保留1位小数。

范围

同NQ055

输入
S
(144...)

输出
(sum,1 decimal)

思路 上方区域是两个三角的交集：i

技巧 交集条件：i

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    char t;
    cin >> t;
    double m[12][12];

    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 0; i < 5; i++)
        for (int j = i + 1; j <= 10 - i; j++)
        {
            c += 1;
            s += m[i][j];
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}
```

Python

```
t=input().strip();s=c=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if i<j and i+j<11:s+=x;c+=1
    print(f'{s if t=="S" else s/c:.1f}')
```

NQ058 数组的左方区域 AcWing 751

"左边区域呢？"小鲁问。小华说："对称的！把上方区域的j>i改成j

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 同NQ055

输入
S
(144...)

输出
(sum,1 decimal)

思路 左方区域与上方区域对称：将条件改为j

技巧 左右对称：互换i和j的角色。上方=去掉左右三角的上部，左方=去掉上下三角的左部。对称思维是编程中的强大工具——写一遍代码，改一行条件就能得到四种区域的遍历。

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    char t;
    cin >> t;

    double m[12][12];
    for (int i = 0; i < 12; i ++ )
        for (int j = 0; j < 12; j ++ )
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 1; i <= 5; i ++ )
        for (int j = 0; j <= i - 1; j ++ )
        {
            s += m[i][j];
            c += 1;
        }
    for (int i = 6; i <= 10; i ++ )
        for (int j = 0; j <= 10 - i; j ++ )
        {
            s += m[i][j];
            c += 1;
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}
```

Python

```
t=input().strip();s=c=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if j<i and i+j<11:s+=x;c+=1
    print(f'{s if t=="S" else s/c:.1f}')
```

NQ059 平方矩阵I AcWing 753

小嘉走进教室，手里拿着一张回字形的图纸：“这在南洋叫‘同心方’——外面一圈是1，往里一圈是2，再往里是3……最中心的值是 $\min(N+1, N+1, N, N)+1$ 。”小鲁看得目瞪口呆：“这么复杂的图案，用一个公式就能生成？！”小华解释：“每个位置的值=到四条边的最近距离+1。外层距离0→值1，往里一层距离1→值2。理解这个‘距离模型’比写100行if更重要。”

输入 多个整数N，以0结束。

输出 每个矩阵占N行，每个数占3字符宽度右对齐。矩阵间空行。

范围 $1 \leq N \leq 100$

输入

1
2
3
0

输出

1

1 1
1 1

1 1 1
1 2 1
1 1 1

思路 数学公式法： $val = \min(i, j, n-1-i, n-1-j) + 1$ (0-indexed)。这个公式的物理意义：每个位置的值等于它到四条边的最短距离加1。最外层距离为0→值=1，往里一层距离为1→值=2，以此类推。理解公式比手写模拟更体现编程智慧。

技巧 到四条边的距离：上i、左j、下n-1-i、右n-1-j。取最小值+1即为该位置的值。这个公式体现了"数学建模"的力量——用数学语言描述复杂的几何规律。

C++

```
#include <cstdio>
int main() {
    int n;
    while (scanf("%d", &n) == 1 && n) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                int v = i < j ? i : j;
                int w = n - 1 - i < n - 1 - j ? n - 1 - i : n - 1 - j;
                int m = v < w ? v : w;
                printf("%3d", m + 1);
            }
            printf("\n");
        }
        printf("\n");
    }
    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
for val in data:
    n = int(val)
    if n == 0: break
    for i in range(n):
        row = []
        for j in range(n):
            v = min(i, j, n-1-i, n-1-j) + 1
            row.append(f'{v:3d}')
        print(''.join(row))
    print()
```

NQ060 数组的右下半部分 AcWing 748

小鲁拿着一面镜子放在矩阵的左上角——"如果从左上三角对称过去……"小华笑道："没错！右上三角($j > i$)的对称是左下三角($j > i$)。四题八遍的遍历模式，如果用'条件表达式'分类，一共就两种本质：主对角线分割和反对角线分割。"

输入

操作类型(S/M)+144浮点数。

输出

保留1位小数。

范围

同NQ055

输入

输出

S (sum,1 decimal)
(144...)

思路 右下三角： $i+j>10$ 。与NQ056的左上三角($i+j<11$)对称。通过前6个矩阵三角题的系统训练，你应该已经掌握了区域遍历的核心技巧：找到正确的行列关系表达式。

技巧 $i+j>10$ 与 $11-i-1$ 通。矩阵遍历的4个三角区域至此全部覆盖。建议画一个5×5矩阵逐格验证——对着纸面比对着屏幕理解更快。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    char t;
    cin >> t;
    double m[12][12];

    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 1; i <= 11; i++)
        for (int j = 12 - i; j <= 11; j++)
        {
            s += m[i][j];
            c += 1;
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
t = data[0]
s = 0.0
idx = 1
for i in range(12):
    for j in range(12):
        x = float(data[idx]); idx += 1
        if i + j > 11: s += x
print(f"{s:.1f}" if t == 'S' else f"{s/66:.1f}")
```

NQ061 数组的左下半部分 AcWing 746

"左下三角就是ji对称，只是方向反过来。"小鲁总结说。小华欣慰地拍拍他："你已经从'逐题应付'升级到'找规律'了。记住这种感觉——以后学任何新算法，第一步都是找pattern：有没有对称性？能不能归约成已知问题？这是从学生到工程师的思维跃迁。"

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 同NQ055

输入

输出

S (sum,1 decimal)
(144...)

思路 左下三角：j<i)对称。遍历条件简单明了——i和j的大小关系决定位置归属。注意不包括主对角线。

技巧 四个三角区域的条件总结：右上j>i、左下j<10-i。掌握这4个条件，所有标准矩阵三角题都不在话下。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    char t;
    cin >> t;

    double m[12][12];
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 0; i < 12; i++)
        for (int j = 0; j <= i - 1; j++)
        {
            s += m[i][j];
            c += 1;
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}
```

Python

```
t=input().strip();s=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if j<i:s+=x
    print(f'{s if t=="S" else s/66:.1f}')
```

NQ062 数组的下方区域 AcWing 750

"哎，下方区域的条件应该就是 $i > j$ 且 $j > 10 - i$ 吧？"小鲁试着推导。"等一下， $j > 10 - i$ 就是 $i + j > 10$ 。"小栋纠正道。"对对对， $i > j$ 且 $i + j > 10$ ——和上方区域的i

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 同NQ055

输入
S
(144...)

输出
(sum,1 decimal)

思路 下方区域： $i > j$ 且 $i + j > 10$ 。与上方区域(i

技巧 对称总结：上方($i < j$, $i + j > 10$)、左方($j < i$, $i + j > 10$)。四个区域构成一个完整的"口"字形框架。

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    char t;
    cin >> t;

    double m[12][12];
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 7; i <= 11; i++)
        for (int j = 12 - i; j <= i - 1; j++)
        {
            c += 1;
            s += m[i][j];
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
t = data[0]
s = 0.0
idx = 1
for i in range(12):
    for j in range(12):
        x = float(data[idx]); idx += 1
        if i > j and i + j > 11: s += x
print(f"{s:.1f}" if t == 'S' else f"{s/30:.1f}")
```

NQ063 数组的右方区域 AcWing 752

小鲁把四道区域题全部AC后，看着屏幕上的"Accepted"，长出了一口气。"8道矩阵区域遍历题全部完成了！"小华说："总结一下——你真正学到了什么？不是8个条件表达式，而是'精确描述几何区域的思维方式'。未来做图像处理、游戏碰撞检测、地图分析，用的还是这个思维——把视觉区域翻译成代码条件。"

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 同NQ055

输入

S
(144...)

输出

(sum, 1 decimal)

思路 右方区域： $j > i$ 且 $i + j > 10$ 。30个元素。至此矩阵区域遍历全部完成。8道题的变化本质是条件表达式的组合。掌握了规律，任何形状的区域都能精确描述。

技巧 四个"口"字区域全部完成后，你的二维数组遍历能力已经达到面试水平。Key takeaway：精确的行列条件表达式=矩阵操作的全部。

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    char t;
    cin >> t;
    double m[12][12];

    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 1; i <= 5; i++)
        for (int j = 12 - i; j <= 11; j++)
        {
            s += m[i][j];
            c += 1;
        }
    for (int i = 6; i <= 10; i++)
        for (int j = i + 1; j <= 11; j++)
        {
            s += m[i][j];
            c += 1;
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
t = data[0]
s = 0.0
idx = 1
for i in range(12):
    for j in range(12):
        x = float(data[idx]); idx += 1
        if j > i and i + j > 11: s += x
print(f"{s:.1f}" if t == 'S' else f"{s/30:.1f}")
```

NQ064 平方矩阵II AcWing 754

"这道比上一道更简单！"小鲁看着平方矩阵II的模式：对角线是1，往外一层是2，再往外是3....."这不就是看每个位置到对角线的距离吗？"小华回答："对！ $\text{abs}(i-j)$ 就是到对角线的'曼哈顿距离'——对角线上 $\text{abs}(i-j)=0 \rightarrow$ 值 $0+1=1$ ，偏离一格 $\text{abs}=1 \rightarrow$ 值 $1+1=2$ 。两种平方矩阵展示了两种数学建模：一个是到四条边的距离，一个是到对角线的距离。"

输入

多个整数N（0结束）。

输出

N×N矩阵，每个数占3字符宽。矩阵间空行。

范围

 $1 \leq N \leq 100$

输入

3
0

输出

1 2 3
2 1 2

3 2 1

思路 $|i-j|+1$ 公式产生以主对角线为对称轴的矩阵。对角线上 $|i-j|=0 \rightarrow$ 值=1，离对角线每远一步值增加1。这比平方矩阵I的"四边距离"公式更简单，展示了另一种数学建模方式。

技巧 $\text{abs}(i-j)+1$ 。C++用stdlib的abs()，Python内置abs()。注意绝对值函数对负数的处理——自动取正。这种"距离模型"在矩阵题中非常常见。

C++

```
#include <stdio>
#include <stdlib>
int main() {
    int n;
    while(scanf("%d",&n)==1&&n) {
        for(int i=0;i<n;i++) {
            for(int j=0;j<n;j++) printf("%3d",abs(i-j)+1);
            printf("\n");
        }
        printf("\n");
    }
    return 0;
}
```

Python

```
while True:
    n=int(input())
    if n==0:break
    for i in range(n):
        print(''.join(f'{abs(i-j)+1:3d}' for
j in range(n)))
    print()
```

NQ065 平方矩阵III AcWing 755

小鲁看着第三个平方矩阵： $2^0, 2^1, 2^2 \dots$ "这是指数增长！""对！"小华说，"两个技巧：第一，用位运算 $1 \ll (i+j)$ 比 $\text{pow}(2,i+j)$ 快很多；第二， $N \leq 15$ 的限制是精心设计的—— $N=15$ 时最大值 $2^{28} \approx 2.68$ 亿，刚好在int32范围内。如果N再大就要用long long甚至大整数了。题目设计者考虑得很周全。"

输入

多个整数N（0结束）。

输出

每个数右对齐到最大数的宽度+1。矩阵间空行。

范围

 $1 \leq N \leq 15$

输入

3
0

输出

```
1 2 4
2 4 8
4 8 16
```

思路 $2^{(i+j)}$ 模式。C++用 $1 \ll (i+j)$ 位运算高效计算2的幂。Python可以 $2^{**}(i+j)$ 或 $1 \ll (i+j)$ 。右对齐宽度=最长数字的位数+1。注意 $N \leq 15$ 的限制： $N=15$ 时 $2^{28} \approx 2.68 \times 10^8$ (9位数)。

技巧 C++的 $1 \ll$

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    int n;
    while (cin >> n, n)
    {
        for (int i = 0; i < n; i ++ )
        {
            for (int j = 0; j < n; j ++ )
            {
                int v = 1;
                for (int k = 0; k < i + j; k ++ ) v *= 2;
                cout << v << ' ';
            }
            cout << endl;
        }

        cout << endl;
    }

    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
for val in data:
    n = int(val)
    if n == 0: break
    for i in range(n):
        row = []
        for j in range(n):
            v = 1
            for k in range(i + j):
                v *= 2
            row.append(str(v))
        print(' '.join(row) + ' ')
    print()
```

NQ066 蛇形矩阵 AcWing 756

"最后一道是大boss——蛇形矩阵！"小鲁聚精会神地盯着屏幕。小华拿出了他的'镇山法宝'："四向数组： $dx=[0,1,0,-1]$ ， $dy=[1,0,-1,0]$ ——分别代表右、下、左、上。从(0,0)出发向右走，撞墙或遇到已填的格子就顺时针转90度。方向数组是网格类问题的屠龙刀。"小栋补充道："其实你玩的贪吃蛇游戏引擎，底层就是这个算法。"

输入 一行两个整数n和m。

输出 n行m列的蛇形矩阵。

范围 $1 \leq n, m \leq 100$

输入	输出
3 3	1 2 3 8 9 4 7 6 5

思路 方向数组+边界检测。 $dx=[0,1,0,-1]$ ， $dy=[1,0,-1,0]$ 依次控制 $\rightarrow \downarrow \leftarrow \uparrow$ 。撞墙或已填则转向 $d=(d+1)\%4$ 。while循环填充所有格子。方向数组是解决螺旋/蛇形/遍历类问题的通用模板——只需改变方向顺序就能实现不同的遍历模式。

技巧 方向数组是处理网格遍历的万能钥匙。记住4个方向的顺序和dx/dy的对应关系。Python中提前分配二维数组：`[[0]*m for _ in range(n)]`（注意不能用`[[0]*m]*n`——那是浅拷贝的经典陷阱！）。

C++

```
#include <iostream>

using namespace std;

int res[100][100];

int main()
{
    int n, m;
    cin >> n >> m;

    int dx[] = {0, 1, 0, -1}, dy[] = {1, 0, -1, 0};

    for (int x = 0, y = 0, d = 0, k = 1; k <= n * m; k++)
    {
        res[x][y] = k;
        int a = x + dx[d], b = y + dy[d];
        if (a < 0 || a >= n || b < 0 || b >= m || res[a][b])
        {
            d = (d + 1) % 4;
            a = x + dx[d], b = y + dy[d];
        }
        x = a; y = b;
    }

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++) cout << res[i][j] << ' ';
        cout << endl;
    }

    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
a = [[0] * m for _ in range(n)]
dx = [0, 1, 0, -1]
dy = [1, 0, -1, 0]
d = x = y = 0
for v in range(1, n * m + 1):
    a[x][y] = v
    nx, ny = x + dx[d], y + dy[d]
    if nx < 0 or nx >= n or ny < 0 or ny >= m or a[nx][ny]:
        d = (d + 1) % 4
        nx, ny = x + dx[d], y + dy[d]
    x, y = nx, ny
for r in a:
    print(' '.join(map(str, r)) + '
')
```

本章知识点总结

- **8种矩阵区域**：4个三角（各66元素）+ 4个"口"字边（各30元素）= 精确遍历矩阵任意形状
- **3种填充模式**：四边距离min公式、对角线距离abs公式、2的指数幂模式——用数学替代模拟
- **方向数组**：dx=[0,1,0,-1], dy=[1,0,-1,0] → 控制四个方向旋转，蛇形/螺旋/遍历通用模板
- **对称思维**：上下、左右区域只是i和j角色的互换。找规律>逐题应付——从学生到工程师的思维跃迁
- **关键技巧**：Python二维列表[[0]*m for _ in range(n)]（不能用*操作符——那是浅拷贝陷阱！）
- **厦大精神**：自强不息，止于至善。12道矩阵题过后，你的多维数组能力已经达到面试水平。

字符的世界

第6章 · 字符串处理 · 12题 · C++ & Python 双语对照

小鲁已经学会了数字的处理——整数、浮点数、数组。但真实世界的程序处理得最多的不是数字，而是文字。从搜索引擎到聊天软件，从代码编辑器到语音助手——字符串处理是每个程序员的基本功。而Python在字符串处理上的优势几乎是碾压级别的：切片、join、split、replace……C++需要好几行的操作，Python往往一行解决。

本章12道题，涵盖了字符串六大核心操作：长度与遍历、字符分类、修改与替换、切片与拼接、频次统计、分词与重构。学完之后你会发现——用Python处理字符串是编程中最快乐的事之一。

本章角色

-  **小鲁**（初学者）· 从"数字世界"迈入"文字世界"，发现Python字符串处理的魔法
-  **小华**（算法高手）· 擅长对比C++和Python的字符串处理差异，强调"理解原理"
-  **小栋**（工程师）· 用"文本编辑器""搜索引擎"等实际场景出题
-  **小嘉**（校主精神）· 偶尔提及密码学和南洋通信的故事，赋予字符串处理历史深度

Python字符串处理四大法宝

- **切片**：s[a:b]截取[a,b)区间。s[:i]前缀、s[i:]后缀、s[::-1]反转
- **分割与拼接**：s.split(sep)按分隔符拆分，sep.join(list)按分隔符拼接
- **查找与替换**：s.find(sub)、s.replace(old,new)、sub in s 成员判断
- **字符分类**：isdigit()/isalpha()/isupper()/islower()/isspace() 自动分类
- **频次统计**：collections.Counter(s) 一行建字典。字符→频次的O(1)映射

C++注意：字符串分为std::string（类）和char[]（C风格），后者需要strlen/strcpy/strcat。现代C++推荐用std::string。

NQ067 字符串长度 AcWing 760

小鲁输入了一串文字，想知道它有多长。"这还用编程？数一下不就行了……"话没说完，小华就打断了他："如果有人给你一本1000页的书，你一个字一个字数吗？Python用len(s)一秒搞定。C++用s.length()或strlen()。字符串长度是所有文本处理的基础——搜索、替换、截取，第一步永远是知道字符串有多长。"

输入 一行，一个字符串（不超过100字符）。

输出 一个整数，表示字符串长度。

范围 字符串长度≤100

输入

hello world

输出

11

思路 字符串长度是最基本的信息。Python: `len(s)`, C++: `s.length()`或`s.size()` (string类) 或`strlen(s)` (C风格char数组)。注意空格也算长度——"hello world"是11个字符不是10个。

技巧 Python的`len()`适用于所有序列类型 (字符串/列表/元组), 是一个全局函数而非方法。C++的`length()`和`size()`对于string完全等价。C风格字符串用`strlen()`但需要`#include`。

C++

```
#include <cstdio>

int main()
{
    char str[101];

    fgets(str, 101, stdin);

    int len = 0;
    for (int i = 0; str[i] && str[i] != '\n'; i++) len++;

    printf("%d\n", len);

    return 0;
}
```

Python

Python solution

NQ068 字符串中的数字个数 AcWing 761

"帮我统计这篇文章里有多少个数字字符....."小栋拿着一份报纸对小鲁说。小鲁刚要用手指一个一个点, 小华按住他: "遍历字符串的每个字符, 用`isdigit()`判断是不是数字。Python一行搞定——`sum(c.isdigit() for c in s)`。C++则是for循环判断`c>='0' && c<='9'`。"小鲁这才意识到字符串里的每个字符都可以被'分类'——数字、字母、空格、标点.....每种分类都有对应的判断方法。

输入 一行, 一个字符串 (不超过100字符)。

输出 字符串中数字字符的个数。

范围 字符串长度 ≤ 100

输入	输出
I am 20 years old.	2

思路 分类统计模式: 遍历每个字符, 判断是否为数字 ('0'-'9')。Python: `sum(c.isdigit() for c in s)`, C++: 循环+条件判断。`isdigit()`是Python字符串方法的强大之处。

技巧 Python字符串方法: `s.isdigit()` (数字)、`s.isalpha()` (字母)、`s.isalnum()` (字母数字)、`s.isspace()` (空白)。记住这些方法能让字符串处理效率翻倍。

C++

```
#include <cstdio>

int main()
{
    char str[101];

    fgets(str, 101, stdin);

    int cnt = 0;
    for (int i = 0; str[i]; i++)
        if (str[i] >= '0' && str[i] <= '9')
            cnt++;

    printf("%d\n", cnt);

    return 0;
}
```

Python

```
s = input()
print(sum(1 for c in s if c.isdigit()))
```

NQ069 循环相克令 AcWing 763

小鲁和小栋在玩"锤子剪刀布"的游戏——不过他们的版本叫"Hunter/Bear/Gun"。"猎人怕熊，熊怕枪，枪怕猎人——形成一个循环克制关系。"小华说，"这种循环相克的问题可以用字符串比较解决。但更优雅的是数学方法：把每个角色映射成0/1/2，然后胜者 $=(a-b+3)\%3$ 。"小鲁惊奇地发现——一个简单的取模竟然能判断所有博弈结果。

输入 一个整数N。然后N行每行两个字符串（Hunter/Bear/Gun）。

输出 对每行输出Player1赢还是Player2赢。

范围 $1 \leq N \leq 100$

输入	输出
3	Player2
Hunter Bear	Player1
Bear Hunter	Player1
Gun Bear	

思路 字符串比较法：if-elif分支判断所有9种组合。数学法：将三个角色映射为0,1,2，则P1赢当且仅当 $(a-b+3)\%3==2$ 。数学法将9个if压缩成一行——用数学替代分支的艺术。

技巧 Python用法：dic={'Hunter':0,'Bear':1,'Gun':2}，然后if (dic[a]-dic[b]+3)%3==2: P1赢。C++用map或直接if-else。理解取模在循环问题中的通用性更关键。

C++

```
#include <stdio>
#include <string>
int main() {
    char a[20],b[20];
    while(scanf("%s%s",a,b)==2) {
        if(!strcmp(a,b)) printf("Tie\n");
        else if((!strcmp(a,"Hunter")&&!strcmp(b,"Gun"))||(!strcmp(a,"Gun")&&!strcmp(b,"Bear"))||(!strcmp(a,"Bear")&&!strcmp(b,"Hunter"))) printf("Player1\n");
        else printf("Player2\n");
    }
    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
p1, p2 = data[0], data[1]
if p1 == p2: print('Tie')
elif (p1=='Hunter' and p2=='Gun') or (p1=='Bear' and p2=='Hunter') or (p1=='Gun' and p2=='Bear'):
    print('Player1')
else:
    print('Player2')
```

NQ070 字符串加空格 AcWing 765

小鲁在排版一个标题——需要给每个字符之间加上空格。"这要手动打多少个空格……"小华笑了："Python的join()方法就是为此而生的：''.join(s)——用空格把字符串的每个字符连接起来。C++需要遍历输出每个字符后面加空格然后去掉末尾空格。两种语言对比，Python的join()堪称字符串处理的魔法。"

输入 一行，一个字符串（不含空格）。

输出 每个字符间加空格后的字符串。

范围 字符串长度≤100

输入

hello

输出

h e l l o

思路 join()方法将一个可迭代对象用分隔符连接。''.join(s)把s的每个字符用空格连接。C++版需要for循环逐个输出注意首尾空格。Python的join()是最常用的字符串操作之一。

技巧 Python的join()和split()是互逆操作。'sep'.join(list)把列表用sep连起来，str.split(sep)把字符串按sep拆开。这两个方法是字符串处理的基础中的基础。

C++

```
#include <iostream>

using namespace std;

int main()
{
    string a;
    getline(cin, a);

    string b;
    for (auto c : a) b = b + c + ' ';

    b.pop_back();

    cout << b << endl;

    return 0;
}
```

Python

```
print(" ".join(input()))
```

NQ071 替换字符 AcWing 769

小鲁在编辑文本时发现有些字符需要换成别的："比如把所有的'a'换成'b'....."小华说："Python直接s.replace(old, new)一行搞定。C++要手动遍历替换。不过注意——Python的字符串是不可变的，replace()返回的是新字符串，原字符串不变。理解'不可变性'是理解Python字符串操作的关键。"

输入 一行字符串（可能含空格）。然后一行，两个字符（要替换的和替换成的）。

输出 替换后的字符串。

范围 字符串长度≤100

输入

```
hello world
o e
```

输出

```
helle world
```

思路 字符替换：遍历每个字符，如果等于目标则输出新字符，否则原样输出。Python: s.replace(old, new)。C++: 用for+if或std::replace()。注意Python的字符串不可变性——replace()返回新的。

技巧 Python字符串不可变意味着所有'修改'操作都返回新字符串。这保证了字符串的安全性（不会被意外修改），但创建大量新字符串时有性能开销。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    char str[31];
    scanf("%s", str);

    char c;
    scanf("\n%c", &c);

    for (int i = 0; str[i]; i++)
        if (str[i] == c)
            str[i] = '#';

    puts(str);

    return 0;
}
```

Python

```
s = input()
c = input()
print(s.replace(c, "#"))
```

NQ072 字符串插入 AcWing 773

"如果我想在字符串的第3个位置插入另一个字符串怎么办?"小鲁问。小华说:"拿到前面部分、后面部分,然后拼接: `s[:p]+sub+s[p:]`。注意位置`p`是从0开始还是从1开始——这是字符串操作中最容易出错的off-by-one问题。"小鲁立刻想到:"所以插入的本质就是'切片+拼接'——把字符串切成两段,中间塞进新内容。"

输入 三行: 原字符串、要插入的字符串、插入位置。

输出 插入后的字符串。

范围 字符串长度 ≤ 100 , 位置从0开始

输入

```
hello
world
3
```

输出

```
helworldlo
```

思路 Python切片拼接: `s[:p]+sub+s[p:]`。C++: `substr()`取前后两部分再相加。切片是Python最优雅的特性之一——`s[a:b]`精确截取`[a,b)`区间, 语义清晰。

技巧 Python切片`s[a:b]`的区间是`[a,b)` (包含`a`不包含`b`)。插入位置`p`意味着新内容从`p`开始——`s[:p]`取`p`之前的字符, `s[p:]`取`p`及之后的字符。

C++

```
#include <iostream>
#include <cstring>
using namespace std;
int main()
{
    string a,b;
    while(cin>>a>>b){

        int p=0;
        for (int i=0;i<a.size();i++)
            if (a[i]>a[p]) p=i; //找到最大的位置

        cout<<a.substr(0,p+1)+b+a.substr(p+1)<<endl;

    }
    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
if len(data) >= 2:
    a = data[0]
    b = data[1]
    p = 0
    for i in range(len(a)):
        if a[i] > a[p]:
            p = i
    print(a[:p+1] + b + a[p+1:])
```

NQ073 只出现一次的字符 AcWing 772

"我要在一段文字中找到第一个只出现一次的字符。"小栋又在给小鲁出题。"像'hello'里——'h','e','o'都只出现一次，但第一个是'h'。"小华说："这需要两遍遍历：第一遍统计每个字符的出现次数，第二遍按顺序找出第一个次数为1的字符。Python用collections.Counter可以一行建好频次字典。"

输入

一个字符串。

输出

第一个只出现一次的字符的位置（1-indexed），如果没有则输出-1。

范围

字符串长度 $\leq 10^5$

输入

hello

输出

1

思路 两遍遍历法：第一遍建立字符频次字典（Counter），第二遍按原字符串顺序查找第一个频次为1的字符。Python用Counter一键统计。注意频次统计不改变顺序——要按原串顺序输出。

技巧 Python: from collections import Counter; cnt=Counter(s); for i,c in enumerate(s): if cnt[c]==1: print(i+1)。C++用int cnt[256]数组直接作哈希表——字符ASCII值作为索引，O(1)频次查找。

C++

```
#include <iostream>
#include <cstring>

using namespace std;

int cnt[26];
char str[100010];

int main()
{
    cin >> str;

    for (int i = 0; str[i]; i++) cnt[str[i] - 'a']++;

    for (int i = 0; str[i]; i++)
        if (cnt[str[i] - 'a'] == 1)
        {
            cout << str[i] << endl;
            return 0;
        }

    puts("no");

    return 0;
}
```

Python

```
s = input()
from collections import Counter
cnt = Counter(s)
for c in s:
    if cnt[c] == 1:
        print(c)
        break
else:
    print("no")
```

NQ074 字符串匹配 AcWing 762

"在一大段文字里，找一小段模式——这就是字符串匹配。"小华在白板上写着，"比如在'abababa'里找'aba'。最朴素的方法：从每个位置开始，逐个字符比较——这叫暴力匹配， $O(n \times m)$ 。后续你会学到更高效的KMP算法，但暴力匹配首先让你理解匹配的本质：滑动窗口+逐位比较。"

输入

两行：文本串T和模式串P。

输出

P在T中第一次出现的位置（1-indexed），若不存在输出-1。

范围

 $1 \leq |T|, |P| \leq 100$

输入

```
abababa
aba
```

输出

1

思路 暴力匹配：对于i从0到n-m，检查T从i开始的m个字符是否等于P。Python直接用T.find(P)一行，但手写暴力匹配理解原理更重要。find()返回首次出现位置（-1表示不存在）。

技巧 Python的T.find(P)底层是高效的混合算法（Boyer-Moore-Horspool），比手写暴力快得多。但手写暴力匹配是KMP和BM算法的基础——理解"为什么慢"才能理解"如何变快"。

C++

```
#include <stdio>
#include <string>
int main() {
    double k;
    char a[110], b[110];
    scanf("%lf%s", &k, a, b);
    int match=0;
    int la=strlen(a), lb=strlen(b);
    for(int i=0; i+lb<=la; i++) {
        if(!strcmp(a+i, b, lb)) {match=1; break;}
    }
    printf(match?"yes\n":"no\n");
    return 0;
}
```

Python

```
import sys
lines = sys.stdin.read().splitlines()
a = lines[0] if len(lines) > 0 else ''
b = lines[1] if len(lines) > 1 else ''
print("yes" if b in a else "no")
```

NQ075 信息加密 AcWing 767

小鲁对密码学产生了兴趣："如果能写个程序把每个字母往后移1位——a变b, b变c.....z回到a——那就成了一个最简单的加密程序！"小华说："这就是凯撒密码，两千年前的加密方法。编程关键是处理循环——'z'+1要变回'a'。可以用取模： $(c-'a'+1)\%26+'a'$ 。大写和小写分别处理。"

输入 一个字符串（只含字母和空格）。

输出 字母后移1位，空格不变。

范围 字符串长度 ≤ 100

输入

abc XYZ

输出

bcd YZA

思路 字符偏移+循环回绕。对字母： $c='a'+(c-'a'+1)\%26$ 。大写字母同理用'A'和'Z'。非字母（空格等）原样保留。用isalpha()判断是否为字母，用isupper()/islower()判断大小写。

技巧 Python: $\text{chr}((\text{ord}(c)-\text{base}+1)\%26+\text{base})$ 处理循环偏移。C++直接做字符运算。理解ASCII码编码规则——字母是连续排列的，这是字符运算的前提。

C++

```
#include <iostream>
#include <cstring>
using namespace std;

int main(){

    string s;
    getline(cin,s);//读入一行

    for (auto &c:s)//使用auto注意&c直接引用字符串直接修改
        if (c>='a' && c<='z')
            c = 'a' + (c-'a'+1) % 26; //取模运算
        else if (c>='A' && c<='Z')
            c = 'A' + (c-'A'+1) % 26;

    cout<<s<<endl;//输出结果
}
```

Python

```
r = []
for c in input():
    if "a" <= c <= "z":
        r.append(chr((ord(c) - 97 +
1) % 26 + 97))
    elif "A" <= c <= "Z":
        r.append(chr((ord(c) - 65 +
1) % 26 + 65))
    else:
        r.append(c)
print(''.join(r))
```

NQ076 输出字符串 AcWing 764

"这道题有点怪——输入两个字符串A和B，对于B的每个位置，如果它不是A对应位置的字符，就把这个位置到B末尾的子串输出来。"小鲁读了三遍题目才明白。小华解释道："逐位比较A和B。一旦发现某个位置i的字符不同，就输出B从i开始的后缀子串。如果完全相同就输出A。关键是理解'后缀'概念——字符串从某位置到末尾的部分。"

输入 两行：字符串A和B。

输出 按规则输出。

范围 $1 \leq |A|, |B| \leq 100$

输入

abcdef
abcxyz

输出

xyz

思路 逐位比较：if $A[i] \neq B[i]$: 输出 $B[i:]$ 并结束。Python的 $B[i:]$ 切片取后缀。这道题训练的是字符串逐位比较+后缀切片的概念。

技巧 Python切片取后缀： $s[i:]$ 表示从i到末尾。 $s[:i]$ 表示从开头到i-1。切片的三个参数[start:end:step]可以省略任意部分——省略start表示从0开始，省略end表示到末尾。

C++

```
#include <iostream>

using namespace std;

int main()
{
    string a, b;
    getline(cin, a);

    for (int i = 0; i < a.size(); i ++ ) b += (char)(a[i]
+ a[(i + 1) % a.size()]);

    cout << b << endl;

    return 0;
}
```

Python

```
import sys
s = sys.stdin.read().rstrip('\n')
n = len(s)
result = ''
for i in range(n):
    result += chr(ord(s[i]) + ord(s[(i+1) % n]))
print(result)
```

NQ077 单词替换 AcWing 770

小鲁在写文本编辑器的一个功能：把所有出现的某个单词替换成另一个单词。"比如把新闻里所有的'北京'改成'Beijing'....."小华摇摇头："这个比你想象的复杂！如果用简单的replace，'北京大学'就会变成'Beijing大学'。在OJ环境下，输入是以空格分隔的单词——所以用split()分割后逐词判断替换，再用join()拼回去，更安全也更清晰。"

输入 三行：原文句子、要替换的单词、替换成的单词。

输出 替换后的句子。

范围 字符串长度≤1000

输入

I like apple
apple
orange

输出

I like orange

思路 分词→逐词判断→重建句子。Python: words=s.split(); result=' '.join(w if w!=old else new for w in words)。注意区分单词和子串——用split和join保证按词替换而非子串替换。

技巧 Python的split()默认按任意空白字符（空格、Tab、换行）分割。' '.join()则按单空格重新拼接。这两个方法组合是Python文本处理的黄金搭档。

C++

```
#include <iostream>
#include <sstream>

using namespace std;

int main()
{
    string s, a, b;

    getline(cin, s);
    cin >> a >> b;

    stringstream ssin(s);
    string str;
    while (ssin >> str)
        if (str == a) cout << b << ' ';
        else cout << str << ' ';

    return 0;
}
```

Python

```
import sys
lines = sys.stdin.read().splitlines()
s = lines[0] if len(lines) > 0 else ''
a = lines[1] if len(lines) > 1 else ''
b = lines[2] if len(lines) > 2 else ''

words = s.split()
result = ' '.join(b if w == a else w
for w in words)
print(result)
```

NQ078 最长单词 AcWing 774

"在一句话里找最长的单词——小学生也能做。"小鲁不屑地说。"那如果有两个一样长的呢？输出先出现的那个。"小栋又把条件加复杂了。小华说："用split()分词，然后遍历找最长的。Python一行就能搞定——max(words, key=len)。注意要去掉末尾的句号（这是一个小陷阱）。"

输入

一个英文句子，以'.'结尾。

输出

最长单词。若有多个，输出最先出现的。

范围

句子长度≤500

输入

I love programming.

输出

programming

思路 去掉句号→split()分词→max()找最长。Python: s.rstrip('.').split()得到单词列表，max(words, key=len)选出最长。注意max()默认返回最先出现的（稳定性）。

技巧 Python的max()在key相同时保持原始顺序（稳定排序）。rstrip('.')去掉末尾指定字符——这是去除标点的小技巧。split()不加参数会去掉所有空白字符。

C++

```
#include <iostream>

using namespace std;

int main()
{
    string res, str;

    while (cin >> str)
    {
        if (str.back() == '.') str.pop_back();
        if (str.size() > res.size()) res = str;
    }

    cout << res << endl;

    return 0;
}
```

Python

```
words=input().split()
print(max(words,key=len))
```

本章知识点总结

- 字符串六大操作：长度遍历、字符分类、修改替换、切片拼接、频次统计、分词重构
- Python四大法宝：切片s[a:b]、split+join、find+replace、isdigit+isalpha——一行代码等于C++五行的力量
- 字符串不可变：Python字符串修改操作全部返回新对象。理解这点才能正确使用replace/upper/lower等
- 暴力匹配：理解O(n×m)的朴素算法才能理解KMP的O(n+m)有多快。先学走再学跑
- 编码知识：ASCII码中字母连续排列，这是字符偏移加密的数学基础
- 厦大精神：自强不息，止于至善。从整数到字符串，你在成为更全面的程序员

模块化的力量

第7章 · 函数、递归与库的威力 · 12题 · C++ & Python 双语对照

小鲁已经写了上百行代码了。但他发现一个问题："同样的逻辑我写了好几遍——求最大值、算阶乘、交换变量.....能不能写一次，到处用？"小华笑道："你终于感受到**函数**的需求了！函数是编程中最基本、最重要的抽象手段——把一段逻辑打包装箱，贴个标签，以后直接叫名字就能用。"

本章12道题分三个板块：**函数定义与调用**（NQ083–088）——学会参数传递、返回值、数组传参；**递归思维**（NQ089–093）——函数自己调用自己，基础案例→记忆化优化→回溯搜索；**标准库的威力**——Python的math/lib等内置库让你的代码量减少90%。记住：**写函数的第一原则是**——先看看标准库有没有现成的。

本章角色

-  **小鲁**（初学者）· 从"重复代码"的痛苦中领悟函数的必要性
-  **小华**（算法高手）· 深入讲解C++引用传递vs Python对象引用，递归vs递推
-  **小栋**（工程师）· 用网格走方格、跳台阶等实际问题引入DP思维
-  **小嘉**（校主精神）· 带来《九章算术》的gcd算法，强调数学与编程的千年共鸣

函数三大价值 + C++ vs Python 对比

- **封装**：把逻辑打包→隐藏细节→只暴露接口。调用者不需要知道内部实现
- **复用**：写一次用百次。修改时只改一处
- **抽象**：fact(n)比"从1乘到n"高一个思考层次——用函数名表达意图
- **参数传递**：C++值传递vs引用传递(&)→&使函数能修改外部变量；Python"传对象引用"→可变对象(list)可被修改，不可变对象(int)不能
- **递归vs递推**：递归自顶向下（优雅但耗栈），递推自底向上（高效但需手动管理状态）
- **标准库优先**：math.gcd/factorial/comb, itertools.permutations, @lru_cache——先查库再动手

NQ083 n的阶乘 AcWing 804

小鲁正对着一道数学题发愁："5! = 5×4×3×2×1 = 120，这个我还能手算.....但是如果要算10!怎么办？"小华拉过键盘："函数就是做这个的！你把'计算阶乘'封装成一个fact(n)函数——每次需要算阶乘的时候调用它就行。这就叫'封装'。Python直接import math; math.factorial(10)一行搞定，但理解函数定义与调用的关系，比直接用库更重要。"

- 输入** 共一行，包含一个整数n。
- 输出** 共一行，包含一个整数表示n的阶乘的值。
- 范围** $1 \leq n \leq 10$

输入

3

输出

6

思路 阶乘 $n! = 1 \times 2 \times \dots \times n$ 。用for循环累乘。C++需定义`int fact(int n)`函数，Python可以`def fact(n)`或直接用`math.factorial()`。int在 $n \leq 10$ 范围内安全（最大 $10! = 3,628,800$ ）。

技巧 Python的`math.factorial()`使用高效的C实现。手写版关键是用`result=1`初始化然后`for i in range(1,n+1):result*=i`。初始化是初学者最常犯的bug——用0而不是1会导致结果永远是0。

C++

```
#include <iostream>

using namespace std;

int fact(int n)
{
    int res = 1;
    for (int i = 1; i <= n; i ++ )
        res *= i;
    return res;
}

int main()
{
    int n;
    cin >> n;

    cout << fact(n) << endl;

    return 0;
}
```

Python

```
import math;print(math.factorial(int(
input())))
```

NQ084 x和y的最大值 AcWing 805

"写个函数，输入两个数，返回较大的那个。"小栋对小鲁说。"这还用函数？直接if判断不就行了。"小鲁反驳。小华摇摇头："在大的程序里，你可能要在几十个地方都用max——每次都写if多麻烦，而且如果max逻辑改了（比如要支持更多参数），你要改几十处。有了`max(x,y)`函数，改一次，处处生效。这就是函数的第二个价值：代码复用。"

输入

一行，两个整数x和y。

输出

一个整数，较大者。

范围

 $-100 \leq x, y \leq 100$

输入

3 5

输出

5

思路 函数封装max逻辑：if x>y return x else return y。C++用int max(int x,int y)，Python用def my_max(x,y): return x if x>y else y。理解函数签名（参数名+类型+返回值）是设计函数的第一步。

技巧 C++的三元运算符：return x>y?x:y。Python: return x if x>y else y。两种语法看似不同，语义完全一样——条件→值1→值2。

C++

```
#include <iostream>

using namespace std;

int max(int x, int y)
{
    if (x > y) return x;
    return y;
}

int main()
{
    int x, y;
    cin >> x >> y;

    cout << max(x, y) << endl;

    return 0;
}
```

Python

```
x, y = map(int, input().split())
if x > y:
    print(x)
else:
    print(y)
```

NQ085 最大公约数 AcWing 808

小嘉从南洋经商归来，带回了一道古老的问题："两个正整数，找到能同时整除它们的最大数——这就是《九章算术》里的'更相减损术'，也就是欧几里得算法。"小华在屏幕上写道："gcd(a,b)=gcd(b,a%b)，一直递归到b=0时返回a。这是两千年前的算法，至今仍然是求GCD最高效的方法。Python的math.gcd()内部用的就是它。"

输入 一行，两个整数a和b。

输出 一个整数，a和b的最大公约数。

范围 $1 \leq a, b \leq 10^9$

输入

12 18

输出

6

思路 欧几里得算法：while b: a,b = b,a%b; return a。Python的a,b=b,a%b一行完成交换和取余。C++用while(b){int t=a%b;a=b;b=t;}。时间复杂度 $O(\log \min(a,b))$ ——对 10^9 只需约30次运算。

技巧 Python一行gcd：while b: a,b = b, a%b。这个"元组解包+赋值"的写法是Python递推算法的标志性代码。math.gcd()底层是C实现，更快但手写更理解原理。

C++

```
#include <iostream>

using namespace std;

int gcd(int a, int b)
{
    for (int i = 1000; i; i -- )
        if (a % i == 0 && b % i == 0)
            return i;
    return -1;
}

int main()
{
    int a, b;
    cin >> a >> b;
    cout << gcd(a, b) << endl;

    return 0;
}
```

Python

```
a, b = map(int, input().split())
while b:
    a, b = b, a % b
print(a)
```

NQ086 交换数值 AcWing 811

"交换两个变量的值——这是编程的'hello world'动作。"小华说，"C++需要传引用（&）或者用指针，否则函数内的交换不会影响外部变量。Python的函数参数是'传对象引用'——列表可变会受影响，整数不可变不会。但Python有更优雅的方式：a,b=b,a——直接元组解包交换，不需要函数！"

输入

一行，两个整数x和y。

输出

一行，交换后的两个整数（用空格分开）。

范围

 $-10^9 \leq x, y \leq 10^9$

输入

3 5

输出

5 3

思路 C++的难点：值传递vs引用传递。void swap(int &a,int &b){int t=a;a=b;b=t;}中的&是关键——没有&则函数内的交换不影响调用处。Python直接用x,y=y,x一行交换，无需定义函数。理解这个差异是理解两种语言参数传递模型的关键。

技巧 Python的a,b=b,a是元组解包——等式右边先创建元组(b,a)，然后解包赋值给a和b。这是Python最优雅的语法糖之一。C++用std::swap(a,b)最简单。

C++

```
#include <iostream>

using namespace std;

void swap(int& x, int& y)
{
    if (x == y) return;

    int t = x;
    x = y;
    y = t;
}

int main()
{
    int x, y;
    cin >> x >> y;
    swap(x, y);

    cout << x << ' ' << y << endl;

    return 0;
}
```

Python

```
a,b=map(int,input().split());a,b=b,a;
print(a,b)
```

NQ087 打印数字 AcWing 812

"我想写一个函数print_array(arr, size)，传入一个数组和它的大小，自动打印所有元素。"小鲁跃跃欲试。小华说："C++中数组作为参数时退化为指针，必须显式传size。Python的list自带长度信息，不需要额外参数。这个差异体现了两种语言的设计哲学：C追求零开销，Python追求方便安全。"

输入 第一行n和size，第二行n个数。

输出 打印前size个元素，空格分隔。

范围 $1 \leq \text{size} \leq n \leq 1000$

输入	输出
5 3	1 2 3
1 2 3 4 5	

思路 C++数组参数退化为指针——void print(int a[], int size)。必须传size。Python的list自带__len__，for x in arr自动遍历。数组传参是理解C指针与Python引用差异的典型示例。

技巧 C++中int a[]作参数等同于int* a——丢失了长度信息。这就是为什么C风格的数组操作几乎都带size参数。Python没有这个问题——每个list都知道自己的长度。

C++

```
#include <iostream>

using namespace std;

const int N = 1010;

void print(int a[], int size)
{
    for (int i = 0; i < size; i ++ )
        cout << a[i] << ' ';
    cout << endl;
}

int main()
{
    int n, size;
    int a[N];

    cin >> n >> size;
    for (int i = 0; i < n; i ++ ) cin >> a[i];

    print(a, size);

    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
n = int(data[0])
size = int(data[1])
arr = list(map(int, data[2:2+n]))
print(' '.join(map(str, arr[:size])))
```

NQ088 打印矩阵 AcWing 813

"升级了！现在要打印一个二维矩阵。"小栋把题目升级。小华说："C++需要指定第二维的大小——void print(int a[][100], int row, int col)。注意：只有第一维可以省略，第二维必须指定（编译器需要知道每行的大小来做地址计算）。Python就简单多了——list of lists，遍历即打印。"

输入 第一行row和col，然后row行col列。

输出 按行列格式打印矩阵。

范围 $1 \leq \text{row}, \text{col} \leq 100$

输入	输出
2 3	1 2 3
1 2 3	4 5 6
4 5 6	

思路 C++二维数组参数：void print(int a[][100], int r, int c)。第二维必须指定大小。Python: def print_matrix(mat): for row in mat: print(*row)——键解包输出。

技巧 Python的print(*row)用*解包列表——等价于print(row[0],row[1],row[2])。这是Python打印列表最优雅的写法。C++的二维数组参数中第二维大小是编译器常量限制——动态大小的矩阵需要用vector或动态数组。

C++

```
#include <iostream>

using namespace std;

void print2D(int a[][100], int row, int col)
{
    for (int i = 0; i < row; i ++ )
    {
        for (int j = 0; j < col; j ++ )
            cout << a[i][j] << ' ';
        cout << endl;
    }
}

int main()
{
    int a[100][100];

    int row, col;

    cin >> row >> col;
    for (int i = 0; i < row; i ++ )
        for (int j = 0; j < col; j ++ )
            cin >> a[i][j];

    print2D(a, row, col);

    return 0;
}
```

Python

```
r,c=map(int,input().split())
for _ in range(r):print(' '.join(input().split()))
```

NQ089 递归求阶乘 AcWing 819

"之前我们用循环写阶乘——现在用递归！"小华说，"递归就是函数自己调用自己。fact(n)=n×fact(n-1)，fact(0)=1。代码只有3行，但理解它需要'递归思维'——假设fact(n-1)已知，你只需要乘上n。"小鲁试着在纸上展开fact(4)→4×fact(3)→4×3×fact(2)→...→24。"确实优雅！但感觉比循环多用了空间。"

输入 一个整数n。

输出 n的阶乘。

范围 $0 \leq n \leq 10$

输入

4

输出

24

思路 递归三板斧：1.基条件if n==0 return 1；2.递归调用fact(n-1)；3.组合结果n*fact(n-1)。递归需要调用栈——每层递归占用栈空间。循环O(1)空间，递归O(n)栈空间——这是递归的代价。

技巧 Python递归深度默认限制1000层（可改）。递归优雅但对深度大的问题可能栈溢出（Stack Overflow——是的，那个著名的程序员问答网站名字就这么来的）。小n用递归，大n用循环。

C++

```
#include <iostream>

using namespace std;

int fact(int n)
{
    if (n == 1) return 1;
    return n * fact(n - 1);
}

int main()
{
    int n;
    cin >> n;
    cout << fact(n) << endl;

    return 0;
}
```

Python

```
import sys
sys.setrecursionlimit(200000)
def f(n):return 1 if n<=1 else n*f(n-1)
print(f(int(input())))
```

NQ090 递归求斐波那契数列 AcWing 820

" $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ ——多优雅的递归定义！"小鲁兴奋地写下了递归版的斐波那契。"但是 $n=40$ 时我的电脑卡住了！"小华严肃地说："这就是经典的反面教材——朴素的递归斐波那契做了大量重复计算（ $\text{fib}(5)$ 被算了8次！）。解决方法有两种：循环递推或带缓存的递归。Python的@lru_cache装饰器——一行注解，递归变成 $O(n)$ ，魔法般的效果。"

输入

一个整数 n 。

输出

第 n 项斐波那契数。

范围

 $0 \leq n \leq 60$

输入

5

输出

5

思路 朴素递归 $O(2^n)$ ——指数爆炸。加缓存（Python的@lru_cache或手写字典记录已算过的值） $\rightarrow O(n)$ 。递推循环 $O(n)+O(1)$ 空间。递归+缓存和递推循环等价——前者自顶向下（Top-down DP），后者自底向上（Bottom-up DP）。

技巧 Python: from functools import lru_cache; @lru_cache(None); def fib(n): return n if $n < 2$ else $\text{fib}(n-1) + \text{fib}(n-2)$ 。@lru_cache是Python动态规划的核武器——加一行注解，任何纯函数的重复计算问题瞬间解决。

C++

```
#include <iostream>

using namespace std;

int f(int n)
{
    if (n <= 2) return 1;
    return f(n - 2) + f(n - 1);
}

int main()
{
    int n;
    cin >> n;
    cout << f(n) << endl;

    return 0;
}
```

Python

```
import sys
sys.setrecursionlimit(200000)
def f(n):return n if n<=1 else f(n-1)+f(n-2)
print(f(int(input())))
```

NQ091 跳台阶 AcWing 821

小鲁在操场上练习跳台阶——每次可以跳1级或2级。"10级台阶有多少种不同的跳法？"小华问。小鲁想了想："到第n级的跳法=到第n-1级的跳法（跳1级上来）+到第n-2级的跳法（跳2级上来）。这不就是斐波那契数列吗？"小华笑了："没错！这就是为什么递推思维如此重要——表面上不同的问题，底层可能是同一个数学模型。"

输入 一个整数n。

输出 跳法总数。

范围 $1 \leq n \leq 15$

输入

4

输出

5

思路 $f(n)=f(n-1)+f(n-2)$, $f(1)=1, f(2)=2$ 。和斐波那契的递推公式一样但基条件不同（ $f(2)=2$ 而非1）。注意 $n \leq 15$ 时int足够。Python一行：a,b=1,2;for _ in range(n-1):a,b=b,a+b;print(a)。

技巧 这是最简单的DP入门题。学会识别"可以从前一步/前两步转移过来"的模式。这种"状态转移"思维将贯穿整个算法学习——背包、最长子序列、编辑距离……本质都是这个框架。

C++

```
#include <iostream>

using namespace std;

int n;
int ans;

void f(int k)
{
    if (k == n) ans ++ ;
    else if (k < n)
    {
        f(k + 1);
        f(k + 2);
    }
}

int main()
{
    cin >> n;
    f(0);
    cout << ans << endl;

    return 0;
}
```

Python

```
from functools import lru_cache
@lru_cache(None)
def f(n):return n if n<=2 else f(n-1)
+f(n-2)
n=int(input());print(f(n))
```

NQ092 走方格 AcWing 822

小栋画了一个 $n \times m$ 的网格：从 $(0,0)$ 走到 (n,m) ，每次只能向右或向下走一步——有多少种不同的走法？"小鲁兴奋地说："这不就是组合数 $C(n+m, n)$ 吗？总共要走 $n+m$ 步，选其中 n 步向右。"小华补充道："对——但如果你想用代码算，递推公式 $f[i][j]=f[i-1][j]+f[i][j-1]$ 更通用。Python一行`math.comb(n+m, n)`搞定——标准库的强大之处。"

输入

一行，两个整数 n 和 m 。

输出

走法总数。

范围

 $1 \leq n, m \leq 10$

输入

2 3

输出

10

思路 组合数 $C(n+m, n) = (n+m)! / (n! \times m!)$ 。Python: `math.comb(n+m, n)`一行。递推DP: $f[i][j]=f[i-1][j]+f[i][j-1]$; $f[0][*]=f[*][0]=1$ 。两种思维都正确——前者用数学知识，后者用DP框架。

技巧 Python 3.8+的`math.comb(n,k)`计算组合数——比手写阶乘除法更安全（避免了整数溢出）。C++中 $n+m \leq 20$ 时可用long存组合数。

C++

```
#include <iostream>

using namespace std;

int n, m;
int ans;

void dfs(int x, int y)
{
    if (x == n && y == m) ans ++ ;
    else
    {
        if (y < m) dfs(x, y + 1);
        if (x < n) dfs(x + 1, y);
    }
}

int main()
{
    cin >> n >> m;
    dfs(0, 0);
    cout << ans << endl;

    return 0;
}
```

Python

```
import math;n,m=map(int,input().split())
print(math.comb(n+m,n))
```

NQ093 排列 AcWing 823

"用递归生成1到n的所有排列——这是搜索算法的入门课。"小华在白板上画出搜索树："第1层选第1个数（n种选择），第2层选第2个数（n-1种选择）……一直到选完n个数。每次递归深入一层，用used数组标记哪些数已被用了。这个回溯框架将贯穿你的整个搜索算法学习——从排列到N皇后，从数独到旅行商问题。"

输入

一个整数n。

输出

1~n的所有排列，每行一个排列。

范围

 $1 \leq n \leq 9$

输入

3

输出

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

思路 DFS回溯：dfs(path, used)，当len(path)==n时输出。每次从未被used标记的数中选一个加入path，递归完成后撤销选择（回溯）。Python的itertools.permutations(range(1,n+1))一行搞定——库函数也是用类似的算法实现的。

技巧 Python: `from itertools import permutations; for p in permutations(range(1,n+1)): print(*p)`。手写回溯用递归+used数组，itertools用C实现更快。理解回溯框架>会调库——因为库不能解决所有搜索问题。

C++

```
#include <cstdio>
const int N=10;
int n,path[N];
bool st[N];
void dfs(int u) {
    if(u==n) {
        for(int i=0;i<n;i++) {
            if(i) printf(" ");
            printf("%d",path[i]);
        }
        printf("\n");
        return;
    }
    for(int i=1;i<=n;i++) {
        if(!st[i]) {
            path[u]=i; st[i]=true;
            dfs(u+1); st[i]=false;
        }
    }
}
int main() {
    scanf("%d",&n);
    dfs(0);
    return 0;
}
```

Python

```
from itertools import permutations
n = int(input())
for p in permutations(range(1, n + 1)):
    print(' '.join(map(str, p)))
```

NQ094 数组排序 AcWing 818

"这一章的最后一道题——写一个函数把数组排序。"小华说，"当然Python有`sort()`，C++有`std::sort()`，但手写排序算法的意义在于——理解不同排序的思想：选择排序每次选最小的放前面，冒泡排序相邻比较交换，插入排序像整理扑克牌……这些思维训练让你的算法直觉不断提升。"

输入

第一行 n 和 $size$ ，第二行 n 个数。

输出

排序后的前 $size$ 个元素。

范围

$1 \leq size \leq n \leq 1000$

输入

5 3
3 1 4 1 5

输出

1 1 3

思路 多种排序可选：选择排序（找最小放前面）、冒泡排序（相邻交换）、插入排序（扑克牌式）。复杂度 $O(n^2)$ 对 $n \leq 1000$ 足矣。Python: `arr.sort()`或`sorted(arr)`一行。C++: `sort(arr,arr+n)`使用快速排序 $O(n \log n)$ 。

技巧 Python的`sort()`是Timsort（归并+插入的混合算法），C++的`std::sort()`是Introsort（快速排序+堆排序的

混合)。标准库的排序经过数十年优化——实际开发中绝不要手写排序。

C++

```
#include <iostream>

using namespace std;

void sort(int a[], int l, int r)
{
    for (int i = l; i <= r; i++)
        for (int j = i + 1; j <= r; j++)
            if (a[j] < a[i])
                swap(a[i], a[j]);
}

int main()
{
    int a[1000];
    int n, l, r;
    cin >> n >> l >> r;
    for (int i = 0; i < n; i++) cin >> a[i];

    sort(a, l, r);

    for (int i = 0; i < n; i++) cout << a[i] << ' ';

    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
n, l, r = int(data[0]), int(data[1]),
int(data[2])
a = list(map(int, data[3:3+n]))
a[l:r+1] = sorted(a[l:r+1])
print(' '.join(map(str, a)))
```

本章知识点总结

- **函数三价值**：封装（隐藏细节）、复用（一次编写多次调用）、抽象（更高层次的思考）
- **参数传递差异**：C++值vs引用(&)，Python"传对象引用"——理解这个差异是写出正确函数的前提
- **递归三板斧**：基条件→递归调用→组合结果。注意栈溢出风险和重复计算问题
- **@lru_cache**：一行注解将指数级递归变成多项式级——Python动态规划的核武器
- **回溯框架**：选→递归→撤销。排列问题是搜索算法的入门课，这个框架贯穿整个算法学习
- **标准库优先**：math.gcd/factorial/comb, itertools.permutations, functools.lru_cache——先查库再动手

指针与抽象

第8章 · 结构体、指针与STL容器 · 10题 · C++ & Python 双语对照

小鲁已经掌握了函数和递归。但小华告诉他："真正的工程代码很少只有简单的int和数组——你需要**结构体**把多个数据捆在一起，需要**指针**让数据之间产生链接，需要**标准容器**处理复杂的数据组织需求。"小鲁深吸一口气："所以.....编程世界才刚刚开始展开？"

本章10道题从三个角度切入：**链表基础操作**（NQ096-101）——替换空格到链表合并，学会用指针/引用操作链式结构；**结构体与排序**（NQ102）——C++ struct vs Python tuple, lambda vs 比较函数；**函数重载与数组操作**（NQ103-105）——多态思维和大数组处理。学完本章，你将从"会写代码"升级到"理解编程语言的设计哲学"。

本章角色

-  **小鲁**（初学者）· 第一次接触指针和链表，从"为什么需要这个"到"原来如此巧妙"
-  **小华**（算法高手）· 用两个栈实现队列、反转链表三指针——经典面试题的活字典
-  **小栋**（工程师）· 用"合并两叠排序好的扑克牌"把归并思想讲得生动易懂
-  **小嘉**（校主精神）· 阐释抽象的力量："抽象不是复杂度，抽象是管理的艺术"

C++ vs Python 结构体/容器对比

- **结构体/类**: C++ struct/class → Python class/namedtuple/dataclass
- **链表**: C++指针操作 → Python用list模拟（重在学习原理）
- **栈/队列**: C++ stack/queue → Python list/deque
- **函数重载**: C++同名不同参 → Python鸭子类型天然多态
- **排序**: C++ sort+cmp → Python sorted+lambda

NQ096 替换空格 AcWing 16

小鲁在写一个URL编码程序："网络请求中的空格必须转成%20....."他在循环里逐个字符检查。小华走过来说："这道题看似简单，但考察的是字符串构建的思维。C++中逐字符拼接用+= '%20'，Python直接用replace(' ','%20')一行搞定。注意——不是打印，而是构建一个新字符串返回。理解'构建新对象'和'原地修改'的区别，是进阶编程的重要分水岭。"

- 输入** 一个字符串（可能含空格）。
- 输出** 空格替换为%20后的字符串。
- 范围** $0 \leq \text{字符串长度} \leq 1000$

输入

We are happy.

输出

We%20are%20happy.

思路 字符串构建：遍历每个字符，是空格就追加%20，否则追加原字符。Python: `s.replace(' ', '%20')`一行。C++: `for(char c:str) res+=(c==' '?"%20":string(1,c))`。注意构建新字符串而非原地修改。

技巧 Python的`replace()`返回新字符串（不可变性）。C++的`std::string`支持`+=`拼接。这道题也是理解Python字符串不可变性的好例子——所有的'修改'其实都是'创建新版'。

C++

```
class Solution {
public:
    string replaceSpaces(string &str) {
        string res;
        for (auto c : str)
            if (c == ' ') res += "%20";
            else res += c;
        return res;
    }
};
```

Python

```
import sys
for line in sys.stdin:
    s = line.rstrip('\n')
    if not s:
        continue
    res = []
    for c in s:
        if c == ' ':
            res.append('%20')
        else:
            res.append(c)
    print(''.join(res))
```

NQ097 从尾到头打印链表 AcWing 17

小鲁第一次接触链表："这些节点通过next指针连在一起——只能从头沿着指针走到尾，不能反向走。那怎么能从尾到头打印呢？"小华说："聪明的做法——先遍历链表把值存到数组里，然后翻转数组输出。Python一行`arr[::-1]`翻转。栈的LIFO（后进先出）特性天然适合'反向输出'——这个思维后续会用在表达式求值、括号匹配等场景。"

输入

链表节点值序列，以-1结束。

输出

从尾到头的节点值，每行一个。

范围

链表长度 ≤ 1000

输入

1 2 3 -1

输出

3
2
1

思路 先遍历链表收集值→翻转数组→输出。本质是用数组模拟栈（先入后出）。Python: `arr[::-1]`翻转。C++: `reverse(res.begin(), res.end())`。如果面试要求不用额外空间，可以递归（利用函数调用栈天然的后进先出）。

技巧 Python翻转：arr[::-1]（切片）或arr.reverse()（原地）。链表是基础数据结构中最重要的一种——理解指针/引用的跳转模型是理解所有链式结构（树、图）的前提。

C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:

    vector<int> printListReversingly(ListNode* head) {
        vector<int> res;
        if (!head) return res;
        auto p = head;
        while(p) {
            res.push_back(p->val);
            p=p->next;
        }
        reverse(res.begin(), res.end());
        return res;
    }
};
```

Python

```
import sys
for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    nums = list(map(int, line.split(
)))
    nums.pop() # remove -1
    for x in reversed(nums):
        print(x)
```

NQ098 用两个栈实现队列 AcWing 20

"栈是后进先出（LIFO），队列是先进先出（FIFO）——怎么用两个栈模拟一个队列？"小栋把这个经典面试题抛给了小鲁。小华在纸上画起来："第一个栈负责push（入队），第二个栈负责pop（出队）。当需要pop时，如果第二个栈为空，就把第一个栈的所有元素倒进第二个栈——顺序自然就反过来了。这就是计算机科学中最巧妙的设计模式之一。"

输入 一系列操作指令（push x / pop / empty）。

输出 对pop操作输出弹出的值。

范围 操作数 ≤ 100

输入	输出
push 1	1
push 2	2
pop	3
push 3	empty!
pop	
pop	
empty	

思路 双栈模拟队列：stack1负责push，stack2负责pop。pop时若stack2为空则将stack1全部倒入stack2（翻转顺序）。时间复杂度：每个元素最多入栈2次出栈2次，均摊 $O(1)$ 。这是理解“均摊分析”的经典案例。

技巧 Python用list作栈：append()=push, pop()=pop。两个list分别模拟两个栈。虽然Python有collections.deque，但理解手写实现比调用库更重要——面试常考题。

C++

```
class MyQueue {
public:
    stack<int> stk, cache;
    /** Initialize your data structure here. */
    MyQueue() {}

    /** Push element x to the back of queue. */
    void push(int x) {
        stk.push(x);
    }

    void copy(stack<int>& a, stack<int>& b){
        while (a.size()) {
            b.push(a.top());
            a.pop();
        }
    }

    /** Removes the element from in front of queue and returns that element. */
    int pop() {
        copy(stk, cache); // 拷贝到cache数组
        int res = cache.top();
        cache.pop();
        copy(cache, stk);
        return res;
    }

    /** Get the front element. */
    int peek() {
        copy(stk, cache); // 拷贝到cache数组
        int res = cache.top();
        copy(cache, stk);
        return res;
    }

    /** Returns whether the queue is empty. */
    bool empty() {
        return stk.empty();
    }
};

/**
 * Your MyQueue object will be instantiated and called as such:
 * MyQueue obj = MyQueue();
 * obj.push(x);
 * int param_2 = obj.pop();
 * int param_3 = obj.peek();
 * bool param_4 = obj.empty();
 */
```

Python

```
import sys

stack1 = []
stack2 = []

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    parts = line.split()
    if parts[0] == 'push':
        stack1.append(int(parts[1]))
    elif parts[0] == 'pop':
        if not stack2:
            while stack1:
                stack2.append(stack1.pop())
        if stack2:
            print(stack2.pop())
    elif parts[0] == 'peek':
        if not stack2:
            while stack1:
                stack2.append(stack1.pop())
        if stack2:
            print(stack2[-1])
    elif parts[0] == 'empty':
        print('true' if (not stack1 and not stack2) else 'false')
```

NQ099 斐波那契数列(类) AcWing 21

"这次我们用面向对象的方式实现斐波那契!"小华说,"定义一个类/结构体,用成员变量存储状态。C++的class或struct都可以——区别是struct默认public, class默认private。Python的class简单直观。注意N=39时斐波那契数已经超过6千万, C++要用long long存储。"

输入 一个整数n。
输出 第n项斐波那契数。
范围 $0 \leq n \leq 39$

输入	输出
5	5

思路 用类封装: C++ `class Solution { public: int Fibonacci(int n) {...} }`。递推法 $O(n)$, $a, b = b, a + b$ 模式。 $n \leq 39$ 时int够用 ($F(39) \approx 6.3 \times 10^7 < 2^{31}$)。 $n \geq 40$ 需long long。

技巧 Python的类不需要声明成员变量类型——构造函数__init__中直接赋值。C++的struct和class唯一区别是默认访问权限。此题的核心是理解"将算法封装成类的方法"——OO思想的萌芽。

C++

```
class Solution {
public:
    int Fibonacci(int n) {
        if (n <= 1) return n;
        return Fibonacci(n - 1) + Fibonacci(n - 2);
    }
};
```

Python

```
n = int(input())
a, b = 0, 1
for _ in range(n):
    a, b = b, a + b
print(a)
```

NQ100 反转链表 AcWing 35

"把链表反转—— $1 \rightarrow 2 \rightarrow 3 \rightarrow \text{null}$ 变成 $3 \rightarrow 2 \rightarrow 1 \rightarrow \text{null}$ 。"小华在白板上画箭头,"关键操作:把每个节点的next指针指向前一个节点。你需要三个指针——prev (前一个)、curr (当前)、next (下一个,暂存防止断链)。每次迭代:保存下一个→当前指向prev→prev和curr前移。"小鲁试了三遍才理解——"原来指针重新定向这么容易出错,一步顺序都不能乱!"

输入 链表节点值序列,以-1结束。
输出 反转后的链表节点值,每行一个。
范围 链表长度 ≤ 1000

输入	输出
1 2 3 -1	

3
2
1

思路 迭代反转：prev=null, curr=head; while(curr): next=curr->next; curr->next=prev; prev=curr; curr=next。三指针法——prev(已反转部分)、curr(当前)、next(暂存)。注意顺序：必须先保存next再改curr->next，否则链表就断了。

技巧 反转链表是所有链表操作的基础——合并链表、回文链表、环形链表……都建立在对指针重定向的理解之上。Python虽然没有指针，但用list模拟链表时理解原理同样重要。

C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        if (!head || !head->next) return head;

        auto p = head, q = p->next;
        while (q)
        {
            auto o = q->next;
            q->next = p;
            p = q, q = o;
        }
        head->next = NULL;

        return p;
    }
};
```

Python

```
import sys
text = sys.stdin.read().strip()
parts = text.split(">")
values = [p for p in parts if p != "NULL"]
values.reverse()
print(">".join(values) + ">NULL")
```

NQ101 合并两个排序链表 AcWing 36

"现在有两个已经从小到大排好序的链表——把它们合并成一个排好序的链表。"小华说，"这就是归并排序中的'归并'步骤。用一个虚拟头节点dummy简化边界处理，然后比较l1和l2的当前值，较小的接入新链表。"小鲁恍然："这就像两叠从小到大排好的扑克牌——每次比较各自最上面那张，小的拿出来放到新牌堆。"

输入 两行，每行链表节点值序列（以-1结束）。

输出 合并后链表节点值，空格分隔。

范围 链表长度 ≤ 1000

输入

输出


```
1 3 5 -1
2 4 6 -1
```

```
1 2 3 4 5 6
```

思路 双指针归并：dummy头简化逻辑，tail跟随。每次比较l1->val和l2->val，较小的接入tail->next，tail后移。最后把剩余链表接入。时间O(n+m)，空间O(1)。Python用list模拟：while i

技巧 用dummy头节点是链表题的常用技巧——避免处理空链表这种边界情况。合并是归并排序(merge sort)的核心操作——后续会学到完整的归并模板。

C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    ListNode* merge(ListNode* l1, ListNode* l2) {
        auto dummy = new ListNode(-1), tail = dummy;
        while (l1 && l2)
            if (l1->val < l2->val)
            {
                tail->next = l1;
                l1 = l1->next;
            }
            else
            {
                tail->next = l2;
                l2 = l2->next;
            }

            if (l1) tail->next = l1;
            if (l2) tail->next = l2;

            return dummy->next;
    }
};
```

Python

```
import sys
lines = sys.stdin.read().strip().split('\n')
a = [int(x) for x in lines[0].split("->") if x != "NULL"]
b = [int(x) for x in lines[1].split("->") if x != "NULL"]
i = j = 0
res = []
while i < len(a) and j < len(b):
    if a[i] <= b[j]:
        res.append(str(a[i]))
        i += 1
    else:
        res.append(str(b[j]))
        j += 1
res.extend(str(x) for x in a[i:])
res.extend(str(x) for x in b[j:])
print("->".join(res))
```

NQ102 三元组排序 AcWing 862

"给你N个三元组(x,y,z)，按x从小到大排序——这就是结构体排序的经典题。"小华说，"C++中定义一个struct Triple{int x; double y; string z;}，然后sort+自定义比较函数。Python更优雅——用list of tuples，sorted(arr, key=lambda t: t[0])一行排序。这就是为什么Python在数据处理领域如此流行——一行lambda替代C++十几行代码。"

输入

第一行N，然后N行每行三个值。

输出

按x升序输出三元组（y保留2位小数）。

范围

 $1 \leq N \leq 100$

输入

```
3
1 3.5 abc
2 1.2 def
1 2.8 ghi
```

输出

```
1 3.50 abc
1 2.80 ghi
2 1.20 def
```

思路 结构体/元组排序。C++: struct+sort+cmp; Python: sorted(arr, key=lambda x: x[0])。Python的lambda是匿名函数——快速定义比较逻辑而不需要单独定义函数。理解排序的稳定性（x相同时保持原序）。

技巧 Python: sorted(arr, key=lambda t: t[0])。lambda是Python函数式编程的利器——它就是一个只能写一行表达式的匿名函数。C++可用lambda: [](auto &a, auto &b){return a.x

C++

```
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 10010;

struct data {
    int x;
    double y;
    string z;
    bool operator< (const data & t) const
    {
        return x < t.x;
    }
} a[N];

int main() {
    int n;
    cin >> n;
    for (int i = 0; i < n; i++) cin >> a[i].x >> a[i].y >> a[i].z;
    //结构体排序
    sort(a, a+n);
    //输出结果
    for (int i = 0; i < n; i++)
        printf("%d %.2lf %s\n", a[i].x, a[i].y, a[i].z.c_str());
    return 0;
}
```

Python

```
n = int(input())
data = []
for _ in range(n):
    parts = input().split()
    x = int(parts[0])
    y = float(parts[1])
    z = parts[2]
    data.append((x, y, z))
data.sort(key=lambda t: t[0])
for x, y, z in data:
    print(f'{x} {y:.2f} {z}')
```

NQ103 绝对值 AcWing 810

"定义一个abs函数——但有一个要求：支持int和double两种类型的参数。"小华说，"这就是函数重载（Overloading）。C++可以写两个同名的abs函数——C++根据参数类型自动选择正确的版本。Python不需要重

载——它是动态类型，同一个函数可以接收任何支持<0和-操作的类型。两种语言的设计哲学差异在这里体现得淋漓尽致。"

输入 一个整数或浮点数。

输出 其绝对值。

范围 $-10^9 \leq x \leq 10^9$

输入	输出
-5	5

思路 C++函数重载：int abs(int x)和double abs(double x)可以共存。编译器根据参数类型自动选择。Python不需要——def abs(x): return -x if x<0 else x就行（鸭子类型）。Python内置abs()已经是多态的。

技巧 C++函数重载的核心：同名不同参。编译器根据参数类型/数量自动匹配。Python通过鸭子类型（duck typing）实现类似效果——只要对象支持所需操作就行。

C++

```
#include <iostream>

using namespace std;

int abs(int x)
{
    if (x > 0) return x;
    return -x;
}

int main()
{
    int x;
    cin >> x;
    cout << abs(x) << endl;

    return 0;
}
```

Python

```
x = int(input())
print(x if x >= 0 else -x)
```

NQ104 复制数组 AcWing 814

"写个函数把数组a的内容复制到数组b——但要求用函数封装，且复制后b的内容是a的完全副本。"小华强调，"C++中数组传参会退化为指针——void copy(int a[], int b[], int size)。Python的list直接b=a[:]或b=a.copy()。但注意Python的浅拷贝陷阱——如果list里嵌套了list，需要用copy.deepcopy。"

输入 第一行n和size，第二行n个数。

输出 复制后的数组（空格分隔）。

范围 $1 \leq \text{size} \leq n \leq 1000$

输入

5 3
1 2 3 4 5

输出

1 2 3

思路 数组复制函数：遍历赋值 $b[i]=a[i]$ 。C++用for循环或memcpy，Python用切片 $a[:]$ （浅拷贝）。理解浅拷贝vs深拷贝——对于int这种不可变对象，浅拷贝安全；对于嵌套list需要用copy.deepcopy。

技巧 Python切片 $a[:]$ 创建新列表——最常用的复制方式。list.copy()和list(a)也创建浅拷贝。C++的memcpy是原始内存复制——对非POD类型不安全，现代C++推荐std::copy。

C++

```
#include <iostream>
#include <cstring>

using namespace std;

const int N = 110;

void copy(int a[], int b[], int size)
{
    memcpy(b, a, size * 4);
}

int main()
{
    int a[N], b[N];
    int n, m, size;
    cin >> n >> m >> size;
    for (int i = 0; i < n; i++) cin >> a[i];
    for (int i = 0; i < m; i++) cin >> b[i];

    copy(a, b, size);

    for (int i = 0; i < m; i++) cout << b[i] << ' ';
    cout << endl;

    return 0;
}
```

Python

```
n, m, size = map(int, input().split())
a = list(map(int, input().split()))
b = list(map(int, input().split()))
b[:size] = a[:size]
print(' '.join(map(str, b)))
```

NQ105 数组翻转 AcWing 816

"最后一道——把数组的前size个元素翻转。"小栋说。小华点头："这可以验证你对数组操作、函数传参和对称交换的理解。标准做法是从两端向中间交换——arr[0]和arr[size-1]交换，arr[1]和arr[size-2]交换……直到中间相遇。Python用arr[:size::-1]切片翻转，C++用reverse或手写交换循环。"

输入

第一行n和size，第二行n个数。

输出

翻转前size个元素后的数组。

范围

$1 \leq \text{size} \leq n \leq 1000$

输入

5 3
1 2 3 4 5

输出

3 2 1 4 5

思路 部分翻转：for i in range(size//2): swap(arr[i], arr[size-1-i])。只交换前size个元素，后面的不动。时间 $O(\text{size}/2)$ ，空间 $O(1)$ 。Python用arr[:size]=arr[size-1::-1]结合切片赋值。

技巧 Python切片翻转arr[::-1]最简洁但创建新列表。原地翻转用reverse()或手写交换。理解'对称交换'是理解更多高级翻转（旋转、循环移位）的基础。

C++

```
#include <iostream>

using namespace std;

void reverse(int a[], int size)
{
    for (int i = 0, j = size - 1; i < j; i ++, j -- )
        swap(a[i], a[j]);
}

int main()
{
    int a[1000];
    int n, size;

    cin >> n >> size;
    for (int i = 0; i < n; i ++ ) cin >> a[i];
    reverse(a, size);

    for (int i = 0; i < n; i ++ ) cout << a[i] << ' ';

    return 0;
}
```

Python

```
n, size = map(int, input().split())
a = list(map(int, input().split()))
a[:size] = reversed(a[:size])
print(' '.join(map(str, a)))
```

本章知识点总结

- **链表五操作**：遍历、反向输出、反转、合并、两栈模拟队列——每个都是面试高频题
- **三指针法**：prev/curr/next——反转链表的标准模板，一步顺序都不能乱
- **dummy头节点**：合并链表时用虚拟头简化边界——链表题的通用技巧
- **结构体排序**：C++ struct+sort+cmp vs Python tuple+lambda——一行lambda替代十几行C++
- **函数重载vs鸭子类型**：C++编译期多态 vs Python运行期多态——两种语言哲学的核心差异
- **厦大精神**：自强不息。指针和链表是CS专业的标志——突破这道坎，你就真正进入了计算机科学的世界

分治之美

第9章 · 排序与二分 · 7题 · C++ & Python 双语对照

从本章开始，小鲁正式进入**算法**的世界。前面的语法和数据结构是工具，算法才是让程序"聪明"起来的灵魂。"排序——把混乱变成有序——是计算机科学中最基础也最深刻的主题。快排、归并、二分……这些经典算法经过几十年的打磨，至今仍是每个程序员的基本功。"小华严肃地说。

本章7道题分三个板块：**排序模板**（快排+归并）——理解分治思想并将模板变成肌肉记忆；**排序应用**（快速选择+逆序对）——在排序过程中顺便解决更复杂的问题；**二分查找**（整数+实数）——利用有序性将搜索从 $O(n)$ 降到 $O(\log n)$ ，这是算法优化的第一思维。

本章角色

-  **小鲁**（进阶者）· 从语法期毕业，进入算法训练营——分治、排序、二分
-  **小华**（算法队长）· "快排模板写100遍——背下来、默出来、成为肌肉记忆"
-  **小栋**（工程师）· "为什么不能直接调用sort()?"——理解算法原理和工程实践的关系

⚡ 排序与二分核心模板

- **快排模板**：取中值基准→do-while双指针→交换→递归 $[l, j]$ 和 $[j+1, r]$
- **归并模板**：二分递归→双指针合并→复制回原数组。稳定 $O(n \log n)$
- **整数二分**：左边界 $mid = l + r >> 1$, $r = mid$ 。右边界 $mid = l + r + 1 >> 1$, $l = mid$
- **实数二分**：while($r - l > \text{eps}$) $mid = (l + r) / 2$ 。比整数二分简单——不需处理+1/-1
- **Python**：sort()/sorted()快速排序，bisect_left/right二分。标准库是工程首选

NQ106 快速排序 AcWing 785

"排序——这是计算机科学里研究得最透彻的问题之一。"小华郑重地说，"给你 n 个乱序的整数，用快速排序把它们排好。快排的核心思想是'分治'——选一个基准数，把比它小的放左边，大的放右边，然后左右分别递归排序。这个模板你会写100遍——背下来，默出来，成为肌肉记忆。"小鲁试了试："do-while循环和指针的配合确实巧妙，不小心就会无限循环……"

- 输入** 第一行整数 n ，第二行 n 个整数。
- 输出** 升序排列的 n 个整数，空格分隔。
- 范围** $1 \leq n \leq 100000$ ，所有整数在 $1 \sim 10^9$ 范围内

输入

5
3 1 2 4 5

输出

1 2 3 4 5

思路 快排模板：选基准 $x = arr[l+r \gg 1]$ ， $i = l - 1, j = r + 1$ 双指针向中间移动， i 找 $\geq x$ 的， j 找 $\leq x$ 的，交换，直到 $i \geq j$ ，然后递归 $[l, j]$ 和 $[j+1, r]$ 。时间 $O(n \log n)$ 平均，最坏 $O(n^2)$ 。注意边界：用do-while避免死循环，用 j 而非 $i-1$ 防止无限递归。

技巧 快排背板要点：1)取中间值作基准避免最坏情况；2) i 和 j 初始为 $l-1$ 和 $r+1$ 配合do-while；3)递归用 j 和 $j+1$ （不用 i ）。Python版`sorted()`内部用Timsort，实践中永远比手写快排好。但手写快排是理解分治思想的必修课。

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;

long long *numbers;

void quicksort(long long nums[], int left, int right)
{
    if (left >= right)
        return;
    //选取分割数
    long long target = nums[left]; //选取第一个快排的分割数
    int i = left - 1, j = right + 1;
    while (i < j)
    {
        do
            i++;
        while (nums[i] < target); //i指针定位
        do
            j--;
        while (nums[j] > target); //j指针定位
        if (i < j)
            swap(nums[i], nums[j]); //交换
    }
    //分治
    quicksort(nums, left, j); //j左边的都是不比Target大的
    quicksort(nums, j + 1, right); // j右边的都比Target大
}

int main()
{
    //!cin\cout提速
    ios::sync_with_stdio(false); //来打消iostream的输入输出缓存
    cin.tie(0); //cin.tie(0)来解除cin与cout的绑定,0表示NULL

    int n;
    //创建动态数组
    cin >> n;
    numbers = new long long[n];
    for (int i = 0; i < n; i++)
        cin >> numbers[i];
    quicksort(numbers, 0, n - 1); //注意边界

    for (int i = 0; i < n; i++)
        cout << numbers[i] << " ";

    delete numbers;
    return 0;
}

```

Python

```

# 快速排序
def quicksort(arr, l, r):
    if l >= r:
        return
    x = arr[(l + r) // 2]
    i, j = l - 1, r + 1
    while i < j:
        i += 1
        while arr[i] < x:
            i += 1
        j -= 1
        while arr[j] > x:
            j -= 1
        if i < j:
            arr[i], arr[j] = arr[j], arr[i]
    arr[i]
    quicksort(arr, l, j)
    quicksort(arr, j + 1, r)

n = int(input())
arr = list(map(int, input().split()))
quicksort(arr, 0, n - 1)
print(' '.join(map(str, arr)))

```

NQ107 第k个数 AcWing 786

"如果是找第k小的数——而不是排序整个数组呢？"小栋问。小华说："用快排的变种——快速选择算法。每次划分后，看第k个数落在左半边还是右半边，只递归那一边。平均 $O(n)$ 时间找到第k小，比先排序 $O(n \log n)$ 再取第k个快。这就是'不需要的信息就不处理'的思维——算法优化的本质就是只做必要的工作。"

输入 第一行n和k，第二行n个整数。

输出 第k小的数。

范围 $1 \leq n \leq 100000$, $1 \leq k \leq n$

输入	输出
5 3	3
2 4 1 5 3	

思路 快速选择：在快排划分基础上，设左半长度为 $sl=j-l+1$ 。若 $k \leq sl$ 则在左半递归找第k小；否则在右半递归找第 $k-sl$ 小。平均 $O(n)$ ，最坏 $O(n^2)$ （可通过随机选基准优化）。Python直接用`nth_element`或排序后取`arr[k-1]`。

技巧 快速选择的核心是`partition`后的决策——只递归一边。这和二分查找的思想一样：每一步排除一半候选。最坏 $O(n^2)$ 可以用随机化基准数来规避——C++的`nth_element`就是这个算法。

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;

long long *numbers;

long long quickFind(long long nums[], int left, int right, int k)
{
    if (left == right)
        return nums[left]; //只有一个元素，左开右闭
    //选取分割数
    long long target = nums[left]; //选取第一个快排的分割数
    int i = left - 1, j = right + 1;
    while (i < j)
    {
        do
            i++;
        while (nums[i] < target); //i指针定位
        do
            j--;
        while (nums[j] > target); //j指针定位
        if (i < j)
            swap(nums[i], nums[j]); //交换
    }
    //判断i与k的关系
    int sumofLeft = j - left + 1; //计算左边区间数字个数，用于与k比较

    if (k <= sumofLeft) //分治
        quickFind(nums, left, j, k); //j左边的都是不比Target大的
    else
        quickFind(nums, j + 1, right, k - sumofLeft); //j右边的都比Target大
}

int main()
{
    //!cin\cout提速
    ios::sync_with_stdio(false); //来打消iostream的输入输出缓存
    cin.tie(0); //cin.tie(0)来解除cin与cout的绑定,0表示NULL

    int n, k;
    //创建动态数组
    cin >> n >> k;
    numbers = new long long[n];
    for (int i = 0; i < n; i++)
        cin >> numbers[i];
    //查找第k小数，输出之
    cout << quickFind(numbers, 0, n - 1, k) << endl; //注意边界

    delete numbers;
    return 0;
}

```

Python

```

import sys

def quick_select(nums, left, right, k):
    if left == right:
        return nums[left]
    target = nums[left]
    i, j = left - 1, right + 1
    while i < j:
        i += 1
        while nums[i] < target:
            i += 1
        j -= 1
        while nums[j] > target:
            j -= 1
        if i < j:
            nums[i], nums[j] = nums[j], nums[i]
    sl = j - left + 1
    if k <= sl:
        return quick_select(nums, left, j, k)
    else:
        return quick_select(nums, j + 1, right, k - sl)

data = sys.stdin.read().split()
n, k = int(data[0]), int(data[1])
arr = list(map(int, data[2:2 + n]))
print(quick_select(arr, 0, n - 1, k))

```

NQ108 归并排序 AcWing 787

"除了快排，还有一种经典排序——归并排序。"小华画出一个分叉图，"先把数组不断对半分，分到只有一个元素（自然有序），然后从下往上两两合并有序子数组。归并排序的优点是稳定且时间复杂度始终 $O(n \log n)$ ，缺点是需要额外 $O(n)$ 空间。快排vs归并——这是算法课永恒的话题。"

输入 第一行 n ，第二行 n 个整数。

输出 升序排列的 n 个整数。

范围 $1 \leq n \leq 100000$

输入

5
3 1 2 4 5

输出

1 2 3 4 5

思路 归并三步骤：1)递归划分 $mid=l+r>>1$ ， $merge_sort(l,mid)$ 和 $merge_sort(mid+1,r)$ ；2)双指针合并两个有序子数组到tmp；3)复制tmp回原数组。稳定排序，时间 $O(n \log n)$ ，空间 $O(n)$ 。逆序对问题可以天然嵌入归并过程。

技巧 归并是稳定排序——相等元素的相对位置不变。快排不稳定。Python用`sorted()`（稳定），C++用`stable_sort`（归并）和`sort`（快排）。理解归并对后续学习逆序对、CDQ分治至关重要。

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;
const int N = 100007;
int n;
int numbers[N], tmp[N];

void mergeSort(int nums[], int left, int right)
{
    if (left >= right) return;

    int mid = left + (right - left) / 2; // 防止溢出

    mergeSort(nums, left, mid), mergeSort(nums, mid + 1, right); // 先调用归并排序
    // 合二为一
    int k = 0, i = left, j = mid + 1; // 左右两部分数组的起点

    while (i <= mid && j <= right) // 扫描还未抵达终点
        if (nums[i] <= nums[j]) // 把小的放到tmp数组里面
            tmp[k++] = nums[i++];
        else tmp[k++] = nums[j++];

    while (i <= mid) tmp[k++] = nums[i++]; // 把左边剩余的部分插入tmp
    while (j <= right) tmp[k++] = nums[j++]; // 把右边剩余的部分插入tmp

    // 把tmp的部分复制回数组
    for (i = left, k = 0; i <= right; i++, k++) nums[i] = tmp[k];
}

int main()
{
    // 数据大, 使用printf和scanf
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &numbers[i]);

    mergeSort(numbers, 0, n - 1);

    for (int i = 0; i < n; i++) printf("%d ", numbers[i]);

    return 0;
}

```

Python

```

# 归并排序
def merge_sort(arr, l, r):
    if l >= r:
        return
    mid = (l + r) // 2
    merge_sort(arr, l, mid)
    merge_sort(arr, mid + 1, r)
    tmp = []
    i, j = l, mid + 1
    while i <= mid and j <= r:
        if arr[i] <= arr[j]:
            tmp.append(arr[i])
            i += 1
        else:
            tmp.append(arr[j])
            j += 1
    tmp.extend(arr[i:mid + 1])
    tmp.extend(arr[j:r + 1])
    arr[l:r + 1] = tmp

n = int(input())
arr = list(map(int, input().split()))
merge_sort(arr, 0, n - 1)
print(' '.join(map(str, arr)))

```

NQ109 逆序对的数量 AcWing 788

"统计数组中逆序对的数量——即 $a[i]$ 的对数。"小华说, "暴力两重循环 $O(n^2)$ 太慢。巧妙的方法? 在归并排序的合并过程中统计! 当左半的 $a[i]$ 大于右半的 $a[j]$ 时, 左半从 i 到 mid 的所有数都大于 $a[j]$, 逆序对增加 $mid - i + 1$ 个。排序+统计一气呵成。注意答案可能超过 int 范围——用 $long\ long$!"

输入 第一行 n , 第二行 n 个整数。

输出 逆序对总数。

范围 $1 \leq n \leq 100000$, 答案可能超 int

输入

```
6
2 3 4 5 6 1
```

输出

```
5
```

思路 归并排序嵌入计数：当 $a[i] > a[j]$ 时， $\text{cnt} += \text{mid} - i + 1$ （左半剩余元素全部形成逆序对）。归并过程中自然完成了逆序对统计。时间复杂度 $O(n \log n)$ ，比暴力 $O(n^2)$ 快4个数量级（ $n=10^5$ 时）。逆序对数量是衡量数组“混乱程度”的指标。

技巧 C++必须用long long存储cnt—— $n=10^5$ 时逆序对最多约 5×10^9 超过int。Python自带大整数。这道题是归并排序最重要的应用之一——理解‘在归并过程中顺便统计’是分治算法的精髓。

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;
typedef long long LL;

const int N = 100007;
int n;
int numbers[N], tmp[N];
//最大的数可能会是 n^2/2
LL mergeSort(int nums[], int left, int right)
{
    if (left >= right) return 0; //如果左指针越过右指针, 则返回

    int mid = left + (right - left) / 2; //防止溢出

    LL result = mergeSort(nums, left, mid) + mergeSort(nums, mid + 1, right); //先调用递归并排序
    //合二为一
    int k = 0, i = left, j = mid + 1; //左右两部分数组的起点

    while (i <= mid && j <= right) //扫描还未抵达终点
        if (nums[i] <= nums[j]) //把小的放到tmp数组里面
            tmp[k++] = nums[i++];
        else {
            //计算逆序对的个数mid-i+1
            result += mid - i + 1;
            tmp[k++] = nums[j++];
        }

    while (i <= mid) tmp[k++] = nums[i++]; //把左边剩余的部分插入tmp
    while (j <= right) tmp[k++] = nums[j++]; //把右边剩余的部分插入tmp

    //把tmp的部分复制回数组
    for (i = left, k = 0; i <= right; i++, k++) nums[i] = tmp[k];

    return result;
}

int main()
{
    //数据大, 使用printf和scanf
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &numbers[i]);

    cout << mergeSort(numbers, 0, n - 1) << endl;

    return 0;
}

```

Python

```

import sys
sys.setrecursionlimit(200000)
def merge(a, l, r):
    if l >= r: return 0
    m = (l + r) // 2
    cnt = merge(a, l, m) + merge(a, m + 1, r)

    tmp = []
    i, j = l, m + 1
    while i <= m and j <= r:
        if a[i] <= a[j]: tmp.append(a[i]); i += 1
        else: tmp.append(a[j]); j += 1
    cnt += m - i + 1
    tmp.extend(a[i:m + 1]); tmp.extend(a[j:r + 1])
    a[l:r + 1] = tmp
    return cnt
data = sys.stdin.read().split()
n = int(data[0])
arr = list(map(int, data[1:1 + n]))
print(merge(arr, 0, n - 1))

```

NQ110 数的范围 AcWing 789

"在一个升序数组里, 某个数出现了多次——请找出它第一次和最后一次出现的位置。"小栋出题。小鲁说: "遍历一遍就行!" 小华摇头: "数组有 10^5 个元素, 查询 10^4 次—— $O(n \times q)$ 要 10^9 次操作, 超时。需要用二分

—— $O(\log n)$ 每次查询。二分的精髓：用mid+1或mid不停缩小范围，直到找到边界。整数二分的边界处理是90%的人都会写错的地方。”

输入 第一行n和q，第二行n个升序整数，然后q行每行一个查询k。

输出 每个查询输出k的起始和终止位置，不存在输出-1 -1。

范围 $1 \leq n \leq 100000$, $1 \leq q \leq 10000$

输入	输出
6 3	3 4
1 2 2 3 3 4	1 2
3	-1 -1
2	
5	

思路 二分查找左右边界：左边界→while(l>1; if(a[mid]>=k) r=mid else l=mid+1。右边界→while(l>1; if(a[mid]<=k) l=mid else r=mid-1。注意右边界二分的mid要+1避免死循环。Python可用bisect_left和bisect_right。

技巧 整数二分最关键的差异：左边界mid=l+r>>1，右边界mid=l+r+1>>1（不+1会死循环）。记住：l=mid时mid要+1，r=mid时不用。二分是OI/ACM中最基础也是最容易写错的算法——花时间理解透彻值得。

C++

```

#include <iostream>
#include <algorithm>
using namespace std;
const int N = 100007;

int nums[N];

int main()
{
    // 第一行包含整数n和q，表示数组长度和询问个数。
    int n, q;
    scanf("%d%d", &n, &q);
    // 第二行包含n个整数（均在1~10000范围内），表示完整数组。
    for (int i = 0; i < n; i++)
        scanf("%d", &nums[i]);
    // 接下来q行，每行包含一个整数k，表示一个询问元素。
    while(q--){
        //读入要查找的数，并且二分查找其范围
        int k;
        scanf("%d",&k);// 读入k

        int left = 0, right = n-1;//初始化left和right
        while (left < right) //模板2
        {
            int mid = left + right>>1;
            if (nums[mid]>=k) right = mid;//模板1,
            else left = mid +1;
        }
        //left就是左端点值
        //判断是否找得到
        if (nums[left]!=k) cout<<"-1 -1"<<endl;//找不到x
        else
        {
            cout<<left<<" ";
            //继续寻找右边界
            left=0, right=n-1;//重新初始化left,riight,进行第二
            次二分搜索
            while (left < right)
            {
                int mid = left+right+1>>1;
                if (nums[mid]<=k) left=mid;
                else right = mid -1;
            }
            cout<<left<<endl;
        }
    }

    return 0;

    // int bsearch_1(int left, int right)
    // {
    //     while (left < right)
    //     {
    //         int mid = left + right >> 1;
    //         if (check(mid)) right = mid;
    //         else left = mid + 1;
    //     }
    //     return left;
    // }

```

Python

```

import sys
import bisect
data = sys.stdin.read().split()
n, q = int(data[0]), int(data[1])
a = list(map(int, data[2:2+n]))
idx = 2 + n
out = []
for _ in range(q):
    x = int(data[idx]); idx += 1
    l = bisect.bisect_left(a, x)
    r = bisect.bisect_right(a, x) - 1
    if l < n and a[l] == x: out.append(f'{l} {r}')
    else: out.append('-1 -1')
print('\n'.join(out))

```



```
// int bsearch_2(int left, int right)
// {
//     while (left < right)
//     {
//         int mid = left + right + 1 >> 1;
//         if (check(mid)) left = mid;
//         else right = mid - 1;
//     }
//     return left;
// }
```

NQ111 数的三次方根 AcWing 790

"整数二分搞定了，那实数二分呢？"小鲁问。"比如计算 n 的三次方根，精确到 10^{-6} 。"小华说，"实数二分比整数二分简单——不需要处理 $+1/-1$ 的边界。设定精度阈值（如 $1e-8$ ）， $\text{while}(r-l>1e-8)$ 不断对半分。浮点二分的优势：不需要纠结 mid 的取值，收敛极快—— $\log_2(10^8)\approx 27$ 步即可到目标精度。"

- 输入** 一个浮点数 n 。
- 输出** n 的三次方根，保留6位小数。
- 范围** $-10000 \leq n \leq 10000$

输入
1000.00

输出
10.000000

思路 实数二分：设 $l=-10000, r=10000$ （范围包含答案）， $\text{while}(r-l>1e-8)$: $\text{mid}=(l+r)/2$; $\text{if}(\text{mid}^3 \geq n)$ $r=\text{mid}$ else $l=\text{mid}$ 。输出 r 保留6位小数（ $\text{printf}("%.6lf", r)$ ）。收敛步数 $=\log_2(\text{范围}/\text{精度}) \approx \log_2(20000/1e-8) \approx 41$ 步——非常快。

技巧 实数二分不需要处理整数二分的 $\text{mid}+1$ 问题。直接 $\text{mid}=(l+r)/2$ 即可。精度通常比要求多两位（要求 $1e-6$ 则迭代到 $1e-8$ ）。C++用`double`，Python用`float`。注意 n 为负数时三次方根也为负。

C++

```
#include <iostream>
#include <algorithm>
using namespace std;

int main()
{
    double number;
    scanf("%lf", &number);
    double left = -10000, right = 10000;
    while (right - left >= 1e-8)
    {
        double mid = (right + left) / 2; //二分
        if (mid * mid * mid >= number)
            right = mid;
        else
            left = mid;
    }
    //输出left
    printf("%lf", left);
}
```

Python

```
# 数的三次方根
n = float(input())
l, r = -10000.0, 10000.0
while r - l > 1e-8:
    mid = (l + r) / 2.0
    if mid ** 3 >= n:
        r = mid
    else:
        l = mid
print(f'{r:.6f}')
```

NQ112 菱形 AcWing 727

"最后来一道'画图题'放松一下——打印一个菱形！"小华说，"看起来简单，但其实考察的是曼哈顿距离的几何直觉。菱形中心到四边的曼哈顿距离是 $n/2$ ，每个位置如果到中心距离 $\leq n/2$ 就填*否则填空格。这道题把几何和格式化输出结合起来——是算法竞赛中'观察规律→数学建模→代码实现'的完美微型案例。"

输入

一个奇数 n 。

输出

 n 行菱形图案，中心行有 n 个*。

范围

 $1 \leq n \leq 100$ ， n 为奇数

输入

5

输出

```
  *
 ***
*****
 ***
  *
```

思路 曼哈顿距离模型：设中心 $cx=cy=n/2$ 。对于每个位置 (i,j) ，如果 $abs(i-cx)+abs(j-cy) \leq n/2$ 则填*否则空格。这种'距离阈值'的判断方式在心形、沙漏等图案题中都通用。时间复杂度 $O(n^2)$ 。

技巧 曼哈顿距离 $=|x_1-x_2|+|y_1-y_2|$ 。菱形就是曼哈顿距离 $\leq$ 阈值的点集。换欧几里得距离就是圆形。理解这种几何距离与图形形状的关系，是计算机图形学的基础。

C++

```
#include <iostream>
#include <cmath>

using namespace std;

int main()
{
    int n;
    cin >> n;

    int cx = n / 2, cy = n / 2;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
            if (abs(i - cx) + abs(j - cy) <= n / 2) cout <<
'*';
        else cout << ' ';
        cout << endl;
    }

    return 0;
}
```

Python

```
import sys
n = int(sys.stdin.read().split()[0])
cx = cy = n // 2
for i in range(n):
    row = []
    for j in range(n):
        if abs(i-cx) + abs(j-cy) <=
n//2: row.append('*')
        else: row.append(' ')
    print(''.join(row))
```

本章知识点总结

- 快排模板： $O(n \log n)$ 平均。do-while+双指针+分治递归。非稳定排序
- 归并模板： $O(n \log n)$ 保证。稳定排序。额外 $O(n)$ 空间。可在合并过程统计逆序对
- 快速选择：基于快排划分， $O(n)$ 平均找第k小。只递归一边——“不需要的信息就不处理”
- 整数二分：左边界和右边界模板不同——注意mid是否需要+1避免死循环
- 实数二分：无需处理+1/-1边界。迭代到目标精度+2位即可
- 厦大精神：自强不息。把这些模板练成肌肉记忆——它们是你算法之路的基石

预处理的智慧

第10章 · 前缀和、差分与双指针 · 7题 · C++ & Python 双语对照

小鲁上节课学会了排序——把混乱变有序。这节课要学的是：有了有序性或预处理之后，如何以光速回答问题。小华说："排序之后数组有序了，二分查找 $O(\log n)$ ；但如果你经常要查区间和呢？每次都要 $O(n)$ 累加还是太慢——所以有了前缀和， $O(1)$ 回答。有了前缀和就有差分——修改一个区间只需两次操作。有了滑动窗口就有双指针——两个指针各走一遍解决 $O(n^2)$ 的问题。这些技巧的核心思想都是：预处理和挖掘有序结构。"

本章角色

-  小鲁（进阶者）· 学习用预处理换取查询速度——"计算一次，查询千次"
-  小华（算法队长）· 画图解释二维前缀和的容斥原理、差分矩阵的四角操作
-  小栋（工程师）· 带真实场景来问：大数据量的区间查询和快速修改

1 2 3 4 前缀和·差分·双指针 核心模板

- 一维前缀和： $s[i]=s[i-1]+a[i]$ ，查询 $[l,r]=s[r]-s[l-1]$
- 二维前缀和： $s[i][j]=s[i-1][j]+s[i][j-1]-s[i-1][j-1]+a[i][j]$ （容斥模式）
- 一维差分： $b[l]+=c$ ， $b[r+1]-=c$ （区间修改 $O(1)$ ）
- 二维差分：正角 $(x1,y1)+c$ ， $(x2+1,y2+1)+c$ ；负角 $(x1,y2+1)-c$ ， $(x2+1,y1)-c$
- 滑动窗口：维护 $[l,r]$ 区间，while条件移动l，答案不断更新
- 对撞指针：i从左向右，j从右向左，根据条件判断移动哪个

NQ113 前缀和 AcWing 795

小栋拿着一个巨大的数组找到小鲁："我要查询100000次——每次求第l到第r个元素的和。直接每次循环累加的话要 $100000 \times 50000 = 50$ 亿次操作， $O(n \times q)$ 太慢了！"小华接过键盘："前缀和——预处理一个前缀数组 $s[i]=$ 前i个元素的和，查 $[l,r]$ 的和只需 $s[r]-s[l-1]$ ， $O(1)$ 时间。用 $O(n)$ 的预处理换 $O(1)$ 的查询——这就是'预处理思维'的第一课。"

输入 第一行n和m，第二行n个整数，然后m行每行l r。

输出 每行输出 $a[l..r]$ 的和。

范围 $1 \leq n, m \leq 100000$

输入	输出
5 3	3
2 1 3 6 4	6
1 2	10

```
1 3
2 4
```

思路 预处理前缀数组: $s[i]=s[i-1]+a[i]$ (1-indexed)。查询 $[l,r]=s[r]-s[l-1]$, $O(1)$ 。Python用accumulate。C++注意使用1-indexed数组简化边界处理。前缀和的核心思想: 把重复计算变成一次计算+多次查表。

技巧 用1-indexed数组避免l-1越界—— $s[0]=0$ 。Python: $s=\text{list}(\text{accumulate}([0]+\text{arr}))$ 。前缀和可以扩展到异或和、乘积和、最大值/最小值 (需要线段树)。理解预处理思维是算法优化的核心。

C++

```
#include <iostream>

using namespace std;
const int N = 1000007;
int n, m;

int a[N], s[N];

int main()
{
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; i++)
        scanf("%d", &a[i]);
    s[0] = 0; //刻意初始化s[0]为0, 此句可以不写, 写也无妨
    for (int i = 1; i <= n; i++)
        s[i] = s[i - 1] + a[i]; //打表法
    //m次询问
    while (m--)
    {
        int l, r;
        scanf("%d%d", &l, &r);
        printf("%d\n", s[r] - s[l - 1]); //求区间和, 直接从s
        //中读取数值
    }

    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
a = list(map(int, data[2:2+n]))
s = [0]*(n+1)
for i in range(1,n+1): s[i] = s[i-1] + a[i-1]
idx = 2 + n
out = []
for _ in range(m):
    l, r = int(data[idx]), int(data[idx+1])
    idx += 2
    out.append(str(s[r] - s[l-1]))
print('\n'.join(out))
```

NQ114 子矩阵的和 AcWing 796

"前缀和能不能推广到二维?" "小鲁问。" "当然!" "小华在白板上画出矩阵, "二维前缀和: $s[i][j]=s[i-1][j]+s[i][j-1]-s[i-1][j-1]+a[i][j]$ ——包含关系的加减。查询子矩阵 $(x1,y1)$ 到 $(x2,y2)$ 的和 = $s[x2][y2]-s[x1-1][y2]-s[x2][y1-1]+s[x1-1][y1-1]$ 。看这个'容斥'模式——包含-排除+补回。"

输入 第一行 n,m,q 。然后 $n \times m$ 矩阵。然后 q 行每行 $x1 \ y1 \ x2 \ y2$ 。

输出 每行子矩阵元素和。

范围 $1 \leq n,m \leq 1000, 1 \leq q \leq 200000$

输入

输出

3 4 3	17
1 7 2 4	27
3 6 2 8	21
2 1 2 3	
1 1 2 2	
2 1 3 4	
1 3 3 4	

思路 二维前缀和：预处理 $s[i][j]$ 包含从 $(1,1)$ 到 (i,j) 的矩形和。公式如上述。查询子矩阵和=大矩形减两翼加回重叠部（容斥原理）。 $O(nm)$ 预处理 $\rightarrow O(1)$ 每次查询。Python同样用accumulate思想。

技巧 注意预处理和查询的加减符号—— $s[i][j]=s[i-1][j]+s[i][j-1]-s[i-1][j-1]+a[i][j]$ （包含关系）。查询时=大矩形-上矩形-左矩形+左上矩形（容斥）。1-indexed简化边界。理解这个模式后三维前缀和也是同样的加减法。

C++

```
#include <iostream>
using namespace std;
const int N = 1007;
int n, m, q;
int a[N][N], s[N][N]; //子矩阵

int main()
{
    //读入矩阵
    scanf("%d%d%d", &n, &m, &q);
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            scanf("%d", &a[i][j]);

    s[0][0] = 0; //刻意初始化s[0]为0，此句可以不写，写也无妨
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            s[i][j] = s[i-1][j] + s[i][j-1] - s[i-1][j-1] + a[i][j]; //打表法
    //q次询问
    while (q--)
    {
        int x1, x2, y1, y2;
        scanf("%d%d%d%d", &x1, &y1, &x2, &y2);
        //求子矩阵和，直接从s中读取数值
        printf("%d\n", s[x2][y2] - s[x2][y1-1] - s[x1-1][y2] + s[x1-1][y1-1]);
    }
    return 0;
}
```

Python

```
import sys

data = sys.stdin.buffer.read().split()
n, m, q = int(data[0]), int(data[1]), int(data[2])
idx = 3

a = [[0] * (m + 1) for _ in range(n + 1)]
for i in range(1, n + 1):
    for j in range(1, m + 1):
        a[i][j] = int(data[idx])
        idx += 1

s = [[0] * (m + 1) for _ in range(n + 1)]
for i in range(1, n + 1):
    for j in range(1, m + 1):
        s[i][j] = s[i-1][j] + s[i][j-1] - s[i-1][j-1] + a[i][j]

out = []
for _ in range(q):
    x1, y1, x2, y2 = int(data[idx]), int(data[idx+1]), int(data[idx+2]), int(data[idx+3])
    idx += 4
    ans = s[x2][y2] - s[x1-1][y2] - s[x2][y1-1] + s[x1-1][y1-1]
    out.append(str(ans))
sys.stdout.write('\n'.join(out))
```

NQ115 差分 AcWing 797

"前缀和是前缀 $\rightarrow$ 查询。那它的逆运算是什么？"小华问。"如果我对数组的 $[l,r]$ 区间每个元素都加 c ，怎么最快？"小鲁想了想："差分！构造 b 数组使 a 是 b 的前缀和。对 $[l,r]$ 加 c 只需 $b[l]+=c$, $b[r+1]-=c$ ——两次操作！对整

个区间每个元素加同一个值——这是差分最经典的用法。 $O(1)$ 修改， $O(n)$ 还原。”

输入 第一行 n, m 。第二行 n 个整数。然后 m 行每行 $l\ r\ c$ 。

输出 所有操作后的数组。

范围 $1 \leq n, m \leq 100000$

输入

```
6 3
1 2 2 1 2 1
1 3 1
3 5 1
1 6 1
```

输出

```
3 4 5 3 4 2
```

思路 差分数组 b : $b[i]=a[i]-a[i-1]$ (用 $a[i]$ 的差分量), 则 $a[i]=b[1]+\dots+b[i]$ 。区间 $[l, r]$ 加 c : $b[l]+=c, b[r+1]-=c$ 。最后做前缀和还原。这是'区间修改 $\rightarrow$ 点查询'的最优方案—— $O(1)$ 修改 $O(n)$ 还原。

技巧 差分=前缀和的逆运算。核心操作: $b[l]+=c, b[r+1]-=c$ 。Python实现可直接用数组操作。理解差分的妙处: 两次修改影响整个区间。竞赛中差分和前缀和是黄金搭档——90%的区间问题都可以用这俩之一解决。

C++

```
#include <iostream>
using namespace std;
const int N = 100007;
int n, m;
int a[N], b[N];

//差分的插入公式
void insert(int l, int r, int c)
{
    b[l] += c;
    b[r + 1] -= c;
}

int main()
{
    cin >> n >> m;
    //读入数组
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    //构造差分数组b
    for (int i = 1; i <= n; i++)
        insert(i, i, a[i]);
    //输出查询结果
    while (m--)
    {
        int l, r, c;
        cin >> l >> r >> c;
        insert(l, r, c);
    }
    //重新复原a数组(前缀和)
    for (int i = 1; i <= n; i++)
        a[i] = a[i - 1] + b[i];
    //输出a
    for (int i = 1; i <= n; i++)
        cout << a[i] << " ";
    cout << endl;
    return 0;
}
```

Python

```
# 差分
n, m = map(int, input().split())
a = [0] + list(map(int, input().split()))
b = [0] * (n + 2)
for i in range(1, n + 1):
    b[i] = a[i] - a[i - 1]
for _ in range(m):
    l, r, c = map(int, input().split())
    b[l] += c
    b[r + 1] -= c
res = []
cur = 0
for i in range(1, n + 1):
    cur += b[i]
    res.append(str(cur))
print(' '.join(res))
```

NQ116 差分矩阵 AcWing 798

"一维差分搞定了——二维呢?"小栋继续追问。小华画出矩阵:"二维差分——对子矩阵(x1,y1)到(x2,y2)加c只需四步: $b[x1][y1] += c$, $b[x2+1][y1] -= c$, $b[x1][y2+1] -= c$, $b[x2+1][y2+1] += c$ 。注意四个角——加主对角减副对角。最后做二维前缀和还原。两次操作,影响整个子矩阵——这就是数学建模的优雅。"

输入 第一行n,m,q。然后n×m矩阵。然后q行每行x1 y1 x2 y2 c。

输出 所有操作后的矩阵。

范围 $1 \leq n, m \leq 1000, 1 \leq q \leq 100000$

输入

```
3 4 3
1 2 2 1
3 2 2 1
```

输出

```
2 3 4 1
4 3 4 1
2 2 2 2
```



```
1 1 1 1
1 1 2 2 1
1 3 2 3 2
3 1 3 4 1
```

思路 二维差分： $b[i][j]$ 使 $a[i][j]=b[1..i][1..j]$ 的和。子矩阵加 c 的四步操作如上述——两个正角 $(x1,y1$ 和 $x2+1,y2+1)$ 两个负角 $(x2+1,y1$ 和 $x1,y2+1)$ 。最后二维前缀和还原。 $O(1)$ 修改 $\rightarrow O(nm)$ 还原。

技巧 记忆四角公式：正正负负。左上角 $(x1,y1)+c$ ，右上角 $(x1,y2+1)-c$ ，左下角 $(x2+1,y1)-c$ ，右下角 $(x2+1,y2+1)+c$ 。二维差分比一维多了一对操作。这是图像处理中滤镜效果的基础——用极少操作影响大区域。

C++

```

#include <iostream>
using namespace std;
const int N = 1007;
int n, m, q;
int a[N][N], b[N][N]; //子矩阵
//二维差分核心操作
void insert(int x1, int y1, int x2, int y2, int c)
{
    b[x1][y1] += c;
    b[x1][y2 + 1] -= c;
    b[x2 + 1][y1] -= c;
    b[x2 + 1][y2 + 1] += c;
}
int main()
{
    //读入矩阵
    scanf("%d%d%d", &n, &m, &q);
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            scanf("%d", &a[i][j]);

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            insert(i, j, i, j, a[i][j]); //构造二维差分矩阵

    //q次询问
    while (q--)
    {
        int x1, x2, y1, y2, c;
        scanf("%d%d%d%d%d", &x1, &y1, &x2, &y2, &c);
        //差分核心操作
        insert(x1, y1, x2, y2, c);
    }
    //重建a[][],即求二维前缀和
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            a[i][j] = a[i - 1][j] + a[i][j - 1] - a[i - 1][j - 1] + b[i][j];
    //输出a[][]
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++)
            cout << a[i][j] << " ";
        cout << endl;
    }
    return 0;
}

```

Python

```

# 差分矩阵
n, m, q = map(int, input().split())
a = [[0] * (m + 1)]
for _ in range(n):
    a.append([0] + list(map(int, input().split())))
b = [[0] * (m + 2) for _ in range(n + 2)]
for i in range(1, n + 1):
    for j in range(1, m + 1):
        b[i][j] = a[i][j] - a[i - 1][j] - a[i][j - 1] + a[i - 1][j - 1]
for _ in range(q):
    x1, y1, x2, y2, c = map(int, input().split())
    b[x1][y1] += c
    b[x2 + 1][y1] -= c
    b[x1][y2 + 1] -= c
    b[x2 + 1][y2 + 1] += c
for i in range(1, n + 1):
    row = []
    cur = 0
    for j in range(1, m + 1):
        b[i][j] += b[i - 1][j] + b[i][j - 1] - b[i - 1][j - 1]
        row.append(str(b[i][j]))
    print(' '.join(row))

```

NQ117 最长连续不重复子序列 AcWing 799

"给一个数组，找最长的一段，里面的数都不重复。"小华说，"暴力三重循环 $O(n^3)$ 太慢。优化——用双指针！维护窗口 $[l, r]$ ，用计数数组记录窗口内每个数出现多少次。当 $a[r]$ 重复时，不断 $l++$ 直到消除重复。双指针的复杂度是 $O(2n)=O(n)$ ——每个元素最多进窗口一次，出窗口一次。"

输入

第一行 n ，第二行 n 个整数。

输出

最长连续不重复子序列的长度。

范围

 $1 \leq n \leq 100000$

输入

```
5
1 2 2 3 5
```

输出

3

思路 双指针滑动窗口：维护 $[l, r]$ 区间无重复。遍历 r ，每次 $\text{cnt}[a[r]]++$ 。若 $\text{cnt}[a[r]] > 1$ 说明有重复， $\text{while}(\text{cnt}[a[r]] > 1) \text{cnt}[a[l]]--, l++$ 。答案 $= \max(\text{ans}, r-l+1)$ 。时间复杂度 $O(n)$ ——每个元素最多入窗一次离开一次。

技巧 双指针核心：两指针都只向前移动，从不后退——保证 $O(n)$ 。重复检测用计数数组作哈希（值域小时）。Python用字典记录上次出现位置可优化到只移一次。双指针是处理'连续子数组'类问题的常用武器。

C++

```
#include <iostream>
using namespace std;
const int N=100007;
int num[N],visited[N];

int main(){
    int n;
    cin>>n;
    for (int i=0; i<n; i++ ) cin>>num[i]; //读入n个数
    //双指针扫描
    int res=0; //初始化为最小值
    for (int i=0, j=0; i<n; i++){
        visited[num[i]]++;
        while(j<=i && visited[num[i]]>1){
            visited[num[j]]--;
            j++; //去掉重复数
        }
        res= max(res,i-j+1); //j--i 之间的数字个数
    }
    cout<<res<<endl;
    return 0;
}
```

Python

```
# 最长连续不重复子序列
n = int(input())
a = list(map(int, input().split()))
cnt = {}
l = 0
ans = 0
for r in range(n):
    cnt[a[r]] = cnt.get(a[r], 0) + 1
    while cnt[a[l]] > 1:
        cnt[a[l]] -= 1
        l += 1
    ans = max(ans, r - l + 1)
print(ans)
```

NQ118 数组元素的目标和 AcWing 800

"两个升序数组A和B，找一对 $a[i]+b[j]=x$ 。" "如果暴力双循环 $O(n \times m)$ 太慢——但既然两个数组都是升序的....."小鲁已经有了思路，"用双指针！i从A的开头开始向右走，j从B的末尾开始向左走。 $a[i]+b[j]$ 比x大就j--，比x小就i++。对撞双指针——各自走一遍， $O(n+m)$ 。"

输入

第一行 n, m, x 。第二行 n 个升序整数(A)。第三行 m 个升序整数(B)。

输出

i和j (0-indexed)。

范围

$1 \leq n, m \leq 100000$ 。保证有唯一解。

输入

输出

```

4 5 6
1 2 4 7
3 4 6 8 9
1 1

```

思路 对撞双指针：i=0从左，j=m-1从右。当a[i]+b[j]>x则j--（太大，缩右边的），当a[i]+b[j]

技巧 双指针三种模式：滑动窗口（同向）、对撞指针（反向）、快慢指针。本题是对撞指针——从两端向中间逼近。抓住while循环的两个关键：条件(a[i]+b[j] vs x)和移动方向(谁移动)。

C++

```

#include <iostream>
#include <algorithm>
using namespace std;
const int N=100007;
int n,m,x;
int numA[N],numB[N];

int main(){
    //读入n,m,x其中x为目标和
    scanf("%d%d%d",&n,&m,&x);
    for (int i=0; i<n; i++) scanf("%d",&numA[i]);
    for (int j=0; j<m; j++) scanf("%d",&numB[j]);
    //双指针查找和

    for (int i=0, j= m-1; i<n; i++){
        //check(i,j)
        while (j>=0 && numA[i]+numB[j]>x) j--;
        if (numA[i]+numB[j]==x)//判断是否和相等
        {
            printf("%d %d\n",i,j);
            break;
        }
    }
    return 0;
}

```

Python

```

# 数组元素的目标和
n, m, x = map(int, input().split())
a = list(map(int, input().split()))
b = list(map(int, input().split()))
i, j = 0, m - 1
while i < n and j >= 0:
    s = a[i] + b[j]
    if s == x:
        print(i, j)
        break
    elif s < x:
        i += 1
    else:
        j -= 1

```

NQ119 判断子序列 AcWing 2816

"最后一个双指针问题：判断a是不是b的子序列——即a的元素按相同顺序出现在b中（不一定连续）。"小华说，"双指针i指a、j指b。遍历b的每个元素，如果b[j]==a[i]就i++。最后如果i==n说明a的所有元素都按顺序找到了。这个'匹配式'双指针——j不停走，i只在匹配时走——O(n+m)。"

输入 第一行n,m。第二行n个整数(a)。第三行m个整数(b)。

输出 是子序列输出Yes，否则No。

范围 $1 \leq n \leq m \leq 100000$

输入

```

3 5
1 3 5

```

输出

Yes

1 2 3 4 5

思路 匹配双指针：i=0指a，j=0指b。遍历j，if b[j]==a[i]: i++。最后i==n则匹配完成。j一直移动，i只在匹配成功时前移。O(n+m)时间O(1)空间。此模式适用于有序序列的判断和验证问题。

技巧 判断子序列是双指针的经典入门题——理解'只有匹配时才移动i'的节奏。这个模式也是KMP算法的前身——都是字符串/序列匹配，但双指针版更简单直白。竞赛中大量'验证是否满足某约束'的题都在用这个思路。

C++

```
#include <iostream>
#include <cstdio>
using namespace std;
const int N = 1e5+7; //我的风格
int a[N], b[N];

int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    for (int i = 0; i < m; i++) scanf("%d", &b[i]);

    int p = 0; //指向a数组的指针
    for (int i = 0; i < m; i++)
    {
        if (p < n && a[p] == b[i]) p++;
    }
    //完全匹配
    if (p == n) puts("Yes");
    else puts("No");

    return 0;
}
```

Python

```
# 判断子序列
n, m = map(int, input().split())
a = list(map(int, input().split()))
b = list(map(int, input().split()))
i = 0
for x in b:
    if i < n and a[i] == x:
        i += 1
print('Yes' if i == n else 'No')
```

本章知识点总结

- 前缀和 (1D/2D)：O(n)预处理→O(1)查询。"重复查询→预处理"思维的典型应用
- 差分 (1D/2D)：O(1)区间修改→O(n)还原。"修改操作远多于查询"场景的最优方案
- 双指针三模式：滑动窗口、对撞指针、匹配指针——核心是不回溯，O(n)解决O(n²)问题
- 厦大精神：自强不息。这些预处理技巧，在真实工程中（大数据、图像处理）同样大放异彩

计算的边界

第11章 · 高精度、位运算与离散化 · 7题 · C++ & Python 双语对照

小鲁在上个月的语法训练中，抱怨过"C++的int不够大"。小华说："今天你终于遇到了C++的短板——高精度计算。但先别急——这也正是Python最闪耀的时刻：int无精度上限，四则运算零负担。"本章的位运算部分教二进制思维——取最低位、计数、快速幂——底层优化与竞赛高频考点。离散化和区间合并则是空间压缩和贪心的经典案例。

本章角色

 **小鲁**（进阶者）· 第一次深刻感受Python的震撼——高精度四题C++手写100行，Python四行

 **小华**（算法队长）· 讲解lowbit(x&-x)的二进制原理和补码证明

Python震撼教育：高精度与位运算

- **高精度加/减/乘/除**：C++手写30-50行 → Python: `int(input()) (+ - * //)` 一行
- **lowbit**：x & -x 取最低位的1。树状数组的核心操作
- **bit_count**：Python `int.bit_count()` / C++ `__builtin_popcount()`
- **离散化**：收集坐标→排序去重→二分映射→前缀和。处理稀疏大范围的标准手段
- **区间合并**：按左端点排序→贪心遍历→合并重叠。O(n log n)

NQ120 高精度加法 AcWing 791

"C++的int最大21亿，long long最大 9×10^{18} ——但如果你要算 $10^{200} + 10^{200}$ 呢？"小华严肃地说。"这就是高精度计算。C++需要手写大整数——用字符串读入，按位加，处理进位。"小鲁很惊讶："这么基础的操作还要手写？！"小华笑道："对。但这正是Python最震撼的地方——Python的int无精度上限，直接a+b就行。C++手写30行，Python一行。这就是'用对语言'的力量。"

输入 两行，每行一个正整数（长度 ≤ 100000 位）。

输出 两数之和。

范围 数字长度 ≤ 100000

输入

123456789
987654321

输出

1111111110

思路 C++高精度加法：字符串读入→倒序存入vector→按位相加处理进位→倒序输出。核心代码：for i from 0 to max(len): t+=A[i]+B[i]; C.push_back(t%10); t/=10。Python: `print(int(input())+int(input()))`——震撼教育时刻。

技巧 Python的int底层用30-bit数组存储，自动扩展——可以处理任意大的整数（只受内存限制）。这是Python在算法竞赛中的最大优势之一——高精度计算零负担。C++手写高精度是理解数字表示和进位机制的好练习。

C++

```
#include <iostream>
#include <cstring>
#include <vector>
using namespace std;
const int N = 1e6+7;

vector<int> add(vector<int> &A, vector<int> &B){
    vector<int> C;
    int t=0;
    for (int i=0; i<A.size()||i<B.size(); i++) {
        if (i<A.size()) t+=A[i];
        if (i<B.size()) t+=B[i];
        //对t取模，取进位
        C.push_back(t %10); //把t模10的余数入数组，商为进位
        t /= 10; //算出进位，供下一次循环使用
    }
    if(t) C.push_back(1); //最高位进位
    return C;
}

int main(){
    //输入为字符串
    string a,b;
    vector<int> A,B;

    cin >> a>>b; // a="786654"
    //倒序存储A, B, a[0]乃是最低位
    for(int i= a.size()-1; i>=0; i--) A.push_back(a[i]-
'0');
    for(int i= b.size()-1; i>=0; i--) B.push_back(b[i]-
'0');

    auto C = add(A,B);

    for(int i= C.size()-1; i>=0; i--) cout<<C[i]; //打印C
    return 0;
}
```

Python

```
import sys
sys.set_int_max_str_digits(1000000)
data = sys.stdin.read().split()
a = int(data[0])
b = int(data[1])
print(a + b)
```

NQ121 高精度减法 AcWing 792

"加法有进位，减法有借位——高精度减法的核心是借位处理。"小华说，"首先判断A和B谁大，保证大减小。然后逐位减——如果不够减就向高位借1（当前位+10）。最后去掉前导0。C++手写减法比加法多一步：处理借位+前导0。Python——print(int(input())-int(input()))，又是零负担。"

输入

两行，每行一个正整数。

输出

A-B的结果。

范围

数字长度 ≤ 100000

输入

1000

1

输出

999

思路 C++高精度减法：先判断 $A \geq B$ ，若A

技巧 借位处理：if A[i]

C++

```

#include <iostream>
#include <cstring>
#include <vector>
using namespace std;
const int N = 1e6 + 7;
bool isGreater(vector<int> &A, vector<int> &B)
{
    if (A.size() != B.size()) //比较大小
        return A.size() > B.size();
    //否则大小相等，逐位比较大小
    for (int i = A.size() - 1; i >= 0; i--)
        if (A[i] != B[i])
            return A[i] > B[i];
    return true; //每个数字都相同，长度相同，两数相等
}
//C= A-B
vector<int> sub(vector<int> &A, vector<int> &B)
{
    vector<int> C;
    //A是比较大的数
    int t = 0; //第一轮减法进位为0
    for (int i = 0; i < A.size(); i++)
    {
        t = A[i] - t;
        if (i < B.size())
            t -= B[i]; //A[i]-B[i] + t
        C.push_back((t + 10) % 10); //得到对应位
        if (t < 0)
            t = 1;
        else
            t = 0; //判断是否借一位
    }
    //去掉开头的0
    while (C.size() > 1 && C.back() == 0) C.pop_back();

    return C;
}

int main()
{
    //输入为字符串
    string a, b;
    vector<int> A, B;

    cin >> a >> b; // a="786654"
    //倒序存储A, B, a[0]乃是最低位
    for (int i = a.size() - 1; i >= 0; i--)
        A.push_back(a[i] - '0');
    for (int i = b.size() - 1; i >= 0; i--)
        B.push_back(b[i] - '0');
    //判断A与B的大小
    if (isGreater(A, B))
    {
        auto C = sub(A, B);
        for (int i = C.size() - 1; i >= 0; i--)
            cout << C[i]; //打印C
    }
    else
    {
        auto C = sub(B, A);
        cout << "-";
        for (int i = C.size() - 1; i >= 0; i--)

```

Python

```

# 高精度减法
import sys
sys.set_int_max_str_digits(1000000)
a = int(input())
b = int(input())
print(a - b)

```

```

        cout << C[i]; //打印C
    }
    return 0;
}

```

NQ122 二进制中1的个数 AcWing 801

"给你一个整数，求它的二进制表示中1的个数。"小华说，"这就是'汉明重量'——编译器优化和密码学中常用的操作。C++用__builtin_popcount()或while(x) cnt+=x&1, x>>=1。Python 3.8+用x.bit_count()。还有lowbit技巧：x&-x取最低位的1，while(x) x-=x&-x, cnt++——比逐位检查更快。"

输入 一个整数n。

输出 n的二进制表示中1的个数。

范围 $0 \leq n \leq 10^9$

输入	输出
5	2

思路 三种方法：1)逐位检查while(n): cnt+=n&1; n>>=1; 2)lowbit法while(n): n-=n&-n; cnt++——复杂度=1的个数而非位数；3)__builtin_popcount/Python bit_count()一行搞定。lowbit=n&-n是位运算中最精妙的trick之一。

技巧 x&-x取x的最低位的1——证明：负数用补码表示，-x=~x+1，x&~x+1就是最低位1的位置。这个trick在树状数组（Fenwick Tree）中是核心操作。bit_count()是Python 3.8+新特性，C++20有std::popcount。

C++

```

#include <iostream>
using namespace std;
inline int lowbit(int &x)
{
    return x & -x; //x的二进制的最后一个1所构成的整数
}
int main()
{
    int n, x;
    cin >> n;
    while (n--)
    {
        //读入一个数，并且计算1的个数，并且输出
        cin >> x;
        int res = 0; //计数变量
        while (x)
            x -= lowbit(x), res++; //使用,表达式,简化代码去
        括号
        cout << res << " "; //输出运算结果
    }
    return 0;
}

```

Python

```

import sys
data = sys.stdin.read().split()
n = int(data[0])
out = []
for i in range(1, n + 1):
    out.append(str(bin(int(data[i])).count('1')))
sys.stdout.write(' '.join(out) + ' ')

```

NQ123 高精度乘法 AcWing 793

"高精度乘高精度——C++手写要三重循环：A的每一位乘B的每一位，结果加到对应位置。"小华在黑板上演示。小鲁问："那乘法是不是比加减法难很多？""对——加减 $O(n)$ ，朴素乘法 $O(n^2)$ 。还有更快的Karatsuba和FFT算法可以在 $O(n \log n)$ 内完成，但那已经是大数据运算的科研前沿了。Python——`print(int(input())*int(input()))`。"

输入 两行，每行一个正整数。

输出 乘积。

范围 数字长度 ≤ 100000

输入	输出
12	408
34	

思路 C++高精度乘法：两重循环 $C[i+j] += A[i] * B[j]$; $C[i+j+1] += C[i+j] / 10$; $C[i+j] \% = 10$; 最后去前导0。时间复杂度 $O(\text{len}(A) \times \text{len}(B))$ 。Python的int乘法底层用Karatsuba算法优化——对超长整数自动切换，比手写快得多。

技巧 Python int乘法内部根据数字大小自动选择算法：小数字用基础乘法，大数字切到Karatsuba($O(n^{1.585})$)，超大数字用Toom–Cook。这就是为什么Python数据处理生态如此强大——底层的数学运算经过极致优化。

C++

```

#include <iostream>
#include <cstring>
#include <vector>
using namespace std;
const int N = 1e6 + 7;

//C= A*b,t的用法很精彩!
vector<int> mul(vector<int> &A, int b){
    vector<int> C;
    int t=0;
    for(int i=0; i<A.size() || t; i++){
        //如果t还有剩余或者A还有剩余, 乘法结果C需要加新的位
        if(i<A.size()) t+=A[i]*b;//把b当作整体做乘法
        C.push_back(t % 10);//存储第i位
        t /=10; //更新t
    }
    while(C.size()>1 && C.back()==0) C.pop_back();//去掉
    开头的0
    return C;
}

int main()
{
    //输入为字符串
    string a;
    int b;//输入的b比较小, 作为一个整体处理
    vector<int> A,C;

    cin >> a >> b; // a="786654"
    //倒序存储A, B, a[0]乃是最低位
    for (int i = a.size() - 1; i >= 0; i--)
        A.push_back(a[i] - '0');
    C=mul(A,b);
    for (int i = C.size() - 1; i >= 0; i--)
        cout << C[i]; //打印C
    return 0;
}

```

Python

```

# 高精度乘法
import sys
sys.set_int_max_str_digits(1000000)
a = int(input())
b = int(input())
print(a * b)

```

NQ124 高精度除法 AcWing 794

"除法是四则运算中最复杂的——需要从高位到低位逐位试商。"小华说。高精度除以低精度相对简单（逐位除）；高精度除以高精度则需要二分法试商。"好消息是——Python `print(int(input())/int(input()))` 一行搞定，还能同时得到商和余数用 `divmod(a,b)`。高精度四则运算到此全部覆盖——C++手写超越100行，Python四行收工。"

输入

两行，每行一个正整数。

输出

第一行商，第二行余数。

范围

数字长度 ≤ 100000

输入

10
3

输出

3
1

思路 C++高精度除以低精度：从高位到低位， $r=r*10+A[i]$ ； $C.push_back(r/B)$ ； $r\%=B$ 。高精度除以高精度需二分或减法模拟。Python: `divmod(a,b)`返回(商,余数)元组。

技巧 Python高精度总结：`input()`，`int()`，四则运算——零负担。C++手写高精度的意义在于理解底层运算原理——数字如何存储、进位借位如何处理。两种路径都有价值——但竞赛时用Python省下的时间可以做更多题。

C++

```
#include <iostream>
#include <cstring>
#include <vector>
#include <algorithm>

using namespace std;
const int N = 1e6 + 7;

//C= A/b 余数为r
vector<int> div(vector<int> &A, int b, int &r){
    vector<int> C;
    r=0;//余数
    for(int i=A.size()-1; i>=0; i--){
        r = r* 10 + A[i]; //从最高位开始，当前余数r需要上一次运
        算的r乘10
        C.push_back(r/b);
        r %= b;
    }
    reverse(C.begin(), C.end()); //定义的数是最低位在C[0]，因此
    需要取逆
    while(C.size()>1 && C.back()==0) C.pop_back(); //去掉C
    开头的0字符
    return C;
}

int main()
{
    //输入为字符串
    string a;
    int b; //输入的b比较小，作为一个整体处理
    vector<int> A;

    cin >> a >> b; // a="786654"
    //倒序存储A, B, a[0]乃是最低位
    for (int i = a.size() - 1; i >= 0; i--)
        A.push_back(a[i] - '0');

    int r;
    auto C=div(A,b,r); //使用auto自动辨别变量类型

    for (int i = C.size() - 1; i >= 0; i--)
        cout << C[i]; //打印C
    cout<<endl<<r<<endl; //打印余数
    return 0;
}
```

Python

```
import sys
sys.set_int_max_str_digits(1000000)
data = sys.stdin.read().split()
a = int(data[0])
b = int(data[1])
print(a // b)
print(a % b)
```

"有一个无限长的数轴，在几个坐标上加了值。现在要查询若干区间和。"小华说，"但坐标范围是 -10^9 到 10^9 ——不能开 10^9 大小的数组。怎么办？"离散化！把所有用到的坐标收集起来，排序去重，映射成0,1,2,...的连续下标。然后用前缀和——问题变成熟悉的区间查询。离散化就是'压缩稀疏坐标'的技术——只存储和处理用得到的数据。"

输入 第一行n,m。然后n行每行x c（位置加值）。然后m行每行l r（查询区间和）。

输出 每行查询结果。

范围 $-10^9 \leq x \leq 10^9$, $n, m \leq 100000$

输入	输出
3 3	8
1 2	0
3 6	5
7 5	
1 3	
4 6	
7 8	

思路 离散化三步骤：1)收集所有坐标(x, l, r)；2)排序去重→映射为0..k-1；3)在压缩后的数组上做前缀和+查询。用二分查找原坐标对应的压缩下标。Python用bisect_left。时间 $O((n+2m)\log(n+2m))$ 。

技巧 离散化是处理稀疏大范围数据的标准手段——坐标0和 10^9 之间的空间不需要全部存储。Python: `all_x=sorted(set(coords)); idx={v:i+1 for i,v in enumerate(all_x)}` (1-indexed)。结合前缀和，大范围区间查询轻松解决。

C++

```

#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;
typedef pair<int, int> PII; //自定义类型PII, 表示容器类型是<int, int>的pair

const int N = 300007; //需要记录n个坐标值, 以及m个区间的坐标, 所以是100000*(1+2)
int n, m;
int a[N], s[N];

vector<int> alls; //存储所有的点坐标, 包括询问点坐标
vector<PII> add, query; //输入操作的坐标, 查询的区间坐标
//在alls之中, 查找值是x的元素值的下标, 并且映射到[1...n]
int find(int x)
{
    int left = 0, right = alls.size() - 1; //vector从0开始计数
    while (left < right)
    {
        int mid = left + right >> 1; //取中值
        if (alls[mid] >= x)
            right = mid;
        else
            left = mid + 1;
    }
    return right + 1; //把坐标值映射到1, 2, 3...而不是从0开始
}

///? 使用离散化的方式来预处理输入数据
///? 把输入的数字映射成{下标, 数字}的pair, 然后再用差分的方法计算区间和
///? 本题目可以参考797. 差分的模板写成
int main()
{
    cin >> n >> m;
    ///!离散化: 构造映射表

    for (int i = 0; i < n; i++)
    {
        ///todo 不直接把数存入a[i], 乃是存入add容器内
        int x, c;
        cin >> x >> c;
        add.push_back({x, c}); ///?将某一位置x上的数加c
        alls.push_back(x); //把x位置加入alls
    }
    // 读入query
    for (int i = 0; i < m; i++)
    {
        int l, r; //区间l和r
        cin >> l >> r;
        query.push_back({l, r});
        alls.push_back(l); //坐标与alls的下标做映射
        alls.push_back(r);
    }
    //alls是下标映射表, 需要去重
    //使用STL算法库的sort和unique
    sort(alls.begin(), alls.end());
    //unique返回不重复区间的下一个元素位置, 用erase去掉尾部
    alls.erase(unique(alls.begin(), alls.end()), alls.end()

```

Python

```

# 区间和
n, m = map(int, input().split())
adds = []
queries = []
all_x = []
for _ in range(n):
    x, c = map(int, input().split())
    adds.append((x, c))
    all_x.append(x)
for _ in range(m):
    l, r = map(int, input().split())
    queries.append((l, r))
    all_x.append(l)
    all_x.append(r)
all_x = sorted(set(all_x))
idx = {v: i + 1 for i, v in enumerate(all_x)}
a = [0] * (len(all_x) + 1)
s = [0] * (len(all_x) + 1)
for x, c in adds:
    a[idx[x]] += c
for i in range(1, len(all_x) + 1):
    s[i] = s[i - 1] + a[i]
for l, r in queries:
    print(s[idx[r]] - s[idx[l] - 1])

```

```

());

//针对离散化的数据, 计算区间和
for (auto it : add) //用c++11语法遍历add vector
{
    //初始化a[i],用离散化后的坐标
    int index = find(it.first);
    a[index] += it.second; //将index位置上的数加c
}
//预处理前缀和
for (int i = 1; i <= alls.size(); i++)
    s[i] = s[i - 1] + a[i];
//处理询问, 并且输出
for (auto item : query)
{
    int l = find(item.first), r = find(item.second);
    cout << s[r] - s[l - 1] << endl; //前缀和计算公式
}
return 0;
}

```

NQ126 区间合并 AcWing 803

"最后一题——把重叠的区间合并。"比如[1,3]和[2,6]重叠→合并成[1,6]。"小华说, "贪心法: 按左端点排序, 然后遍历。维护当前合并区间[st,ed]——如果新区间的左端点 $\leq$ ed说明重叠, 更新 $ed = \max(ed, new_ed)$; 否则当前区间完成, 开启新区间。一次排序 $O(n \log n)$ +一次遍历 $O(n)$ ——总共 $O(n \log n)$ 。"

- 输入** 第一行n。然后n行每行l r表示区间。
- 输出** 第一行合并后区间数。然后每行一个区间。
- 范围** $-10^9 \leq l \leq r \leq 10^9$, $n \leq 100000$

输入	输出
5	3
1 2	1 4
2 4	5 6
5 6	7 9
7 8	
7 9	

思路 区间合并: 1)按左端点升序排序; 2)初始化st,ed为第一个区间; 3)遍历剩余: 如果 $l \leq ed$ 则 $ed = \max(ed, r)$ (合并); 否则输出[st,ed]并开始新区间。注意输入区间可能不完全排序——必须先排序。时间 $O(n \log n)$ 。

技巧 贪心的核心: 按左端点排序后, 重叠区间一定连续出现。边界条件ed

C++

```

#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;
typedef pair<int, int> PII; //自定义类型PII, 表示容器类型是<int, int>的pair

const int N = 100007; //1<=n<=100000
int n; //n个区间

vector<PII> segments; //区间坐标

void merge(vector<PII> &segs)
{
    vector<PII> res;
    //pair排序先按左端点, 后右端点
    sort(segs.begin(), segs.end());

    //数字范围是-10^9<num<10^9, 因此负无穷大初始化为-2e9
    int left = -2e9, right = -2e9; //当前操作的区间[left, right]

    for (auto seg : segs) //遍历segs
    {
        if (right < seg.first) //区间没有交集, 更新当前区间
        {
            //存储合并完毕的区间[left, right]
            if (left != -2e9) res.push_back({left, right});

            //更新当前操作区间
            left = seg.first, right = seg.second;
        }
        else right = max(right, seg.second); //更新右端点, 扩展区间

        if (left != -2e9) res.push_back({left, right}); //把最后一个区间入栈
    }

    segs = res; //更新segs
}

int main()
{
    cin >> n;
    // 读入n组区间
    for (int i = 0; i < n; i++)
    {
        int l, r; //区间l和r
        cin >> l >> r;
        segments.push_back({l, r});
    }
    //合并区间segment
    merge(segments);
    //输出合并后的区间个数
    cout<<segments.size()<<endl; //输出合并后的区间元素个数

    return 0;
}

```

Python

```

import sys
data = sys.stdin.read().split()
n = int(data[0])
segs = []
idx = 1
for _ in range(n):
    l, r = int(data[idx]), int(data[idx+1])
    idx += 2
    segs.append((l, r))
segs.sort()
merged = []
st, ed = segs[0]
for l, r in segs[1:]:
    if l <= ed: ed = max(ed, r)
    else: merged.append((st, ed)); st, ed = l, r
merged.append((st, ed))
print(len(merged))
for l, r in merged: print(l, r)

```

本章知识点总结

- 高精度四则运算：C++手写理解底层（进位/借位/试商），Python直接用——选对语言事半功倍
- 位运算核心：lowbit= $x \& -x$, bit_count, 左移右移——底层优化的利器
- 离散化：稀疏空间→压缩→常规操作——大数据+稀疏坐标的标准处理流程
- 区间合并：排序+贪心遍历—— $O(n \log n)$ 解决最小区间覆盖问题

— 第11章完 · 共7题 · 自强不息 —

结构的魔力

第12章 · 基础数据结构 · 8题 · C++ & Python 双语对照

小鲁已经学会了不少算法技巧——排序、二分、前缀和、双指针。但小华告诉他："真正的程序性能瓶颈往往不在算法，而在**数据结构**。选对数据结构——你的代码可以快100倍。栈来处理后进先出，队列来处理先进先出，单调栈来找下一个更大元素，滑动窗口用单调队列，字符串匹配用KMP，优先队列用堆……本章8道题，每一题都是一个数据结构选择的经典案例。

本章角色

-  **小鲁** (进阶者) · 从数组迈入抽象数据结构——栈/队列/堆/KMP
-  **小华** (算法队长) · 白板上画出KMP的next数组跳转、堆的up/down操作
-  **小栋** (工程师) · "单调队列求滑动窗口最值"——真实大数据场景的最优解

八大基础数据结构速查

- **单链表**：数组模拟——`e[], ne[], head, idx`。全部操作 $O(1)$
- **栈/队列**：数组+指针。Python: `list`=栈, `deque`=队列
- **单调栈**：维护单调性的栈——找左右第一个更小/大值。均摊 $O(n)$
- **单调队列**：维护单调性的双端队列——滑动窗口最值。 $O(n)$
- **KMP**：next数组+失配跳转——字符串匹配 $O(n+m)$
- **堆**：完全二叉树+up/down——优先队列 $O(\log n)$ 插入删除

NQ127 单链表 AcWing 826

小鲁第一次实现链表："每个节点有一个next指针指向下一个节点。但反复new/delete太慢了——竞赛中用的是数组模拟链表！"小华解释道："e[i]存节点i的值，ne[i]存节点i的下一个节点下标。头节点head指向第一个节点，空闲链表管理未用节点。三个数组+一个整数idx（当前可用下标）就搭建了完整的内存池。这就是为什么Oler的代码里满是数组——不是不会用指针，而是数组更快。"

输入 第一行M。然后M行操作：H x(头插)、D k(删第k后)、I k x(在第k后插入)。

输出 链表从头到尾的值。

范围 $1 \leq M \leq 100000$

输入

```
10
H 9
I 1 1
D 1
D 0
H 6
```

输出

```
6 4 6 5
```

```
I 3 6
I 4 5
I 4 5
I 3 4
D 6
```

思路 数组模拟链表：e[]存值，ne[]存next下标，head=头节点下标，idx=当前可用下标。头插法：e[idx]=x; ne[idx]=head; head=idx++。第k个后插入：e[idx]=x; ne[idx]=ne[k]; ne[k]=idx++。删除第k个之后：ne[k]=ne[ne[k]]。

技巧 数组模拟链表=静态链表。每个操作O(1)。Python用list同样高效（append/pop=O(1)）。理解这种'用数组下标替代指针'的思想——后续Trie、并查集、堆都是用数组模拟各种结构。

C++

```

#include <iostream>

using namespace std;

const int N = 100010;

// head 表示头节点的下标, e[N]表示节点i的值
// ne[N]表示节点i的next指针是多少, idx存储当前已经用到了哪个节点
int head, e[N], ne[N], idx;

void init()
{
    head = -1; // 一开始设置为-1
    idx = 0; // 当前用到了0个节点
}

// 将x插入到头节点
void add_to_head(int x)
{
    e[idx] = x, ne[idx] = head, head = idx, idx++;
}

// 将x插到下标是k的点后面
void add(int k, int x)
{
    e[idx] = x, ne[idx] = ne[k], ne[k] = idx, idx++;
}

// 将下标是k的点后面的点删掉
void remove(int k)
{
    ne[k] = ne[ne[k]];
    // idx--;
}

int main()
{
    int m; cin >> m;

    init(); // 初始化

    while (m --)
    {
        int k, x;
        char op; // 操作字符

        cin >> op;
        if (op == 'H')
        {
            cin >> x;
            add_to_head(x);
        }
        else if (op == 'D')
        {
            cin >> k;
            if (!k) head = ne[head]; // 删除头节点
            else remove(k-1);
        }
        else
        {
            cin >> k >> x;
            add(k-1, x);
        }
    }

    // 打印单链表

```

Python

```

import sys

data = sys.stdin.read().split()
m = int(data[0])
idx_ptr = 1

# Initialize
N = 100010
e = [0] * N
ne = [0] * N
head = -1
idx = 0

for _ in range(m):
    op = data[idx_ptr]
    idx_ptr += 1
    if op == 'H':
        x = int(data[idx_ptr])
        idx_ptr += 1
        e[idx] = x
        ne[idx] = head
        head = idx
        idx += 1
    elif op == 'D':
        k = int(data[idx_ptr])
        idx_ptr += 1
        if k == 0:
            head = ne[head]
        else:
            ne[k-1] = ne[ne[k-1]]
    elif op == 'I':
        k = int(data[idx_ptr])
        x = int(data[idx_ptr+1])
        idx_ptr += 2
        e[idx] = x
        ne[idx] = ne[k-1]
        ne[k-1] = idx
        idx += 1

# Output
out = []
i = head
while i != -1:
    out.append(str(e[i]))
    i = ne[i]
print(' '.join(out))

```

```
for (int i = head; i!=-1; i = ne[i]) cout << e[i] << '
';
cout << endl;
}
```

NQ128 模拟栈 AcWing 828

"栈——后进先出(LIFO)。就像叠盘子，最后放上去的最先拿下来。"小华说，"实现一个栈支持push和pop操作。C++有std::stack，Python用list的append()和pop()天然就是栈。但手写一个数组模拟理解底层更透彻——用一个数组stk和指针tt，push时stk[++tt]=x，pop时tt--只移动指针，O(1)时间。"

输入 第一行M。然后M行：push x / pop / empty / query。

输出 按操作输出。

范围 $1 \leq M \leq 100000$

输入	输出
6	5
push 5	5
query	NO
push 6	
pop	
query	
empty	

思路 数组模拟栈：stk[N]和tt（栈顶指针，初始0）。push: stk[++tt]=x；pop: tt--；query: stk[tt]；empty: tt>0?'NO':'YES'。指针只是整数下标，极其高效。Python: list.append()=push, list.pop()=pop, list[-1]=query。

技巧 单调栈就是在普通栈的基础上加一条约束——栈内元素保持单调。这是后续处理'下一个更大元素'等问题的核心数据结构。理解栈+单调性=>O(n)解决看似O(n²)的问题。

C++

```
#include <iostream>

using namespace std;
const int N = 100010;

int m;
int stk[N], tt; // 数组模拟栈

int main()
{
    cin >> m;
    while (m --)
    {
        string op;
        int x;

        cin >> op;
        if (op == "push") // 栈顶指针+1, 把数压入数组
        {
            cin >> x;
            stk[++tt] = x; // 压入一个元素
        }
        else if (op == "pop") tt --; // 修改栈顶指针即可
        else if (op == "empty") cout << (tt? "NO": "YES") << endl; // tt=0表示非空
        else cout << stk[tt] << endl; // 返回栈顶
    }
}
```

Python

```
# 模拟栈
m = int(input())
stk = []
for _ in range(m):
    parts = input().split()
    op = parts[0]
    if op == 'push':
        stk.append(int(parts[1]))
    elif op == 'pop':
        stk.pop()
    elif op == 'query':
        print(stk[-1])
    else:
        print('YES' if not stk else 'NO')
```

NQ129 模拟队列 AcWing 829

"队列——先进先出(FIFO)。就像排队买饭，先到的先服务。"小华画出示意图，"用数组模拟队列需要两个指针：hh(队头)和tt(队尾)。push时q[++tt]=x，pop时hh++——两个指针都只进不退。这就是'同时移动双端'的模式。Python的deque是双端队列，from collections import deque——比list的pop(0)快(O(1) vs O(n))。"

输入 第一行M。然后M行：push x / pop / empty / query。

输出 按操作输出。

范围 $1 \leq M \leq 100000$

输入	输出
6	5
push 5	6
query	NO
push 6	
pop	
query	
empty	

思路 数组模拟队列：q[N]，hh=0队头，tt=-1队尾。push: q[++tt]=x; pop: hh++; query: q[hh]; empty: hh>tt。注意hh>tt表示空。Python: deque.append()=push, deque.popleft()=pop。list的pop(0)是O(n)——

不要用！

技巧 Python中list.pop(0)是O(n)因为要移动所有元素。一定要用collections.deque！单调队列=单调性+双端队列——滑动窗口最值问题的核心数据结构。

C++

```
#include <iostream>
using namespace std;
const int N = 1000007;
int q[N], hh, tt=-1;

int main(){
    int m;
    cin>>m;
    while (m--){
        int x;

        string op;
        cin>>op;

        if (op == "push") {
            //(1) "push x" - 向队尾插入一个数x;
            cin>>x;
            q[++tt] = x;
        } else if (op == "pop") // (2) "pop" - 从队头弹出一个数;
            hh++;
        else if (op == "empty") // (3) "empty" - 判断队列是否为空;
            cout<<(hh==tt?"NO":"YES")<<endl;
        else cout<<q[tt]<<endl; // (4) "query" - 查询队头元素。

        return 0;
    }
}
```

Python

```
# 模拟队列
from collections import deque
m = int(input())
q = deque()
for _ in range(m):
    parts = input().split()
    op = parts[0]
    if op == 'push':
        q.append(int(parts[1]))
    elif op == 'pop':
        q.popleft()
    elif op == 'query':
        print(q[0])
    else:
        print('YES' if not q else 'NO')
```

NQ130 表达式求值 AcWing 3302

"计算'(2+3)×5'——用栈！"小华说，"两个栈：一个存数字，一个存运算符。遇到数字压数字栈；遇到(压运算符栈；遇到)计算直到(被弹出；遇到运算符——如果栈顶优先级≥当前，先算栈顶。核心：维护运算优先级。这是编译原理和计算器的核心。"小鲁试了试："确实巧妙——中缀表达式转后缀求值的经典算法！"

输入 一个中缀表达式（含+、*、/和括号，数字非负）。

输出 表达式结果（整数除法向零截断）。

范围 表达式长度 ≤ 100000

输入

(2+3)*5

输出

25

思路 双栈法求中缀表达式：数字栈+运算符栈。优先级映射：+-为1，*/为2。遇到)弹栈直到(。每次弹出一个运算符和两个数字，计算结果压回数字栈。Python的eval()能直接算但OJ比赛中禁止。理解手写栈求值是数据结构综合应用的经典案例。

技巧 符号优先级存入字典：pri={'+':1,'-':1,'*':2,'/':2}。从左到右扫描。eval()在安全环境下可用但不适用于OJ——学会手写才是目的。注意除法向零截断——Python的//对负数向下取整，需要用int(a/b)。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>
#include <stack>
#include <unordered_map>

using namespace std;

stack<int> num; //存储计算过程中的数字
stack<char> op; //存储操作符

void eval()
{
    auto b = num.top(); num.pop(); //第二个数
    auto a = num.top(); num.pop(); //第一个数
    auto c = op.top(); op.pop();

    int x;
    if (c == '+') x = a + b;
    else if (c == '-') x = a - b;
    else if (c == '*') x = a * b;
    else x = a / b;
    num.push(x);
}

int main()
{
    //优先级, 可扩展为其他符号的优先级
    unordered_map<char, int> pr{{'+', 1}, {'-', 1}, {'*', 2}, {'/', 2}};
    string str; //输入都存储在字符串内
    cin >> str;
    for (int i = 0; i < str.size(); i++)
    {
        auto c = str[i]; //读入每个char
        if (isdigit(c)) //判断是否是数字
        {
            int x = 0, j = i;
            while (j < str.size() && isdigit(str[j]))
                x = x * 10 + str[j++] - '0';
            i = j - 1; // 调整i的位置
            num.push(x); //压栈
        }
        else if (c == '(') op.push(c);
        else if (c == ')')
        {
            //把前面所有的表达式都出栈运算
            while (op.top() != '(') eval();
            op.pop();
        }
        else
        {
            //根据操作符的优先级执行运算
            while (op.size() && op.top() != '(' && pr[op.top()]
>= pr[c])
                eval();
            op.push(c);
        }
    }
    //处理剩余的操作符
    while(op.size()) eval();
    cout<<num.top() <<endl;
}

```

Python

```

# 表达式求值
s = input().strip()
num = []
op = []
pri = {'+': 1, '-': 1, '*': 2, '/': 2}

def eval_op():
    b = num.pop()
    a = num.pop()
    c = op.pop()
    if c == '+':
        num.append(a + b)
    elif c == '-':
        num.append(a - b)
    elif c == '*':
        num.append(a * b)
    else:
        num.append(int(a / b))

i = 0
while i < len(s):
    c = s[i]
    if c == ' ':
        i += 1
        continue
    if c.isdigit():
        j = i
        while j < len(s) and s[j].isdigit():
            j += 1
        num.append(int(s[i:j]))
        i = j
        continue
    if c == '(':
        op.append(c)
    elif c == ')':
        while op[-1] != '(':
            eval_op()
        op.pop()
    else:
        while op and op[-1] != '(' and pri.get(op[-1], 0) >= pri.get(c, 0):
            eval_op()
        op.append(c)
    i += 1
while op:
    eval_op()
print(num[-1])

```

```
return 0;
}
```

NQ131 单调栈 AcWing 830

"找每个数左边第一个比它小的数——暴力 $O(n^2)$ 太慢。"小华说，"单调栈——维护一个单调递增的栈。遍历每个数时，把栈中 $\geq$ 它的数都弹掉（因为它们不会再成为任何后续数的答案）。弹完之后栈顶就是答案，然后当前数入栈。每个数最多入栈一次出栈一次——均摊 $O(n)$ 。单调性把 $O(n^2)$ 降到 $O(n)!$ "

输入 第一行N。第二行N个整数。

输出 每个数左边第一个比它小的数（没有输出-1）。

范围 $1 \leq N \leq 100000$

输入

```
5
3 4 2 7 5
```

输出

```
-1 3 -1 2 2
```

思路 维护单调递增栈：遍历每个数，while栈非空且栈顶 $\geq$ 当前数则pop()。弹出结束后栈顶就是答案。当前数入栈。每个数入栈1次出栈1次——均摊 $O(n)$ 。单调栈利用'大数再也不会被需要'的洞察来去掉无用信息。

技巧 单调栈的变种：找右边第一个大的（从右往左+单调递减），找左边第一个大的（从左到右+单调递减）。记模板不如理解核心：'弹出永远不会用到的元素'。Python: while stack and stack[-1] $\geq$ x: stack.pop()。

C++

```
#include <iostream>
using namespace std;
const int N=100007;
int n;
int stk[N],tt;//数组模拟栈实现单调栈
int main ()
{
    cin>> n;
    for (int i=0; i< n; i++){
        int x; cin>>x;
        while (tt && stk[tt]>= x)//tt为空，并且单调栈顶元素比
x大
            tt--;//出栈，这些数永远不会用到
        if (tt) cout<<stk[tt]<<' '; //栈顶就是比x小的元素，
直接输出
        else cout<<-1<<' '; //x左边没有任何数比它小
        //入栈
        stk[++tt] = x;
    }
    return 0;
}
```

Python

```
# 单调栈
n = int(input())
a = list(map(int, input().split()))
stk = []
res = []
for x in a:
    while stk and stk[-1] >= x:
        stk.pop()
    res.append(str(stk[-1]) if stk else '-1')
    stk.append(x)
print(' '.join(res))
```

NQ132 滑动窗口 AcWing 154

"长度为k的窗口在数组上从左滑到右——输出每个窗口的最小值和最大值。"小栋说，"暴力每个窗口 $O(k)$ 总 $O(n \times k)$ 太慢。"小华回答："单调队列！用双端队列维护窗口内的最小值候选。队头是当前窗口的最小值下标，当队头滑出窗口时弹出，当新元素比队尾更小时——队尾永无翻身之日，弹掉。 $O(n)$ 时间解决。这是单调队列最经典的应用！"

输入

第一行n,k。第二行n个整数。

输出

第一行所有窗口最小值。第二行所有窗口最大值。

范围 $1 \leq n \leq 10^6, 1 \leq k \leq n$

输入

```
8 3
1 3 -1 -3 5 3 6 7
```

输出

```
-1 -3 -3 -3 3 3
3 3 5 5 6 7
```

思路 单调队列维护窗口最值。最小值：维护递增队列，队头存窗口最小值下标。每次新元素x到来：while队尾值 $\geq x$ 则pop_back（队尾不会再成为最小值），然后push_back当前下标。if队头下标 $<$ 窗口左边界则pop_front。队头=当前窗口最小值。最大值类似（维护递减队列）。 $O(n)$ 。

技巧 Python用collections.deque—— $O(1)$ 的popleft()和pop()。队尾弹出while q and arr[q[-1]] $\geq$ x: q.pop()。理解单调队列的本质：维护一个'永远单调'的双端候选集。这是滑动窗口问题的终极答案。

C++

```
#include <iostream>
using namespace std;

const int N=1000007;

int n,k;
int a[N],q[N]; //数组模拟队列
int hh=0,tt=-1; //hh队头,tt对尾

void initQueue(){
    hh=0; tt=-1;
}

int main()
{
    scanf("%d%d",&n,&k);
    for (int i=0; i<n; i++) scanf("%d",&a[i]); //读入数字

    //求滑动窗口内的最小值
    initQueue(); //初始化队列, 队列存储的是数组下标
    for (int i=0; i<n; i++)
    {
        //判断队头是否已经滑出窗口
        if (hh<=tt && i-k+1> q[hh]) hh++; //pop front 删除队头

        //!单调队列, 去掉所有降序的队尾点
        while (hh<=tt && a[q[tt]]>= a[i]) tt--; //pop back 删除队尾

        q[++tt] = i; //把当前i值入队列
        if (i>=k-1) //i在窗口内
            printf("%d ",a[q[hh]]); //队头就是最小值
    }
    puts("");

    //求滑动窗口内的最大值
    initQueue(); //初始化队列, 队列存储的是数组下标
    for (int i=0; i<n; i++)
    {
        //判断队头是否已经滑出窗口
        if (hh<=tt && i-k+1> q[hh]) hh++; //pop front 删除队头

        //!单调队列, 去掉所有降序的队尾点
        while (hh<=tt && a[q[tt]]<=a[i]) tt--; //pop back 删除队尾

        q[++tt] = i; //把当前i值入队列
        if (i>=k-1) //i在窗口内
            printf("%d ",a[q[hh]]); //队头就是最大值
    }
    puts("");
}
```

Python

```
import sys
from collections import deque
data = sys.stdin.buffer.read().split()
n, k = int(data[0]), int(data[1])
a = [int(x) for x in data[2:2+n]]
q = deque()
mins = []
for i in range(n):
    while q and a[q[-1]] >= a[i]: q.pop()
    q.append(i)
    if q[0] <= i - k: q.popleft()
    if i >= k - 1: mins.append(str(a[q[0]]))
q.clear()
maxs = []
for i in range(n):
    while q and a[q[-1]] <= a[i]: q.pop()
    q.append(i)
    if q[0] <= i - k: q.popleft()
    if i >= k - 1: maxs.append(str(a[q[0]]))
sys.stdout.write(' '.join(mins) + '\n')
sys.stdout.write(' '.join(maxs) + '\n')
```

NQ133 KMP字符串 AcWing 831

"字符串匹配——模式串P在文本串T中找所有出现位置。暴力匹配 $O(n \times m)$ 太慢。"小华打开白板，"KMP算法——预处理next数组，利用已经匹配的信息避免重复比较。next[i]=P[0..i-1]的最长公共前后缀长度。匹配时如果失配，j跳到next[j]——不需要回到最初的i+1。时间 $O(n+m)$ 。这是算法课上最优雅的设计之一。"

输入

第一行n和模式串P，第二行m和文本串T。

输出

所有匹配位置的起始下标（0-indexed）。

范围 $1 \leq n \leq 10^5, 1 \leq m \leq 10^6$

输入

```
3
aba
5
ababa
```

输出

```
0 2
```

思路 KMP两步骤：1)求next数组——i从1开始，j=next[i-1]；while j>0且P[i]!=P[j]则j=next[j-1]；若P[i]==P[j]则j++；next[i]=j。2)匹配——i遍历T,j跟踪P；while j>0且T[i]!=P[j]则j=next[j-1]；若T[i]==P[j]则j++；若j==n则找到匹配。Python的P in T或find()底层用高效算法但理解KMP原理是理解字符串算法的基石。

技巧 next数组=失配时跳转的'记忆'——利用模式串自身的前后缀重复避免回溯。KMP保证了每个字符最多被比较两次。Python: T.find(P)一行，但手写KMP理解'失配跳转'的智慧是进阶算法的必修课。

C++

```
#include <iostream>
using namespace std;
const int N = 100007, M = 1000007;
int n, m; //n为待匹配字符串长度
char p[N], s[M]; //p为模板,s搜索范围
int ne[N]; // next数组,初始值为0

int main()
{
    // 第一行输入整数N, 表示字符串P的长度。
    // 第二行输入字符串P。
    // 第三行输入整数M, 表示字符串S的长度。
    // 第四行输入字符串S。
    cin >> n >> p + 1 >> m >> s + 1; //字符串从1开始存储, ci
    n会在末尾加上\0

    // 求next的过程 next[1]=0;从第二个字符开始计算next
    for (int i = 2, j = 0; i <= n; i++) //预处理p串计算前缀
    后缀的值
    {
        //双指针操纵与匹配过程一致
        while (j && p[i] != p[j + 1])
            j = ne[j];
        if (p[i] == p[j + 1])
            j++;
        ne[i] = j; //找到i的回退点j
    }

    // KMP 匹配过程
    for (int si = 1, pj = 0; si <= m; si++) //si,pj为字符串
    指针, 从1开始算
    {
        while (pj && s[si] != p[pj + 1]) // 用p串pj+1字符
        与s串的si比较 (提前看一个字符)
            pj = ne[pj]; //遇到不匹配, 回退p
        j指针
        if (s[si] == p[pj + 1])
            pj++; //pj进一位
        // p串匹配成功
        if (pj == n)
        { //抵达p串的最后一个字符
            cout << si - n << " "; //输出出现位置的下标
            pj = ne[pj]; //回退,预备下一次的查找
        }
    }
    return 0;
}
```

Python

```
# KMP字符串
n = int(input())
p = input().strip()
m = int(input())
s = input().strip()
ne = [0] * n
j = 0
for i in range(1, n):
    while j > 0 and p[i] != p[j]:
        j = ne[j - 1]
    if p[i] == p[j]:
        j += 1
    ne[i] = j
j = 0
res = []
for i in range(m):
    while j > 0 and s[i] != p[j]:
        j = ne[j - 1]
    if s[i] == p[j]:
        j += 1
    if j == n:
        res.append(str(i - n + 1))
        j = ne[j - 1]
print(' '.join(res))
```

NQ134 模拟堆 AcWing 839

"最后一个数据结构——堆。"小华说, "小根堆: 父节点 $\leq$ 子节点。插入: 放在末尾然后向上冒泡(up)。删除堆顶: 把最后一个元素移到堆顶然后向下沉降(down)。用数组模拟完全二叉树: 节点i的父节点 $=i/2$, 左子 $=2i$, 右子 $=2i+1$ 。堆是优先队列的底层——Dijkstra、Huffman编码、堆排序都离不开它。"

输入

第一行n。然后n行: I x(插入)、PM(输出最小)、DM(删除最小)、D k(删除第k个插入)、C k x(修改第k个插入为x)。

输出

按操作输出。

范围 $1 \leq n \leq 100000$

输入	输出
8	1
I 2	3
I 1	
PM	
DM	
D 1	
C 2 8	
I 3	
PM	

思路 数组模拟小根堆：h[i]存值，size存当前堆大小。down(u)：找u和两子中最小的，若子更小则交换并递归down。up(u)：若u<父节点则交换并递归up。插入：h[++size]=x; up(size)。删除堆顶：h[1]=h[size--]; down(1)。支持随机删除需额外维护ph[k]（第k个插入在堆中位置）和hp[i]（堆位置i对应的插入编号）。

技巧 Python用heapq：heappush/ heappop/ heapify。但竞赛中常需要支持随机删除修改——需要手写堆+位置映射。理解堆的up/down操作是理解所有优先队列变种（d-ary堆、fib堆、配对堆）的基础。数组下标从1开始方便计算父子。

C++

```

#include <iostream>
#include <algorithm>
#include <string.h>

using namespace std;

const int N = 100007;

// hp是heap pointer的缩写，表示堆数组中下标到第k个插入的映射
// ph是pointer heap的缩写，表示第k个插入到堆数组中的下标的映射
// hp和ph数组是互为反函数的
int h[N], ph[N], hp[N], cnt;

void heap_swap(int a, int b)
{
    swap(ph[hp[a]], ph[hp[b]]); // 堆数组的下标
    swap(hp[a], hp[b]); // 下标与第几个插入元素的映射
    swap(h[a], h[b]); // 堆元素
}

void down(int u)
{
    int t = u;
    if (u*2 <= cnt && h[u*2] < h[t]) t = u*2; // u*2左儿子
    if (u*2+1 <= cnt && h[u*2+1] < h[t]) t = u*2+1; // u*2+1右儿子
    if (u != t)
    {
        heap_swap(u, t);
        down(t);
    }
}

void up(int u)
{
    while (u/2 && h[u] < h[u/2])
    {
        heap_swap(u, u/2);
        u >>= 1;
    }
}

int main()
{
    int n, m = 0;
    scanf("%d", &n);
    while (n --)
    {
        char op[5];
        int k, x;
        scanf("%s", op);
        // 在堆最后一个元素插入元素
        if (!strcmp(op, "I"))
        {
            scanf("%d", &x);
            cnt++;
            m++;
            ph[m] = cnt, hp[cnt] = m;
            h[cnt] = x;
            up(cnt); // 插入后执行up(cnt)
        }
        else if (!strcmp(op, "PM")) printf("%d\n", h[1]); // 堆顶
    }
}

```

Python

```

# 模拟堆
import sys
n = int(input())
h = [0] * 100010
hp = [0] * 100010
ph = [0] * 100010
sz = 0
idx = 0

def heap_swap(a, b):
    ph[hp[a]], ph[hp[b]] = b, a
    hp[a], hp[b] = hp[b], hp[a]
    h[a], h[b] = h[b], h[a]

def down(u):
    t = u
    if u * 2 <= sz and h[u * 2] < h[t]:
        t = u * 2
    if u * 2 + 1 <= sz and h[u * 2 + 1] < h[t]:
        t = u * 2 + 1
    if u != t:
        heap_swap(u, t)
        down(t)

def up(u):
    while u // 2 > 0 and h[u] < h[u // 2]:
        heap_swap(u, u // 2)
        u //= 2

for _ in range(n):
    parts = sys.stdin.readline().split()
    op = parts[0]
    if op == 'I':
        x = int(parts[1])
        sz += 1
        idx += 1
        ph[idx] = sz
        hp[sz] = idx
        h[sz] = x
        up(sz)
    elif op == 'PM':
        print(h[1])
    elif op == 'DM':
        heap_swap(1, sz)
        sz -= 1
        down(1)
    elif op == 'D':
        k = int(parts[1])
        u = ph[k]
        heap_swap(u, sz)
        sz -= 1
        down(u)
        up(u)
    else:
        k, x = int(parts[1]), int(parts[2])
        u = ph[k]
        h[u] = x

```

是最小值

```

else if (!strcmp(op, "DM"))
{
    heap_swap(1, cnt);
    cnt --;
    down(1); //删除最小值后执行down(1)
}
else if (!strcmp(op, "D"))
{
    scanf("%d", &k); //删除第k个元素
    k = ph[k]; //取第k个元素下标
    heap_swap(k, cnt); //与最后一个元素交换
    cnt--;
    up(k);
    down(k);
}
else {
    scanf("%d%d", &k, &x); //修改第k个数
    k = ph[k];
    h[k] = x;
    up(k);
    down(k);
}
}
return 0;
}

```

```

down(u)
up(u)

```

本章知识点总结

- 数组模拟数据结构：链表/栈/队列/堆都可以用数组+下标模拟—— $O(1)$ 操作且更缓存友好
- 单调栈/单调队列：利用单调性排除无用元素—— $O(n^2) \rightarrow O(n)$ 的关键技术
- KMP：利用自身前后缀重复——字符串匹配 $O(n+m)$ 。理解失配跳转的智慧
- 堆：完全二叉树+up/down——优先队列 $O(\log n)$ 。是Dijkstra/Huffman/堆排序的基础

搜索的艺术

第13章 · 搜索与回溯 · 8题 · C++ & Python 双语对照

小鲁开始学习人工智能最基础的算法——**搜索**。"搜索就是穷举所有可能性——但穷举不等于蛮力。DFS回溯像走迷宫一条路走到黑，BFS像水的波纹层层扩散。加上剪枝——提前砍掉不可能的分支——搜索可以从指数级变成多项式级。理解搜索，是理解所有AI和优化算法的基石。"

🔍 DFS vs BFS 速查

- **DFS**: 深度优先——回溯探索。适合：全排列、N皇后、树遍历。空间O(深度)
- **BFS**: 广度优先——层层扩散。适合：最短路径、八数码、无权图最短路。空间O(宽度)
- **剪枝**: 提前排除不可能的分支——N皇后的col/dg/udg标记
- **状态编码**: 将棋盘/网格编码成字符串作哈希key——八数码的关键技术

NQ135 排列数字 AcWing 842

小鲁开始学习搜索算法："全排列——1到n的所有排列方式。用DFS深度优先搜索，像走迷宫一样一条路走到底再回头。每次选一个没用过的数放入路径，递归进入下一层，然后撤销选择(回溯)。"小华点头："DFS+回溯是搜索算法的灵魂框架。三个要素：路径(已经选了什么)、选择列表(还能选什么)、结束条件(选满了就输出)。这个框架将贯穿你的算法生涯。"

输入 一个整数n。

输出 1~n所有排列，每行一个，按字典序。

范围 $1 \leq n \leq 7$

输入	输出
3	1 2 3 1 3 2 2 1 3 2 3 1 3 1 2 3 2 1

思路 DFS回溯：path存当前路径，used标记已用数字。dfs(u): 若u==n则输出path；否则遍历1..n，若未used则标记used→加入path→dfs(u+1)→撤销标记→弹出path。复杂度O(n!)。注意递归树的深度=n，叶子数=n!。

技巧 Python: itertools.permutations一行。手写DFS重点理解回溯的'撤销'操作——进入递归前做的修改，返回后必须完全复原。回溯=尝试→失败→回退→再尝试。排列问题是回溯第一课，也是理解'搜索状态空间'的

钥匙。

C++

```
#include <iostream>
using namespace std;

const int N = 10; //最大值
int n;           //读入的n
int path[N];     //记录路径每个位的值
bool used[N];    //记录第i个数是否被用过

void dfs(int u) {
    if (u == n) {
        //输出排列
        for (int i = 0; i < n; i++) cout << path[i] << " ";
        cout << endl;
        return;
    }
    //枚举数字1--n
    for (int i = 1; i <= n; i++) {
        if (!used[i]) {
            path[u] = i; //记录当前位path[u]的数字为i
            used[i] = true; //标记数字i为已经使用
            dfs(u + 1); //递归处理下一个位
            used[i] = false; //恢复现场
        }
    }
}

int main() {
    cin >> n;
    dfs(0);
    return 0;
}
```

Python

```
import sys
sys.setrecursionlimit(200000)
# 排列数字
n = int(input())
path = []
used = [False] * (n + 1)

def dfs():
    if len(path) == n:
        print(' '.join(map(str, path)))
        return
    for i in range(1, n + 1):
        if not used[i]:
            used[i] = True
            path.append(i)
            dfs()
            path.pop()
            used[i] = False

dfs()
```

NQ136 n-皇后问题 AcWing 843

"N皇后——在N×N棋盘上放N个皇后，使它们互不攻击（同行、同列、同对角线只有一个）。"小华画出棋盘，"回溯搜索的经典题。按行搜索——每行必放一个皇后，只需决定放在哪一列。用三个布尔数组标记：col[j]（列是否被占）、dg[i+j]（主对角线）、udg[i-j+n]（反对角线）。回溯的剪枝的力量——不满足约束的分支直接跳过。"

输入 一个整数n。

输出 所有方案，每个方案n行，每行n个字符(.Q)表示棋盘。

范围 $1 \leq n \leq 9$

输入

4

输出

```
.Q..
...Q
Q...
..Q.

..Q.
Q...
```

```
...Q
.Q..
```

思路 按行DFS：dfs(r)——在第r行选一列放皇后。对每列，检查col[j], dg[r+j], udg[r-j+n]是否未占。若可放则标记→递归r+1→撤销标记。对角线坐标映射：主对角线r+j为定值([0,2n-2])，反对角线r-j+n为定值。O(n!)但剪枝后实际更快。

技巧 对角线编号技巧：主对角线r+c=常数(0到2n-2)，反对角线r-c+n=常数(n到2n+n)。用三个bool数组O(1)判断冲突——比每次遍历O(n)快得多。回溯+剪枝=搜索的效率保证——剪得越早，省得越多。

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 1007;

int n, m;
int f[N][N];
int v[N], w[N];

int main()
{
    cin >> n >> m;
    for (int i = 1; i <= n; i++) cin >> v[i] >> w[i];

    //完全背包
    // 1.f[i][j] = f[i-1][j]; (不选物品i)
    // 2.f[i][j] = max(f[i-1][j], f[i][j-v[i]]+w[i]) (选若干个物品i)
    for (int i = 1; i <= n; i++)
        for (int j = 0; j <= m; j++)
        {
            f[i][j] = f[i-1][j];
            if (j >= v[i])
                f[i][j] = max(f[i-1][j], f[i][j-v[i]]+w[i]);
        }

    cout << f[n][m] << endl;
    return 0;
}
```

Python

```
import sys
sys.setrecursionlimit(200000)
# n-皇后问题
n = int(input())
g = [['.'] * n for _ in range(n)]
col = [False] * n
dg = [False] * (2 * n)
udg = [False] * (2 * n)

def dfs(r):
    if r == n:
        for row in g:
            print(''.join(row))
        print()
        return
    for c in range(n):
        if not col[c] and not dg[r + c] and not udg[r - c + n]:
            col[c] = dg[r + c] = udg[r - c + n] = True
            g[r][c] = 'Q'
            dfs(r + 1)
            g[r][c] = '.'
            col[c] = dg[r + c] = udg[r - c + n] = False

dfs(0)
```

NQ137 走迷宫 AcWing 844

"从迷宫起点走到终点——每次只能上下左右走一步。"小华说，"DFS会找到一条路径，但不一定最短；BFS广度优先——层层扩展，像水的波纹——第一次遇到终点时走的一定是最短路径！BFS用队列：起点入队→弹出一个位置→扩展四个方向→合法的入队。每个位置只访问一次。BFS是求无权图最短路径的标准算法。"

输入

第一行n,m。然后n×m的迷宫(0可走1墙)。

输出

从(0,0)到(n-1,m-1)的最少步数。

范围 $1 \leq n, m \leq 100$

输入

```
5 5
0 1 0 0 0
0 1 0 1 0
0 0 0 0 0
0 1 1 1 0
0 0 0 1 0
```

输出

```
8
```

思路 BFS模板：queue存(x,y,dist)；vis数组标记访问过。四个方向：dx=[-1,0,1,0],dy=[0,1,0,-1]。每次while queue非空：弹出队头，判断是否终点，否则扩展四个方向，合法且未访问则标记入队。每个位置最多访问一次，O(nm)。

技巧 BFS层序遍历——队列保证先访问近的点。dist不需要额外存储，直接存在队列元素或距离数组中。DFS和BFS的选择：DFS找一条路径(不保证最短)，BFS找最短路径。Python用deque作BFS队列——popleft() O(1)。

C++

```

#include <algorithm>
#include <iostream>
#include <queue>
#include <cstring>
using namespace std;

typedef pair<int, int> PII; // Y总风格pair<int,int>声明

const int N = 107;

int n, m;
int g[N][N], d[N][N];

//广搜
int bfs() {

    queue<PII> q; //队列

    memset(d, -1, sizeof d);
    d[0][0] = 0;
    q.push({0, 0});

    // 四个方向向量
    int dx[] = {-1, 0, 1, 0}, dy[] = {0, 1, 0, -1};

    while (q.size())
    { //队列为空的时候退出广搜
        auto t = q.front(); //取队头
        q.pop(); //弹出队头

        //搜索四个方向
        for (int i = 0; i < 4; i++)
        {
            int x = t.first + dx[i], y = t.second + dy[i]; //
            待搜索的新坐标点

            if (x < 0 || x >= n || y < 0 || y >= m || g[x][y] !
            = 0 || d[x][y] != -1) continue;

            d[x][y] = d[t.first][t.second] + 1;

            q.push({x, y}); //入队

        }
    }

    return d[n-1][m-1];
}

int main() {

    cin >> n >> m;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            cin >> g[i][j];

    cout << bfs() << endl;

    return 0;
}

```

Python

```

import sys
from collections import deque
data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
g = []
idx = 2
for _ in range(n):
    g.append([int(data[idx+j]) for j
    in range(m)])
    idx += m
dist = [[-1] * m for _ in range(n)]
dx = [-1, 0, 1, 0]
dy = [0, 1, 0, -1]
q = deque()
q.append((0, 0))
dist[0][0] = 0
while q:
    x, y = q.popleft()
    for d in range(4):
        nx, ny = x + dx[d], y + dy[d]
        if 0 <= nx < n and 0 <= ny <
        m and g[nx][ny] == 0 and dist[nx][ny]
        == -1:
            dist[nx][ny] = dist[x][y]
            + 1
            q.append((nx, ny))
print(dist[n-1][m-1])

```

NQ138 八数码 AcWing 845

"3×3的格子，8个数+1个空格——通过滑动空格，把乱序排列恢复成123/456/78x。"小华说，"BFS的状态空间搜索！每个状态是一个3×3排列→看成一个字符串。从初始状态开始BFS，每次空格和相邻数字交换生成新状态。用字典记录每个状态到初始状态的距离。终止状态固定为'12345678x'。BFS第一次遇到终止状态时的距离就是最少步数。"

输入 一行9个字符，表示初始状态（x代表空格）。

输出 最少移动步数，无解输出-1。

范围 3×3网格

输入	输出
2 3 4 1 5 x 7 6 8	19

思路 BFS状态空间搜索：状态编码为字符串(9字符)。从初始状态开始BFS，用unordered_map/dict记录距离。每次找到'x'的位置，和上下左右(若合法)交换，生成新状态。复杂度 $O(9!)=362880$ 状态——BFS完全可以。注意逆序对奇偶性判断无解。

技巧 BFS的队列保证层层扩展——第一次遇到目标状态时步数最少。状态总数 $=9!/2=181440$ （一半无解）。Python用字典作距离映射+deque作BFS队列。把3×3网格编码成一维字符串是简化状态的关键。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>
#include <unordered_map>
#include <queue>

using namespace std;

int bfs(string start)
{
    string end = "12345678x";

    queue<string> q;
    unordered_map<string,int> d;//字符串映射到整数

    q.push(start);
    d[start] = 0;//map的用法

    //方向向量
    int dx[4] = {1,-1,0,0},dy[4]={0,0,1,-1};

    //广搜
    while(q.size())
    {
        auto t = q.front();
        q.pop();

        int distance = d[t];
        if (t==end) return distance;//找到目标

        //查询x在字符串中的下标
        int k = t.find('x');
        int x = k / 3, y = k % 3;//转换为宫格坐标

        for (int i = 0; i < 4; i ++ )
        {
            int a = x + dx[i], b = y + dy[i];

            if (a >= 0 && a < 3 && b >= 0 && b < 3)
            {
                swap(t[k], t[a*3+b]);//移动x的位置
                if (!d.count(t))
                {
                    d[t] = distance + 1;
                    q.push(t);//压入队列
                }
                swap(t[k],t[a*3+b]);//还原现场
            }
        }
    }
    return -1;
}

int main()
{
    string c, start;
    //去掉空格
    for (int i = 0; i < 9; i ++ )
    {
        cin>>c;
        start += c;
    }
}

```

Python

```

import sys
from collections import deque
start = ''.join(sys.stdin.read().split())
target = '12345678x'
if start == target: print(0); sys.exit(0)
dx, dy = [-1,0,1,0], [0,1,0,-1]
q = deque([start])
dist = {start: 0}
while q:
    s = q.popleft()
    k = s.index('x')
    x, y = k//3, k%3
    for d in range(4):
        nx, ny = x+dx[d], y+dy[d]
        if 0<=nx<3 and 0<=ny<3:
            nk = nx*3 + ny
            lst = list(s)
            lst[k], lst[nk] = lst[nk], lst[k]
            ns = ''.join(lst)
            if ns not in dist:
                dist[ns] = dist[s] + 1
            if ns == target: print(dist[ns]); sys.exit(0)
            q.append(ns)
print(-1)

```

```

    }

    cout<<bfs(start)<<endl;
}

```

NQ139 树的重心 AcWing 846

"找树的重心——删除重心后剩余各连通块中最大块的最小值。"小华画出树，"DFS遍历树，统计每个节点的子树大小。删除节点u后，连通块有三部分：u的各子树、u的父节点所在的整棵树减去u的子树。取这些中的最大值，然后所有u的这个最大值中的最小值就是答案。树形DP+DFS——理解'树遍历+状态统计'的模式。"

输入 第一行n。然后n-1行每行a b表示边。

输出 重心对应的最大连通块最小值。

范围 $1 \leq n \leq 10^5$

输入	输出
9	4
1 2	
1 7	
1 4	
2 8	
2 5	
4 3	
3 9	
4 6	

思路 DFS统计子树大小：dfs(u,fa)——遍历u的邻居v(排除fa)，dfs(v,u)， $s[u] += s[v]$ 累加子树大小。删除u后最大块= $\max(s[v](\text{各子树}), n-s[u](\text{上方树}))$ 。取所有u的max的最小值。 $O(n)$ 一次DFS。

技巧 树形DFS的关键：用fa参数避免回头，用邻接表存图。Python递归可能超深度——用sys.setrecursionlimit(200000)或手写栈。重心在点分治(CDQ)中起关键作用——保证递归深度为 $\log n$ 。这是树形DP的入门。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007, M = N*2; //无向图

int n;
int h[N], e[M], ne[M], idx;
int ans = N;
bool st[N]; //顶点n是否访问

void add(int a, int b) // 添加一条边a->b
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++ ;
}

int dfs(int u)
{
    st[u] = true;

    int size = 0; //去掉i节点后, i子树的最大子树节点数
    int sum = 0; //i子树的节点总数
    for (int i = h[u]; i != -1; i = ne[i])
    {
        int j = e[i];
        if (st[j]) continue;

        int s = dfs(j); //返回j子树的大小
        size = max(size, s);
        sum += s; //计算所有子树和
    }

    size = max(size, n - sum - 1); //i子树和与 非i子树部分的和,
    取最大值
    ans = min(ans, size);

    return sum+1; //加上i节点本身
}

int main()
{
    cin >> n;
    memset(h, -1, sizeof h);
    for (int i = 0; i < n-1; i ++ )
    {
        int a, b;
        cin >> a >> b;
        add(a, b), add(b, a); //无向边
    }
    dfs(1);
    cout << ans << endl;
    return 0;
}

```

Python

```

# 树的重心
import sys
sys.setrecursionlimit(200000)
n = int(input())
adj = [[] for _ in range(n + 1)]
for _ in range(n - 1):
    a, b = map(int, input().split())
    adj[a].append(b)
    adj[b].append(a)

ans = n
def dfs(u, fa):
    global ans
    sz = 1
    max_part = 0
    for v in adj[u]:
        if v != fa:
            sv = dfs(v, u)
            sz += sv
            max_part = max(max_part, sv)
    max_part = max(max_part, n - sz)
    ans = min(ans, max_part)
    return sz

dfs(1, -1)
print(ans)

```

"给一个有向图，求从1号点到n号点的最短距离（每条边长度为1）。"BFS天然适合！因为边权相同，BFS第一层是距离1的点，第二层距离2.....第一次遇到n号点时的距离就是最短距离。小鲁说："这和迷宫BFS本质上是一样的——只是图的连接关系用邻接表存储，而不是四方向规则。"

输入 第一行n,m。然后m行每行a b表示有向边。

输出 1到n的最短距离，不可达输出-1。

范围 $1 \leq n, m \leq 10^5$

输入	输出
4 5	1
1 2	
2 3	
3 4	
1 3	
1 4	

思路 BFS模板：queue存(node, dist)，vis数组标记访问。邻接表存图：vector adj[N]或list of lists。从1号出发BFS，遇到n时输出dist。每个节点最多访问一次， $O(n+m)$ 。BFS是边权相同的单源最短路径标准算法。

技巧 邻接表：Python用列表嵌套——adj=[[] for _ in range(n+1)]。BFS的queue用deque。dist数组初始化为-1表示未访问，dist[1]=0。BFS层层扩展的性质保证第一次遇到终点时的距离就是最短的。无权图的最短路=BFS。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>
#include <queue>

using namespace std;

const int N = 100007; // 有向图

int n, m;
int h[N], e[N], ne[N], idx;
int d[N];

void add(int a, int b) // 添加一条边a->b
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++ ;
}

int bfs()
{
    memset(d, -1, sizeof d);
    queue<int> q;
    d[1] = 0; // 起始点
    q.push(1);

    while (q.size())
    {
        auto t = q.front();
        q.pop();

        for (int i = h[t]; i != -1; i = ne[i])
        {
            int j = e[i];
            if (d[j] == -1)
            {
                d[j] = d[t] + 1; // 层次加1
                q.push(j);
            }
        }
    }
    return d[n];
}

int main()
{
    cin >> n >> m;
    memset(h, -1, sizeof h);

    for (int i = 0; i < m; i++)
    {
        int a, b;
        cin >> a >> b;
        add(a, b);
    }

    cout << bfs() << endl;
    return 0;
}

```

Python

```

# 图中点的层次
from collections import deque
n, m = map(int, input().split())
adj = [[] for _ in range(n + 1)]
for _ in range(m):
    a, b = map(int, input().split())
    adj[a].append(b)
dist = [-1] * (n + 1)
dist[1] = 0
q = deque([1])
while q:
    u = q.popleft()
    for v in adj[u]:
        if dist[v] == -1:
            dist[v] = dist[u] + 1
            q.append(v)
print(dist[n])

```

NQ141 字符串乘方 AcWing 777

"求一个字符串的最小循环节——即最小的 k 使得 s 等于某个字符串重复 k 次。"小华说，"方法1：枚举子串长度 len （必须是 n 的约数），检查所有对应位置字符是否相同。方法2：KMP的 $next$ 数组——最小循环节 $=n-next[n]$ （当 $n\%(n-next[n])==0$ 时）。前者 $O(n\times\text{约数个数})$ ，后者 $O(n)$ 。两种方法从不同角度展示了'周期性判断'的思维。"

输入 多行，每行一个字符串，以'.'结束。

输出 每行输出最大 k 值。

范围 字符串长度 $\leq 10^6$

输入	输出
abcd	1
aaaa	4
ababab	3
.	

思路 KMP法： $next[n]$ 是 n 的最长公共前后缀长度。若 $n\%(n-next[n])==0$ ，则最小循环节 $len=n-next[n]$ ， $k=n/len$ 。否则整个串是最大循环节($k=1$)。验证：aaaa中 $next[4]=3$ ， $n-next[n]=1$ ， $k=4$ 。枚举法：检查 len 为 n 的约数，验证所有 $s[i]==s[i\%len]$ 。

技巧 KMP的 $next$ 数组不仅用于匹配，还揭示了字符串的周期结构—— $len=n-next[n]$ 是最小可能的循环节长度。这个性质在字符串周期分析和压缩中非常有用。

C++

```
#include <iostream>

using namespace std;

int main()
{
    string str;

    while (cin >> str, str != ".")
    {
        int len = str.size();

        for (int n = len; n; n -- )
            if (len % n == 0)
            {
                int m = len / n;
                string s = str.substr(0, m);
                string r;
                for (int i = 0; i < n; i ++ ) r += s;

                if (r == str)
                {
                    cout << n << endl;
                    break;
                }
            }

        return 0;
    }
}
```

Python

```
# 字符串乘方
while True:
    s = input().strip()
    if s == '.':
        break
    n = len(s)
    ne = [0] * n
    j = 0
    for i in range(1, n):
        while j > 0 and s[i] != s[j]:
            j = ne[j - 1]
        if s[i] == s[j]:
            j += 1
        ne[i] = j
    length = n - ne[n - 1]
    if n % length == 0:
        print(n // length)
    else:
        print(1)
```

NQ142 字符串最大跨距 AcWing 778

"找字符串S中三个子串S1,S2,S3的位置——S1在最左，S2在最右，S3在S1和S2之间——求S1最右位置到S2最左位置的距离。"小鲁说："用find()从左往右找S1的最左出现，用rfind()从右往左找S2的最右出现。然后检查S3是否在它们之间合法存在。Python的str.find/rfind让这题行云流水。"

输入 一行字符串S，以及S1,S2,S3（用逗号隔开）。

输出 最右跨距，若无合法方案输出-1。

范围 S长度≤300

输入	输出
abcd123ab888efghij45ef67kl,ab,ef,45	18

思路 1)S.find(S1)→左边界L。2)S.rfind(S2)→右边界R。3)检查S[L+len1:R]中是否有S3且位置合法。4)R-(L+len1)就是跨距。若L==-1或R==-1或R

技巧 Python的find()返回第一次出现，rfind()返回最后一次出现。这是字符串操作的经典组合——左找+右找+中间检查。逗号分割输入：s=input().split(',')。本题训练的是对字符串多种查找操作的组合运用能力。

C++

```

#include <iostream>

using namespace std;

int main()
{
    string s, s1, s2;

    char c;
    while (cin >> c, c != ',') s += c;
    while (cin >> c, c != ',') s1 += c;
    while (cin >> c) s2 += c;

    if (s.size() < s1.size() || s.size() < s2.size()) put
s("-1");
    else
    {
        int l = 0;
        while (l + s1.size() <= s.size())
        {
            int k = 0;
            while (k < s1.size())
            {
                if (s[l + k] != s1[k]) break;
                k ++ ;
            }
            if (k == s1.size()) break;
            l ++ ;
        }

        int r = s.size() - s2.size();
        while (r >= 0)
        {
            int k = 0;
            while (k < s2.size())
            {
                if (s[r + k] != s2[k]) break;
                k ++ ;
            }
            if (k == s2.size()) break;
            r -- ;
        }

        l += s1.size() - 1;

        if (l >= r) puts("-1");
        else printf("%d\n", r - l - 1);
    }

    return 0;
}

```

Python

```

# 字符串最大跨距
s, s1, s2 = input().split(',')
n = len(s)
l = s.find(s1)
r = s.rfind(s2)
if l == -1 or r == -1 or l + len(s1)
> r:
    print(-1)
else:
    print(r - l - len(s1))

```



本章知识点总结

- DFS回溯：路径+选择列表+结束条件。回溯撤销操作是理解搜索的关键
- BFS：队列+距离数组。层层扩展保证最短路径（边权相同）
- 状态搜索：八数码——将棋盘编码为字符串做状态BFS

- **树形DFS：重心问题——理解树遍历+子树统计的DP模式**

— 第13章完 · 共8题 · 自强不息 —

图的疆域

第14章 · 图论入门 · 8题 · C++ & Python 双语对照

小鲁终于进入了算法竞赛中最大的领域——图论。"图可以描述世间万物——交通网络、社交关系、课程依赖、电路连接……理解图论算法就像获得了一个新的视角。"本章8道题覆盖图论四大基石：拓扑排序(DAG)、最短路(Dijkstra/SPFA/Floyd)、最小生成树(Prim/Kruskal)、二分图判定。"这些算法是竞赛和面试的高频考点——背下来、理解透。"

图论五大算法速查

- **拓扑排序**: Kahn BFS。入度为0节点入队→删除出边。 $O(n+m)$ 。判环
- **Dijkstra**: 贪心+松弛。朴素 $O(n^2)$ 稠密/堆优化 $O(m \log n)$ 稀疏。非负权
- **SPFA**: 队列优化Bellman-Ford。支持负权、判负环。平均 $O(m)$ 最坏 $O(nm)$
- **Floyd**: 三重循环 $dp[k][i][j]$ 。 $O(n^3)$ 。 $n \leq 200$ 多源最短路首选
- **MST**: Prim $O(n^2)$ 稠密, Kruskal $O(m \log m)$ 稀疏 (排序+并查集)

NQ143 拓扑序列 AcWing 848

小鲁在整理课程先修关系："高数→线代→概率论——这些课有先后依赖关系。"小华说："这就是有向图的拓扑排序！Kahn算法：每次选一个入度为0的节点输出，删除它所有出边。用队列维护当前入度为0的节点集合。如果最终输出的节点数小于 n ，说明图中有环——拓扑排序同时也是判环算法。"

输入 第一行 n, m 。然后 m 行每行 $x \ y$ 表示边 $x \rightarrow y$ 。

输出 一个拓扑序列，不存在输出-1。

范围 $1 \leq n, m \leq 10^5$

输入	输出
4 3	1 2 4 3
1 2	
2 3	
4 3	

思路 Kahn算法(BFS拓扑排序): 1)统计所有节点入度; 2)入度为0的节点入队; 3)while队列非空: 弹出队头 t 输出, 遍历 t 的所有邻居, 邻居入度-1, 若入度变为0则入队; 4)输出数

技巧 拓扑排序=有向无环图(DAG)的线性排列。应用: 课程安排、编译依赖、任务调度。Python: 邻接表+deque+入度数组。拓扑排序是算法竞赛中最基础的图论算法之一——必须熟练掌握。

C++

```

#include <iostream>
#include <algorithm>
#include <queue>
#include <cstring>

using namespace std;
#define For(a, begin, end) for (int a = (begin); a < (end); a++)
const int N = 1000007;

int n, m;
int head[N], edge[N], nextVertex[N], idx; //邻接表
int q[N], d[N]; //队列, 入度

void add(int a, int b)
{
    edge[idx] = b;
    nextVertex[idx] = head[a];
    head[a] = idx++;
}

bool topsort()
{
    int qHeadIdx = 0, qTailIdx = -1; //队头, 队尾
    //从前往后遍历入度为0的点, 插入队列
    for (int i = 1; i <= n; i++)
        if (!d[i])
            q[++qTailIdx] = i; //数组模拟队列, 入队是尾指针+1
    while (qHeadIdx <= qTailIdx) //如果头指针小于尾指针
    {
        int t = q[qHeadIdx++]; //取队列头元素,, 出队只是把头指针往后移动一位
        //遍历t的临边, 空指针初始化为-1
        for (int i = head[t]; i != -1; i = nextVertex[i])
        {
            int j = edge[i]; //取到出边
            d[j]--; //入度减一
            if (d[j] == 0)
                q[++qTailIdx] = j; //如若入度为0, 压入队列
        }
    }
    //判断是否所有顶点都入队
    //也就是头指针qTailIdx是否等于n-1, 即所有点都进入队列了
    return qTailIdx==n-1;
}

int main()
{
    cin >> n >> m; //读入n,m
    memset(head, -1, sizeof(head)); //初始化Head数组指向-1顶点

    For(i, 0, m)
    {
        int a, b;
        cin >> a >> b;
        add(a, b); //插入边
        d[b]++; //更新入度
    }

    if (topsort())
        //数组队列里面的元素就是拓扑序

```

Python

```

import sys
from collections import deque
data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
adj = [[] for _ in range(n+1)]
indeg = [0]*(n+1)
idx = 2
for _ in range(m):
    x, y = int(data[idx]), int(data[idx+1])
    idx += 2
    adj[x].append(y); indeg[y] += 1
q = deque([i for i in range(1,n+1) if indeg[i]==0])
res = []
while q:
    u = q.popleft()
    res.append(str(u))
    for v in adj[u]:
        indeg[v] -= 1
        if indeg[v] == 0: q.append(v)
print(' '.join(res) if len(res)==n else -1)

```

```

    for (int i = 0; i < n; i++) printf("%d ", q[i]);
}
else
    puts("-1");
return 0;
}

```

NQ144 Dijkstra I AcWing 849

"求图中从1号点到n号点的最短路径——每条边有正权重。"小华说，"Dijkstra算法——贪心法。每次从未确定最短距离的点中选距离最小的，用它去松弛(relax)它的邻居。朴素版每次 $O(n)$ 找最小值， n 次循环 $\rightarrow O(n^2)$ 。适合稠密图(n 不大但边多)。Python的heapq优化版更适合稀疏图， $O(m \log n)$ 。"

输入 第一行 n, m 。然后 m 行每行 $x \ y \ z$ 表示边 $x \rightarrow y$ 权重 z 。

输出 1到 n 的最短距离。

范围 $1 \leq n \leq 500, 1 \leq m \leq 10^5$

输入	输出
3 3	3
1 2 2	
2 3 1	
1 3 4	

思路 Dijkstra朴素版： $\text{dist}[1]=0$ ，其余 INF 。 n 次循环：每次找未被标记的最近节点 t ，标记 t ，用 t 松弛所有邻居 v ： $\text{dist}[v]=\min(\text{dist}[v], \text{dist}[t]+w[t][v])$ 。 $O(n^2+m)$ 。证明：贪心选择正确性基于边权非负（边权为负时应用Bellman-Ford/SPFA）。

技巧 Dijkstra只适用于非负权图。找最近节点的 $O(n)$ 可以用堆优化降到 $O(\log n)$ 。Python: $\text{heap}=(\text{dist}[i], i)$ 入堆，弹堆时检查是否过期。朴素版 $n \leq 500$ 时 $O(n^2) \approx 250K$ ——完全够用。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 510; // 500*500
const int INF = 0x3f3f3f3f;

int g[N][N];
int dist[N];
bool st[N];

int n, m;

int Dijkstra()
{
    memset(dist, 0x3f, sizeof dist); // 初始化距离 0x3f代表无限大

    dist[1] = 0; // 第一个点到自身的距离为0

    // 有n个点, 迭代n次
    for (int i = 0; i < n; i++)
    {
        int t = -1; // 当前访问点的编号初始为-1

        for (int j = 1; j <= n; j++) // 从点1到点n开始计算dist
            if (!st[j] && (t == -1 || dist[t] > dist[j]))
                t = j; // 更新点t, 直到找到dist最小的点

        st[t] = true; // 标记此点确定

        for (int j = 1; j <= n; j++) // 每个点都要遍历所有点一遍
            dist[j] = min(dist[j], dist[t] + g[t][j]); // 遍历所有的点, 求从点t到点j的最小值
    }

    if (dist[n] == INF) return -1;
    return dist[n]; // 返回第n个点的最短距离
}

int main()
{
    cin >> n >> m;

    memset(g, 0x3f, sizeof g);

    while (m--)
    {
        int x, y, z;
        cin >> x >> y >> z;
        g[x][y] = min(g[x][y], z); // 取重边和自环的最小值, 保留一条边
    }

    cout << Dijkstra() << endl;

    return 0;
}

```

Python

```

# Dijkstra I (朴素版)
n, m = map(int, input().split())
g = [[0x3f3f3f3f] * (n + 1) for _ in range(n + 1)]
for _ in range(m):
    x, y, z = map(int, input().split())
    g[x][y] = min(g[x][y], z)
dist = [0x3f3f3f3f] * (n + 1)
dist[1] = 0
st = [False] * (n + 1)
for _ in range(n):
    t = -1
    for i in range(1, n + 1):
        if not st[i] and (t == -1 or dist[i] < dist[t]):
            t = i
    st[t] = True
    for v in range(1, n + 1):
        if dist[v] > dist[t] + g[t][v]:
            dist[v] = dist[t] + g[t][v]
    [v]
print(dist[n] if dist[n] != 0x3f3f3f3f else -1)

```

NQ145 Dijkstra II AcWing 850

"上一题 $n=500$, $O(n^2)$ 没问题。但如果 $n=10^5$ 呢?"小华说, "需要用堆优化——优先队列每次 $O(\log n)$ 找最近节点。总复杂度 $O(m \log n)$ 。注意堆中可能有过期的旧值——弹堆时检查 $dist$ 是否已更新。STL的`priority_queue`不支持修改键值, 所以有重复入堆的策略。"

输入 第一行 n, m 。然后 m 行每行 $x \ y \ z$ 。

输出 1到 n 的最短距离, 不可达输出 -1 。

范围 $1 \leq n, m \leq 1.5 \times 10^5$, 边权 ≤ 10000

输入	输出
3 3	3
1 2 2	
2 3 1	
1 3 4	

思路 堆优化Dijkstra: $dist[1]=0$, $heap.push(\{0,1\})$ 。while heap非空: 弹出 $\{d,u\}$ (若 $d > dist[u]$ 则skip过期), 标记 $st[u]$, 遍历所有邻居 v : if($dist[v] > dist[u] + w$): $dist[v] = dist[u] + w$; $heap.push(\{dist[v], v\})$ 。 $O(m \log n)$ 。

技巧 Python用`heapq`: `heappush(heap,(dist[v],v))`。过期检测: if $d > dist[u]$: continue。这是C++和Python都通用的堆优化模板——配上邻接表就是标准的Dijkstra优化版。背下这个模板, 面试/竞赛都够用。

C++

```

#include <algorithm>
#include <cstring>
#include <iostream>
#include <queue>

using namespace std;

typedef pair<int, int> PII;

const int N = 1e6 + 10;

int n, m; //行, 列
int h[N], w[N], e[N], ne[N], idx; //数组模拟链表
int dist[N]; //距离矩阵
bool st[N]; //状态矩阵

//建边函数 (数组模拟链表)
void add(int a, int b, int c) {
    e[idx] = b, w[idx] = c, ne[idx] = h[a], h[a] = idx++;
}

//改进版, 使用小根堆
int dijkstra() {
    //初始化距离矩阵
    memset(dist, 0x3f, sizeof dist);
    dist[1] = 0;
    //调用优先队列, 以及greater算子, 创建小根堆
    priority_queue<PII, vector<PII>, greater<PII>> heap;

    //按照距离排序, 因此第一个元素是距离, 第二个元素是顶点编号
    heap.push({0, 1});

    // BFS计算最短距离
    while (heap.size()) {
        //取堆头元素
        auto t = heap.top();
        heap.pop();

        // 取顶点编号、顶点距离
        int ver = t.second, distance = t.first;

        // 顶点遍历过就跳过, 不展开改顶点
        if (st[ver]) continue;
        st[ver] = true;

        //遍历所有相邻顶点
        for (int i = h[ver]; i != -1; i = ne[i]) {
            int j = e[i];
            //判断dist[ver]+w[i]是否比原来顶点j的距离大
            if (dist[j] > dist[ver] + w[i]) {
                dist[j] = dist[ver] + w[i]; //更新顶点j的距离值
                heap.push({dist[j], j}); //把更小的距离值入堆
            }
        }
    }

    //如果第n点距离值没被更新, 表明不可达
    if (dist[n] == 0x3f3f3f3f) return -1;
    //返回点n的距离值
    return dist[n];
}

int main() {

```

Python

```

import sys
import heapq
data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
adj = [[] for _ in range(n+1)]
idx = 2
for _ in range(m):
    x, y, z = int(data[idx]), int(data[idx+1]), int(data[idx+2])
    idx += 3
    adj[x].append((y, z))
INF = 10**15
dist = [INF]*(n+1)
dist[1] = 0
st = [False]*(n+1)
heap = [(0, 1)]
while heap:
    d, u = heapq.heappop(heap)
    if st[u]: continue
    st[u] = True
    for v, w in adj[u]:
        if dist[v] > d + w:
            dist[v] = d + w
            heapq.heappush(heap, (dist[v], v))
print(dist[n] if dist[n] != INF else -1)

```

```

//读入行n, 列m
scanf("%d%d", &n, &m);
//把邻接链表的头指针化为-1, 表明当前为空, 不指向任何链表
memset(h, -1, sizeof h);
//处理m条边
while (m--) {
    int a, b, c;
    scanf("%d%d%d", &a, &b, &c);
    add(a, b, c); //调用add建图
}
//调用改进版的dijkstra()
cout << dijkstra() << endl;

return 0;
}

```

NQ146 spfa求最短路 AcWing 851

"Dijkstra不能处理负权边——Bellman–Ford可以但 $O(n \times m)$ 太慢。SPFA是Bellman–Ford的队列优化版——只有被松弛过的节点才入队去松弛别人。平均 $O(m)$ ，最坏 $O(n \times m)$ 可能被卡。"小华补充，"SPFA还能判断负环——如果一个节点入队超过 n 次，存在负环。虽然竞赛中经常被卡，但实现简单、思路巧妙。"

输入 第一行 n, m 。然后 m 行每行 $x \ y \ z$ （可能有负权）。

输出 1到 n 最短路，有负环输出impossible。

范围 $1 \leq n, m \leq 10^5$

输入	输出
3 3	3
1 2 1	
2 3 2	
1 3 4	

思路 SPFA: $\text{dist}[1]=0$, $q.\text{push}(1)$, $\text{st}[1]=\text{true}$ 。while q 非空: $t=q.\text{front}()$; $q.\text{pop}()$; $\text{st}[t]=\text{false}$ 。遍历 t 的邻居 v : $\text{if}(\text{dist}[v]>\text{dist}[t]+w)$: $\text{dist}[v]=\text{dist}[t]+w$; $\text{if}(!\text{st}[v])$ $q.\text{push}(v)$, $\text{st}[v]=\text{true}$ 。复杂度: 平均 $O(m)$ 最坏 $O(nm)$ 。

技巧 st 数组记录节点是否在队列中——避免重复入队。负环检测: 维护 $\text{cnt}[v]$ =到 v 的最短路边数, 若 $\text{cnt}[v] \geq n$ 则有负环。SPFA容易被构造数据卡掉——正权图用Dijkstra更安全。但理解SPFA的松弛传播思想对理解更多图论算法有帮助。

C++

```

#include <queue>
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007;

int n,m;
int h[N], e[N], w[N], ne[N], idx;
int q[N], dist[N];
bool st[N];

void add(int a, int b, int c) // 添加一条边a->b, 边权为c
{
    e[idx] = b, w[idx] = c, ne[idx] = h[a], h[a] = idx ++;
}

int spfa() // 求1号点到n号点的最短路距离, 如果从1号点无法走到n号点则返回-1
{
    int hh = 0, tt = 0;
    memset(dist, 0x3f, sizeof dist);
    dist[1] = 0;
    q[tt ++ ] = 1;
    st[1] = true;

    while (hh != tt)
    {
        int t = q[hh ++ ];
        if (hh == N) hh = 0;
        st[t] = false;

        for (int i = h[t]; i != -1; i = ne[i])
        {
            int j = e[i];
            if (dist[j] > dist[t] + w[i])
            {
                dist[j] = dist[t] + w[i];
                if (!st[j]) // 如果队列中已存在j, 则不需要
                    将j重复插入
                {
                    q[tt ++ ] = j;
                    if (tt == N) tt = 0;
                    st[j] = true;
                }
            }
        }
    }

    if (dist[n] == 0x3f3f3f3f) return -1;
    return dist[n];
}

int main()
{
    scanf("%d%d", &n, &m);
    memset(h, -1, sizeof h);
    while (m -- )
    {

```

Python

```

import sys
from collections import deque
data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
adj = [[] for _ in range(n+1)]
idx = 2
for _ in range(m):
    x, y, z = int(data[idx]), int(data[idx+1]), int(data[idx+2])
    idx += 3
    adj[x].append((y, z))
INF = 10**15
dist = [INF]*(n+1)
dist[1] = 0
st = [False]*(n+1)
q = deque([1])
st[1] = True
while q:
    u = q.popleft()
    st[u] = False
    for v, w in adj[u]:
        if dist[v] > dist[u] + w:
            dist[v] = dist[u] + w
            if not st[v]:
                q.append(v); st[v] = True
print(dist[n] if dist[n] != INF else 'impossible')

```

```

int a, b, c;
scanf("%d%d%d", &a, &b, &c);
add(a, b, c);
}

int t = spfa();

if (t == -1) puts("impossible");
else printf("%d\n", t);

return 0;
}

```

NQ147 Floyd求最短路 AcWing 854

"如果要查询任意两点之间的最短距离——多源最短路径——怎么办?"小华说, "Floyd-Warshall算法! 三重循环——for k,i,j: dist[i][j]=min(dist[i][j], dist[i][k]+dist[k][j])。这个简洁的三行代码本质是动态规划——dp[k][i][j]表示只经过前k个节点作中间节点时i到j的最短距离, 压缩掉k维就是最终的重循环。 $O(n^3)$ 但 $n \leq 200$ 时完全OK。"

输入 第一行n,m,q。然后m行每行x y z。然后q行每行a b查询。

输出 每行查询结果, 不可达输出impossible。

范围 $1 \leq n \leq 200, 1 \leq q \leq 10^5$

输入	输出
3 3 2	impossible
1 2 1	3
2 3 2	
1 3 4	
2 1	
1 3	

思路 Floyd模板: 初始化dist[i][i]=0, dist[i][j]=INF。读入边权(注意去重——可能有多条边取min)。三重循环: for k: for i: for j: d[i][j]=min(d[i][j], d[i][k]+d[k][j])。时间 $O(n^3)=8 \times 10^6$ ($n=200$), 空间 $O(n^2)$ 。注意INF不要用0x3f3f3f3f以免溢出。

技巧 Floyd是 $n \leq 200$ 的多源最短路径首选。它能同时处理正负权边(不能有负环)。k循环必须在最外层——这是理解Floyd为DP的直接体现。Python三重循环可能较慢, $n=200$ 时约需0.5秒。INF设为 10^9 避免溢出。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 207, INF = 0x3f3f3f3f;

int n, m, k; // k组询问
int d[N][N];

void floyd()
{
    // 用动态规划分析得出这个递推公式，使用三重循环
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}

int main()
{
    scanf("%d%d%d", &n, &m, &k);

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (i == j) d[i][j] = 0;
            else d[i][j] = INF;

    // 处理每条边，去掉重边和自环
    while (m--)
    {
        int a, b, c;
        scanf("%d%d%d", &a, &b, &c);
        d[a][b] = min(d[a][b], c);
    }

    floyd();

    // k组查询
    while (k--)
    {
        int a, b;
        scanf("%d%d", &a, &b);
        int t = d[a][b];
        if (t > INF / 2) puts("impossible");
        else printf("%d\n", t);
    }

    return 0;
}

```

Python

```

import sys
data = sys.stdin.read().split()
idx = 0
n, m, q = int(data[idx]), int(data[idx+1]), int(data[idx+2])
idx += 3
INF = 0x3f3f3f3f
d = [[INF] * (n + 1) for _ in range(n + 1)]
for i in range(1, n + 1):
    d[i][i] = 0
for _ in range(m):
    x, y, z = int(data[idx]), int(data[idx+1]), int(data[idx+2])
    idx += 3
    if z < d[x][y]:
        d[x][y] = z
for k in range(1, n + 1):
    for i in range(1, n + 1):
        if d[i][k] == INF:
            continue
        for j in range(1, n + 1):
            if d[k][j] != INF and d[i][j] > d[i][k] + d[k][j]:
                d[i][j] = d[i][k] + d[k][j]
[k][j]
out = []
for _ in range(q):
    a, b = int(data[idx]), int(data[idx+1])
    idx += 2
    out.append(str(d[a][b]) if d[a][b] < INF // 2 else 'impossible')
sys.stdout.write('\n'.join(out) + '\n')

```

NQ148 Prim AcWing 858

"最小生成树——连接所有节点的最小总权重。"Prim算法——从一个点开始，每次选距离当前集合最近的节点加入。这和Dijkstra几乎一模一样！区别只在于dist的定义：Dijkstra是到起点的距离，Prim是到当前集合的距离。代码只差一行。"小华总结道，"Prim适合稠密图 $O(n^2)$ ，Kruskal适合稀疏图 $O(m \log m)$ 。"

输入

第一行n,m。然后m行每行u v w（无向边）。

输出

最小生成树的总权重，不连通输出impossible。

范围 $1 \leq n \leq 500, 1 \leq m \leq 10^5$

输入

```

4 5
1 2 1
1 3 2
1 4 3
2 3 2
3 4 4

```

输出

```

6

```

思路 Prim朴素版：dist[i]=到当前生成树的最近距离。n次循环：每次找未标记的最近点t，标记t，ans+=dist[t]。用t更新所有邻居v的dist[v]=min(dist[v], w[t][v])。O(n²+m)。与Dijkstra的唯一区别：dist的更新公式是w[t][v]而非dist[t]+w[t][v]。

技巧 Prim=Dijkstra改一行。Dijkstra: dist[v]=min(dist[v], dist[t]+w)。Prim: dist[v]=min(dist[v], w)。这个区别体现了两者的本质差异——一个求路径累积最短，一个求集合距离最近。稠密图(n≤500)用Prim O(n²)，稀疏图用Kruskal。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 507, INF = 0x3f3f3f3f;

int n, m;
int g[N][N];
int dist[N];
bool st[N];

int prim()
{
    memset(dist, 0x3f, sizeof dist);

    int res = 0;
    for (int i = 0; i < n; i++)
    {
        //第一个点
        int t = -1;
        for (int j = 1; j <= n; j++)
            if (!st[j] && (t == -1 || dist[t] > dist[j]))
                t = j;

        if (i && dist[t] == INF) return INF; //没有最小生成树, 不连通

        if (i) res += dist[t];
        st[t] = true;

        for (int j = 1; j <= n; j++) dist[j] = min(dist[j], g[t][j]);
    }
    return res;
}

int main()
{
    scanf("%d%d", &n, &m);

    memset(g, 0x3f, sizeof g);

    while (m--)
    {
        int a, b, c;
        scanf("%d%d%d", &a, &b, &c);
        g[a][b] = g[b][a] = min(g[a][b], c); //无向图, 去重
    }

    int t = prim();
    if (t == INF) puts("impossible");
    else printf("%d\n", t);

    return 0;
}

```

Python

```

import sys
data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
g = [[10**15]*(n+1) for _ in range(n+1)]
idx = 2
for _ in range(m):
    u, v, w = int(data[idx]), int(data[idx+1]), int(data[idx+2])
    idx += 3
    g[u][v] = g[v][u] = min(g[u][v], w)
INF = 10**15
dist = [INF]*(n+1)
st = [False]*(n+1)
ans = 0
for _ in range(n):
    t = -1
    for i in range(1, n+1):
        if not st[i] and (t == -1 or dist[i] < dist[t]): t = i
        if _ > 0 and dist[t] == INF: print('impossible'); sys.exit(0)
        if _ > 0: ans += dist[t]
        st[t] = True
        for v in range(1, n+1):
            if dist[v] > g[t][v]: dist[v] = g[t][v]
print(ans)

```

"Kruskal——另一种MST算法。把所有边按权重排序，从小到大尝试每条边——如果边的两端不在同一集合中（并查集判断），就加入这条边。贪心法！因为排序后轻边优先加入，保证了树的总权重最小。需要并查集维护连通性——find+union操作。Kruskal=排序+并查集——两个简单工具组合出强大算法。"

输入 第一行n,m。然后m行每行u v w。

输出 MST总权重，不连通输出impossible。

范围 $1 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5$

输入	输出
4 5	6
1 2 1	
1 3 2	
1 4 3	
2 3 2	
3 4 4	

思路 Kruskal: 1)所有边按w升序排序; 2)初始化并查集($p[i]=i$); 3)遍历每条边(u,v,w): 若 $\text{find}(u) \neq \text{find}(v)$ 则 $\text{union}(u,v)$, $\text{ans}+=w$, $\text{cnt}++$; 4)若 $\text{cnt}==n-1$ 则连通输出ans, 否则impossible。时间 $O(m \log m)$ (排序主导)。

技巧 Kruskal=排序+并查集。并查集: $p[x]$ 存父节点, $\text{find}(x)$ 路径压缩返回根, $\text{union}(a,b)$ 将a根连到b根。排序 $O(m \log m)$ 是Kruskal的瓶颈——稀疏图($m \approx n$)时 $\log m$ 小因此Kruskal快于Prim $O(n^2)$ 。稠密图反之。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007, M = 2000007, INF = 0x3f3f3f3f;

int n, m;
int p[N]; // 并查集

struct Edge
{
    int a, b, w;
    bool operator< (const Edge &a) const
    {
        return w < a.w;
    }
} edges[M]; // 重载<, 安装权重w比大小

int find(int x) // 并查集
{
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}

int kruskal()
{
    // 1. 排序
    sort(edges, edges + m);
    // 并查集初始化
    for (int i = 1; i <= n; i++) p[i] = i;

    int res = 0, cnt = 0;
    for (int i = 0; i < m; i++)
    {
        int a = edges[i].a, b = edges[i].b, w = edges[i].w;
        a = find(a), b = find(b);
        if (a != b)
        {
            p[a] = b;
            res += w;
            cnt++;
        }
    }
    if (cnt < n - 1) return INF;
    return res;
}

int main()
{
    scanf("%d%d", &n, &m);

    for (int i = 0; i < m; i++)
    {
        int a, b, w;
        scanf("%d%d%d", &a, &b, &w);
        edges[i] = {a, b, w};
    }

    int t = kruskal();

    if (t == INF) puts("impossible");
}

```

Python

```

# Kruskal
n, m = map(int, input().split())
edges = []
for _ in range(m):
    u, v, w = map(int, input().split())
    edges.append((w, u, v))
edges.sort()
p = list(range(n + 1))

def find(x):
    if p[x] != x:
        p[x] = find(p[x])
    return p[x]

ans = 0
cnt = 0
for w, u, v in edges:
    ru, rv = find(u), find(v)
    if ru != rv:
        p[ru] = rv
        ans += w
        cnt += 1
print(ans if cnt == n - 1 else 'impossible')

```

```

else printf("%d\n", t);

return 0;
}

```

NQ150 染色法判定二分图 AcWing 860

"二分图——能把所有节点分成两组，使所有边的两端都在不同组中。用染色法判断：从一个未染色节点出发，染成1，它的所有邻居染成2，邻居的邻居染成1.....如果发现一条边的两端同色——不是二分图。这就是二染色判定——本质是BFS/DFS遍历中检查冲突。是匈牙利算法(最大匹配)的前置知识。"

输入 第一行n,m。然后m行每行u v表示无向边。

输出 是二分图输出Yes否则No。

范围 $1 \leq n, m \leq 10^5$

输入	输出
4 4	Yes
1 2	
2 3	
3 4	
4 1	

思路 二染色判定：color数组初始0(未染色)。遍历每个节点，若未染色则BFS/DFS：染色1，邻居染色-1(或2)，邻居的邻居染色1.....若遇到已染色且颜色与期望不符→冲突→不是二分图。 $O(n+m)$ 。BFS和DFS都行——DFS递归简单但深度大时需注意栈溢出。

技巧 二分图=不包含奇环。染色过程本质是BFS涂色——涂色冲突=奇环存在。Python的DFS递归可能栈溢出→用BFS或迭代DFS。二分图判定是图论中的基础算法——后续最大匹配、最小点覆盖都依赖于此。

C++

```

#include <iostream>
#include <algorithm>
#include <cstring>
using namespace std;
//无向图
const int N=100007,M=200007;//两倍的边
int n,m;
int h[N],e[M],ne[M],idx;
int color[N];//0表示未访问, 1表示颜色1, 2表示颜色2.

void add(int a, int b){
    e[idx]=b, ne[idx]=h[a], h[a]=idx++; //数组模拟链表add操作
}

bool dfs(int u, int c){
    color[u]=c;

    for (int i=h[u]; i!=-1; i=ne[i])
    {
        int j=e[i]; //取i对应的另外一个顶点编号
        if (!color[j]) {
            if (!dfs(j,3-c)) return false;//颜色是1或者2切换, 所以
            用3-color可以交换两种颜色
        }
        else if (color[j]==c) return false;//端点颜色相同
    }
    return true;
}

int main(){

    scanf("%d%d",&n,&m);
    memset(h,-1,sizeof h);//初始化头结点
    //建图
    while (m--){
        int a,b;
        scanf("%d%d",&a,&b);
        add(a,b); add(b,a);
    }
    bool flag = true;//表示染色有矛盾
    for (int i=1; i<=n; i++){
        if (!color[i]) {
            if (!dfs(i,1))
            {
                flag = false;
                break;
            }
        }
    }
    if (flag) puts("Yes"); else puts ("No");

    return 0;
}

```

Python

```

import sys
sys.setrecursionlimit(200000)
# 染色法判定二分图
n, m = map(int, input().split())
adj = [[] for _ in range(n + 1)]
for _ in range(m):
    u, v = map(int, input().split())
    adj[u].append(v)
    adj[v].append(u)
color = [0] * (n + 1)

def dfs(u, c):
    color[u] = c
    for v in adj[u]:
        if color[v] == 0:
            if not dfs(v, 3 - c):
                return False
        elif color[v] == c:
            return False
    return True

ok = True
for i in range(1, n + 1):
    if color[i] == 0:
        if not dfs(i, 1):
            ok = False
            break
print('Yes' if ok else 'No')

```

本章知识点总结

- 图论四大基石：拓扑排序、最短路、最小生成树、二分图——背下模板终身受用
- Dijkstra vs SPFA vs Floyd：正权选Dijkstra(堆)，负权选SPFA(慎用)，多源 $n \leq 200$ 选Floyd

- **Prim vs Kruskal:** Prim=改一行Dijkstra, 稠密 $O(n^2)$ 。Kruskal=排序+并查集, 稀疏 $O(m \log m)$

— 第14章完 · 共8题 · 自强不息 —

状态的艺术

第15章 · 动态规划 · 8题 · C++ & Python 双语对照

小鲁进入了算法竞赛的核心——**动态规划(DP)**。DP不是一种算法，而是一种思想——将大问题拆成小问题，用小问题的解构造大问题的解。DP的精髓在于**状态定义**和**转移方程**。本章8题覆盖DP的四大类型：背包DP、线性DP、区间DP、记忆化搜索。学完本章，你才是一个真正的算法选手。

DP四大类型速查

- **背包DP**：01(倒序)、完全(正序)。物品选或不选→二维压一维的关键是遍历顺序
- **线性DP**：LIS(一维)、LCS(二维)、编辑距离(二维)。字符串/序列问题的标准模型
- **区间DP**：石子合并——len从小到大，k分割区间。 $O(n^3)$ 三重循环模板
- **记忆化搜索**：DFS+备忘录=自顶向下DP。滑雪、树形DP。递归优雅但注意栈深度

NQ151 01背包问题 AcWing 2

小鲁终于进入了算法竞赛的巅峰领域——**动态规划(DP)**。"01背包：N件物品，每件有体积和价值。背包容量V——选哪些物品使总价值最大？"小华说，" $f[i][j]$ =前i件物品用j容量能获得的最大价值。每件选或不选： $f[i][j]=\max(f[i-1][j], f[i-1][j-v[i]]+w[i])$ 。这就是DP的核心——用子问题的解构造原问题的解。优化：将二维压成一维，j从大到小遍历。"

输入 第一行N,V。然后N行每行 $v_i w_i$ 。

输出 最大总价值。

范围 0

输入	输出
4 5	8
1 2	
2 4	
3 4	
4 5	

思路 二维DP： $f[i][j]=\max(f[i-1][j], f[i-1][j-v]+w)$ 。一维优化： $f[j]=\max(f[j], f[j-v]+w)$ ，j从V倒序到v（防止重复使用）。时间 $O(NV)$ ，空间 $O(V)$ 。初始化 $f[0]=0$ （全0初值表示恰好装满？不——恰好装满需 $f[1..V]=-INF$ ）。

技巧 01背包的变形：完全背包(j正序)、多重背包(二进制拆分)。一维优化的关键是j倒序——保证每件物品只被选一次。DP的五步法：1)确定状态定义 2)初始化 3)转移方程 4)遍历顺序 5)答案位置。这个框架适用于所有DP问题。

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 1007;
// 2维的解法
int n, m;
int f[N][N];
int v[N], w[N];

int main()
{
    cin >> n >> m; // 读入N, V
    // 读入v, w
    for (int i = 1; i <= n; i++) cin >> v[i] >> w[i];
    // 朴素的二维dp
    for (int i = 1; i <= n; i++)
        for (int j = 0; j <= m; j++)
        {
            f[i][j] = f[i-1][j];
            if (j >= v[i]) // 剩余体积大于第i项的体积
                f[i][j] = max(f[i][j], f[i-1][j-v[i]] + w[i]);
        }
    // 根据定义，从前n个物品中，选取总体积不超过m的选法集合f[n][m]就是答案
    cout << f[n][m] << endl;
}
```

Python

```
# 01背包问题
N, V = map(int, input().split())
f = [0] * (V + 1)
for _ in range(N):
    v, w = map(int, input().split())
    for j in range(V, v - 1, -1):
        f[j] = max(f[j], f[j - v] + w)
print(f[V])
```

NQ152 完全背包问题 AcWing 3

"01背包每件只能选一次——如果每件可以选无限次呢？"小华问。"那就是完全背包！"小鲁推导道，" $f[i][j] = \max(f[i-1][j], f[i][j-v] + w)$ ——注意第二项是 $f[i]$ 不是 $f[i-1]$ ，因为选了之后 i 仍可选。优化到一维： j 正序从 v 到 V ——正好体现'可重复选'的特点。01倒序、完全正序——两行代码的区别背后是完全不同的DP语义。"

输入 第一行 N, V 。然后 N 行每行 $v_i w_i$ 。

输出 最大总价值。

范围 0

输入	输出
4 5	10
1 2	
2 4	
3 4	
4 5	

思路 完全背包一维：for i: for j from v to V: $f[j] = \max(f[j], f[j-v] + w)$ 。j正序遍历！因为同一件物品可以被多次选取——正序遍历时 $f[j-v]$ 是已更新过的第 i 层状态，允许重复选。时间 $O(NV)$ ，空间 $O(V)$ 。

技巧 完全背包正序、01背包倒序——背后是同一个转移方程在不同遍历顺序下的不同效果。理解这一点，多重背包的二进制拆分也就懂了。完全背包的初始化同样要注意是否恰好装满。

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 1007;

int n, m;
int f[N][N];
int v[N], w[N];

int main()
{
    cin >> n >> m;
    for (int i = 1; i <= n; i++) cin >> v[i] >> w[i];

    //完全背包
    // 1.f[i][j] = f[i-1][j]; (不选物品i)
    // 2.f[i][j] = max(f[i-1][j], f[i][j-v[i]]+w[i]) (选若干个物品i)
    for (int i = 1; i <= n; i++)
        for (int j = 0; j <= m; j++)
        {
            f[i][j] = f[i-1][j];
            if (j >= v[i])
                f[i][j] = max(f[i-1][j], f[i][j-v[i]]+w[i]);
        }

    cout << f[n][m] << endl;
    return 0;
}
```

Python

```
# 完全背包问题
N, V = map(int, input().split())
f = [0] * (V + 1)
for _ in range(N):
    v, w = map(int, input().split())
    for j in range(v, V + 1):
        f[j] = max(f[j], f[j - v] + w)
print(f[V])
```

NQ153 数字三角形 AcWing 898

"一个三角形数塔——从顶部出发，每次可以向下走左或右，求到达底层的最大路径和。"小华画出三角，"这是线性DP的入门—— $f[i][j]$ = 三角形第*i*行第*j*列到达底层的最大和。自底向上推： $f[i][j] = \max(f[i+1][j], f[i+1][j+1]) + a[i][j]$ 。顶部 $f[0][0]$ 就是答案。也可以自顶向下推——但自底向上不需要处理边界。"

输入

第一行n。然后n行三角形数字。

输出

最大路径和。

范围

$1 \leq n \leq 500$, $-10000 \leq \text{值} \leq 10000$

输入

5
7
3 8

输出

30

```
8 1 0
2 7 4 4
4 5 2 6 5
```

思路 自底向上DP：从倒数第二行开始， $f[i][j] = \max(f[i+1][j], f[i+1][j+1]) + a[i][j]$ 。顶部 $f[0][0]$ 为答案。时间 $O(n^2)$ ，空间 $O(n^2)$ (滚动数组可 $O(n)$)。这是最短路径DP和三角形DP的最基本形态。

技巧 自底向上避免了边界检查——每一层总有下一层。自顶向下需要处理 $j=0$ 和 $j=i$ 的边界。这是一个重要经验：DP的迭代方向选择直接影响代码复杂度。数字三角形也是树形DP和网格DP的简化版。

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 507;

int n;
int w[N][N], f[N][N]; //三角矩阵依旧用2维存储

int main()
{
    //读入n, w
    cin >> n;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= i; j++)
            cin >> w[i][j];

    //初始化最底层的f[n][j]
    for (int i = 1; i <= n; i++) f[n][i] = w[n][i];

    //用dp从底层往上计算每个f[i][j]的值
    for (int i = n-1; i; i--)
        for (int j = 1; j < n; j++)
            f[i][j] = max(f[i+1][j] + w[i][j], f[i+1][j+1] + w[i][j]);

    cout << f[1][1] << endl;
    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
n = int(data[0])
idx = 1
a = []
for i in range(n):
    a.append([int(data[idx+j]) for j
in range(i+1)])
    idx += i + 1
for i in range(n-2, -1, -1):
    for j in range(i+1):
        a[i][j] += max(a[i+1][j], a[i+1][j+1])
print(a[0][0])
```

NQ154 最长上升子序列 AcWing 895

"LIS——最长上升子序列。给一个数组，找最长的严格递增的子序列（可以不连续）。"小华说，"朴素DP： $f[i] = \text{以} a[i] \text{结尾的 LIS 长度} = \max(f[j] + 1) \text{ for } j$

输入 第一行 N 。第二行 N 个整数。

输出 LIS长度。

范围 $1 \leq N \leq 100000$

输入

```
7
3 1 2 1 8 5 6
```

输出

```
4
```

思路 朴素 $O(n^2)$: 对每个 i 遍历 j

技巧 LIS的优化是用贪心维护'最小结尾值'——保证tails数组始终是递增的, 可以用二分。这个优化思路也用于LIS的变种 (LNDS/带权LIS)。DP+贪心+二分=算法设计中最经典的技术融合。

C++

```
#include <iostream>
#include <algorithm>
#include <cstring>
using namespace std;
// f[i] 定义为以s[i]结尾的最长的子序列
// 集合: 所有以第i个数结尾的上升子序列集合
// 属性: Max 上升子序列长度的最大值
const int N = 10007;
int n;
int a[N], f[N];
// 状态计算
int main()
{
    cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> a[i]; // 从1开始初始化

    for (int i = 1; i <= n; i++)
    {
        f[i] = 1; // a[i]只有一个元素, 所以f[i]=1;
        for (int j = 1; j < i; j++) // 求i之前的f[j]
            if (a[j] < a[i])
                f[i] = max(f[i], f[j] + 1);
    }

    int res = 1;
    for (int i = 1; i <= n; i++)
        res = max(res, f[i]);
    cout << res << endl;
    return 0;
}
```

Python

```
import sys
sys.setrecursionlimit(200000)
# 最长上升子序列
n = int(input())
a = list(map(int, input().split()))
tails = []
for x in a:
    l, r = 0, len(tails)
    while l < r:
        mid = (l + r) // 2
        if tails[mid] >= x:
            r = mid
        else:
            l = mid + 1
    if l == len(tails):
        tails.append(x)
    else:
        tails[l] = x
print(len(tails))
```

NQ155 最长公共子序列 AcWing 897

"LCS——两个字符串的最长公共子序列。"小华说, "DP经典中的经典—— $f[i][j]$ =A前 i 个和B前 j 个的LCS长度。若 $A[i]=B[j]$ 则 $f[i][j]=f[i-1][j-1]+1$; 否则 $f[i][j]=\max(f[i-1][j], f[i][j-1])$ 。二维DP的范式。LCS的应用贯穿生物信息学 (DNA比对)、diff工具、拼写纠错——是字符串DP的核心模型。"

输入

第一行 n, m 。第二行A串, 第三行B串。

输出

LCS长度。

范围

 $1 \leq n, m \leq 1000$

输入

```
4 5
acbd
abedc
```

输出

3

思路 二维DP: $f[i][j]$ 定义如上。双重循环填充: 若匹配则左上+1, 否则取上/左的最大值。最终答案在 $f[n][m]$ 。时间 $O(nm)$, 空间 $O(nm)$ 可压到 $O(\min(n,m))$ 滚动数组。这是编辑距离的前身——只是代价函数不同。

技巧 LCS的转移图示: 方向箭头——左上(匹配)、上(跳过A[i])、左(跳过B[j])。空间优化: 只用两行交替($f[cur]$ 和 $f[prev]$)。LCS可以扩展为带权版本、多串版本——理解基本模型后扩展到海量变种。

C++

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

const int N = 1010;
int f[N][N];

int main()
{
    int n, m;
    string a, b;
    cin >> n >> m;
    cin >> a >> b;

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
        {
            f[i][j] = max(f[i-1][j], f[i][j-1]);
            if (a[i-1] == b[j-1]) f[i][j] = max(f[i][j],
            f[i-1][j-1] + 1);
        }

    cout << f[n][m] << endl;
    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
a, b = data[2], data[3]
dp = [[0]*(m+1) for _ in range(n+1)]
for i in range(1, n+1):
    for j in range(1, m+1):
        if a[i-1] == b[j-1]: dp[i][j]
        = dp[i-1][j-1] + 1
        else: dp[i][j] = max(dp[i-1]
        [j], dp[i][j-1])
print(dp[n][m])
```

NQ156 石子合并 AcWing 282

"区间DP的经典——N堆石子排成一排, 每次合并相邻两堆, 代价为两堆石子数之和, 求合并成一堆的最小总代价。"区间DP范式: $f[l][r]$ =合并 $[l,r]$ 区间的最小代价= $f[l][k]+f[k+1][r]+sum[l..r]$ 。三重循环——len从小到大($1 \rightarrow n$)、l从0开始、k在 $[l,r]$ 分割。最终答案 $f[0][n-1]$ 。区间DP是理解DP最优子结构的最佳入口。"

输入

第一行N。第二行N个整数表示每堆石子数。

输出 最小总代价。**范围** $1 \leq N \leq 300$

输入

4
1 3 5 2

输出

22

思路 区间DP模板: for len=2 to n: for l=0 to n-len: r=l+len-1; f[l][r]=INF; for k=l to r-1: f[l][r]=min(f[l][r], f[l][k]+f[k+1][r]+s[r+1]-s[l])。前缀和s[i]=前i堆的总和快速求区间和。O(n³), n≤300时OK。

技巧 区间DP三要素: len从小到大(保证子区间已算好)、l遍历起点、k遍历分割点。前缀和O(1)求区间和。这题可以用平行四边形优化到O(n²)但不在本课范围。理解这个三层循环模式, 所有区间DP问题都是它的变体。

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 307;

int n;
int s[N]; // 前缀和
int f[N][N]; // 所有将[i,j]合并成一堆的方案数

int main()
{
    cin>>n;
    for (int i = 1; i <=n; i++) cin>>s[i], s[i] += s[i-1]; // 计算前缀和

    // 区间DP, 先枚举长度, 后枚举端点
    for (int len = 2; len <= n; len++)
        for (int i = 1; i+len-1 <= n; i++) // 区间左端点
        {
            // 区间右端点
            int j = i + len - 1;
            f[i][j] = 1e9; // 初始化为无穷大
            for (int k = i; k < j; k++) // 区间dp, 枚举分隔点
                f[i][j] = min(f[i][j], f[i][k]+f[k+1][j]+s[j]-s[i-1]);
        }

    cout<<f[1][n]<<endl;
    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
n = int(data[0])
a = list(map(int, data[1:1+n]))
s = [0] * (n + 1)
for i in range(1, n+1): s[i] = s[i-1] + a[i-1]
INF = 10**9
f = [[0]*n for _ in range(n)]
for length in range(2, n+1):
    for l in range(n-length+1):
        r = l + length - 1
        f[l][r] = INF
        for k in range(l, r):
            f[l][r] = min(f[l][r], f[l][k] + f[k+1][r] + s[r+1] - s[l])
print(f[0][n-1])
```

"把字符串A变成字符串B——每次可以增、删、改一个字符，最少操作次数？"这是编辑距离(Levenshtein Distance)。" $f[i][j]$ =A前i个变B前j个的最小操作数。增： $f[i][j-1]+1$ ；删： $f[i-1][j]+1$ ；改： $f[i-1][j-1]+(A[i] \neq B[j])$ 。三重min。编辑距离是所有拼写检查、DNA比对、文档diff的底层算法——LCS的泛化。"

输入

第一行n和A串。第二行m和B串。

输出

最小操作次数。

范围

 $1 \leq n, m \leq 1000$

输入

```
4
AGTCTGACGC
3
AGTAAGTAGGC
```

输出

(见OJ实际输出)

思路 编辑距离DP: $f[i][j]$ 如上。初始化 $f[i][0]=i$ (全删), $f[0][j]=j$ (全增)。三重min转移。时间 $O(nm)$, 空间 $O(nm) \rightarrow$ 滚动数组 $O(\min(n, m))$ 。与LCS的区别在于可以有替换操作且替换 $\text{cost}=1$ 而非跳过代价。

技巧 编辑距离的初始化不要忘——空串变j字符=j次增加。三个操作对应三个方向：上=删、左=增、左上=改。每个方向+1(替换是否+1取决于字符是否相同)。这个DP是所有字符串距离算法的根基。

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;
const int N = 1007;

int n,m;
char a[N],b[N];
int f[N][N]; // 所有將a[i]變為b[i]的操作步驟

int main()
{
    cin>>n>>a+1; // 下標從1開始
    cin>>m>>b+1;

    // 初始化
    for (int i = 0; i < N; i ++ ) f[i][0] = i; // 把a[i]變為
    空串
    for (int i = 0; i < N; i ++ ) f[0][i] = i; // 把空串變為b
    [i]

    // DP
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
        {
            f[i][j] = min(f[i-1][j] + 1, f[i][j-1] + 1);
            if (a[i] == b[j]) f[i][j] = min(f[i][j], f[i-1][j-1]);
            else f[i][j] = min(f[i][j], f[i-1][j-1] + 1);
        }

    cout<<f[n][m]<<endl;

    return 0;
}
```

Python

```
import sys
sys.setrecursionlimit(200000)
# 最短编辑距离
n = int(input())
a = input().strip()
m = int(input())
b = input().strip()
f = [[0] * (m + 1) for _ in range(n + 1)]
for i in range(n + 1):
    f[i][0] = i
for j in range(m + 1):
    f[0][j] = j
for i in range(1, n + 1):
    for j in range(1, m + 1):
        if a[i - 1] == b[j - 1]:
            f[i][j] = f[i - 1][j - 1]
        else:
            f[i][j] = min(f[i - 1][j], f[i][j - 1], f[i - 1][j - 1] + 1)
print(f[n][m])
```

NQ158 滑雪

AcWing 901

"记忆化搜索——DP和搜索的完美结合。"华说，"给定一个二维矩阵表示雪山高度，从某点开始滑只能从高往低滑到四个方向。求最长滑行路径。dp[i][j]=从(i,j)出发的最长路径=1+max(dp[nx][ny])对所有合法邻居(高度更低)。用DFS+备忘录——递归第一次算，后续直接查表。记忆化=Top-down DP，递推=Bottom-up DP。"

输入

第一行R,C。然后R×C矩阵。

输出

最长滑行长度。

范围

 $1 \leq R, C \leq 300$

输入

```
5 5
1 2 3 4 5
16 17 18 19 6
15 24 25 20 7
```

输出

25

```
14 23 22 21 8
13 12 11 10 9
```

思路 记忆化DFS: $dp[i][j] = 1 + \max(\text{dfs}(nx, ny) \text{ for 所有高度更低的邻居})$ 。每次dfs先检查 $dp[i][j]$ 是否已算(不等于-1), 已算直接返回。每个状态算一次, $O(RC)$ 。比纯DFS快是因为避免了重复搜索——这正是DP的威力。

技巧 记忆化搜索=自顶向下DP。优势: 按需计算 (只计算能到达的状态)。劣势: 递归栈深度限制。DFS中四个方向的顺序不影响结果。Python递归可能需要`sys.setrecursionlimit(1000000)`。这道题展示了搜索→DP的自然过渡。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 307;

int g[N][N]; // 地图高度
int f[N][N]; // 从[i,j]开始滑的最长的轨迹

int n,m; // 宽高

int dx[4] = {-1, 0, 1, 0}, dy[4] = {0, 1, 0, -1};

int dp(int x, int y)
{
    // 记忆化搜索
    if (f[x][y] != -1) return f[x][y];
    // 否则, 从x,y开始滑
    f[x][y] = 1; // 当前位置至少是1格
    // 分上下左右四个方向滑雪
    for (int i = 0; i < 4; i++)
    {
        int a = x + dx[i], b = y + dy[i];
        if (a < 1 || a > n || b < 1 || b > m) continue; // 越界
        if (g[a][b] >= g[x][y]) continue; // 不能走
        // 四个方向深搜, 更新f[a][b], 取最大值
        f[x][y] = max(f[x][y], dp(a,b) + 1);
    }

    return f[x][y]; // 返回f[x][y]
}

int main()
{
    cin >> n >> m;
    // 读入高度图
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            cin >> g[i][j];

    memset(f, -1, sizeof f); // 初始化为-1, 表示该点未被访问

    // 把每个点作为起点深搜
    int res = 0;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            res = max(res, dp(i,j));

    cout << res << endl;
    return 0;
}

```

Python

```

# 滑雪
import sys
sys.setrecursionlimit(1000000)
R, C = map(int, input().split())
h = [list(map(int, input().split()))
      for _ in range(R)]
dp = [[-1] * C for _ in range(R)]
dx = [-1, 0, 1, 0]
dy = [0, 1, 0, -1]

def dfs(x, y):
    if dp[x][y] != -1:
        return dp[x][y]
    dp[x][y] = 1
    for d in range(4):
        nx, ny = x + dx[d], y + dy[d]
        if 0 <= nx < R and 0 <= ny < C and h[nx][ny] < h[x][y]:
            dp[x][y] = max(dp[x][y],
                           dfs(nx, ny) + 1)
    return dp[x][y]

ans = 0
for i in range(R):
    for j in range(C):
        ans = max(ans, dfs(i, j))
print(ans)

```

本章知识点总结

- **DP五步法**：定义状态→初始化→转移方程→遍历顺序→答案位置。模板先于变种
- **背包进化**：01倒序、完全正序——遍历顺序决定是否重复选

- **字符串DP三连**：LIS(一维+二分优化)、LCS(二维+状态转移)、编辑距离(三维min)
- **区间DP**：len从小到大、l枚举起点、k枚举分割。前缀和加速区间和查询
- **记忆化搜索**：DFS+备忘录=优雅的Top-down。滑雪、树形DP的标准写法

— 第15章完 · 共8题 · 自强不息 —

终极试炼

第16章 · 数学、贪心与综合实战 · 8题 · C++ & Python 双语对照

最后一章！小鲁回顾了这一路风景——从第一个A+B，到循环、数组、字符串、函数、STL、排序、二分、前缀和、差分、双指针、高精度、位运算、离散化、数据结构、搜索、图论、动态规划.....16章的征程即将画上句号。最后一章，我们用数学和贪心来收尾——有时候最优解不需要复杂算法，一个定理就够了。“编程的最高境界——不是写最多的代码，而是用最少的代码解决最难的问题。”

🏆 终极速查

- **贪心**：区间选点(排序右端点)、合并果子(Huffman+堆)——每次局部最优
- **并查集**：路径压缩+按秩合并——近乎 $O(1)$ 的集合操作
- **带权并查集**：食物链——维护到根的距离，模运算分类关系
- **数论**：快速幂 $O(\log b)$ 、筛质数(线性筛 $O(n)$)、中位数选址——数学直接给答案

NQ159 区间选点 AcWing 905

"在数轴上有N个区间，选出最少的点，使每个区间至少包含一个点。"小华说，"贪心经典题——按右端点排序，每次选当前未覆盖区间中最小的右端点作为新点。因为右端点排序后，选右端点覆盖最多后续区间。贪心的核心：'每次做当前看起来最好的选择'——但关键是证明贪心策略的正确性。排序+遍历= $O(n \log n)$ 。"

输入 第一行N。然后N行每行l r表示区间。

输出 最少点数。

范围 $1 \leq N \leq 10^5, -10^9 \leq l \leq r \leq 10^9$

输入	输出
3	2
-1 1	
2 4	
3 5	

思路 贪心策略：按右端点升序排序。 $ans=0, last=-INF$ 。遍历每个区间 $[l,r]$ ：若 $l > last$ 则需新点 $ans++$, $last=r$ 。证明：按右端点排序后，每次选最小的右端点保证了对后续区间的最大覆盖能力。

技巧 贪心的正确性证明通常用'交换论证'或'归纳法'。区间问题的通用解法：先排序再贪心——排序确定处理的顺序，贪心确定每一步的选择。Python: `intervals.sort(key=lambda x: x[1])`按右端点排序。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007;

struct Range
{
    int l, r;
    bool operator< (const Range & R) const
    {
        return r < R.r; // 重载比较符, 根据r排序
    }
} range[N];

int main()
{
    int n;
    scanf("%d", &n);
    // 读入区间
    for (int i = 0; i < n; i++)
    {
        int l, r;
        scanf("%d%d", &l, &r);
        range[i] = {l, r};
    }
    // 按照右端点排序
    sort(range, range + n);
    // 根据右端点计算覆盖的区间个数
    int res = 0, ed = -2e9; // 已统计区间的右端点
    for (int i = 0; i < n; i++)
    {
        if (range[i].l > ed)
        {
            res++;
            ed = range[i].r;
        }
    }
    printf("%d\n", res);

    return 0;
}

```

Python

```

# 区间选点
n = int(input())
segs = []
for _ in range(n):
    l, r = map(int, input().split())
    segs.append((l, r))
segs.sort(key=lambda x: x[1])
ans = 0
last = -2 * 10**9
for l, r in segs:
    if l > last:
        ans += 1
        last = r
print(ans)

```

NQ160 合并果子 AcWing 148

"N堆果子，每次合并任意两堆，代价为两堆之和。求合并成一堆的最小总代价。"这就是Huffman编码！"每次选最小的两堆合并——用小根堆维护。贪心+Huffman= $O(n \log n)$ 。"小华说，"Huffman树的每个内部节点对应一次合并，叶子是原始堆。最优性证明：最小的两堆一定在Huffman树的最深层——交换论证。"

输入 第一行n。第二行n个整数。

输出 最小总代价。

范围 $1 \leq n \leq 10000$

输入

输出

3
1 2 9

15

思路 小根堆(优先队列): 每次pop两个最小值, 合并(=和), 将和push回堆, $ans += \text{和}$ 。重复直到堆中只剩一个元素。时间 $O(n \log n)$ 。Python用heapq: `heapify(arr); while len>1: a,b=heappop,heappop; heappush(a+b); ans+=a+b。`

技巧 Huffman编码=最优二叉树构造。堆的选择: 小根堆取最小值最快($O(\log n)$)。这是优先队列最重要、最经典的应用——Dijkstra、Prim都用到了堆。Python的heapq模块简单高效——记住heappush+heappop组合。

C++

```
#include <iostream>
#include <queue>

using namespace std;

int main()
{
    int n; cin>>n;
    // 优先队列, 使用greater作比较函数, 把默认的大根堆变为小根堆
    priority_queue<int, vector<int>, greater<int>> heap;
    // 把读入数据压入堆
    while (n--)
    {
        int x; cin>>x;
        heap.push(x);
    }

    int res = 0;
    while(heap.size() > 1)
    {
        int x = heap.top(); heap.pop();
        int y = heap.top(); heap.pop();
        res += x + y; // 累计合并值
        heap.push(x+y); // 把当前合并的值压入堆
    }

    cout<<res<<endl;

    return 0;
}
```

Python

```
import sys, heapq
data = sys.stdin.read().split()
n = int(data[0])
a = list(map(int, data[1:1+n]))
heapq.heapify(a)
ans = 0
while len(a) > 1:
    x = heapq.heappop(a)
    y = heapq.heappop(a)
    ans += x + y
    heapq.heappush(a, x + y)
print(ans)
```

NQ161 合并集合 AcWing 836

"并查集(Disjoint Set Union)——处理集合的合并和查询。"小华画出带箭头的图, "每个集合用一个树表示, 根节点是集合代表。find(x): 找x的根。union(x,y): 将x根连到y根。两种优化: 路径压缩(find时把所有路径节点直接连根)和按秩合并(小树连到大树)。均摊复杂度近乎 $O(1)$ ——这是数据结构设计中最精妙的成就之一。"

输入

第一行 n, m 。然后 m 行: $M \ a \ b$ (合并)或 $Q \ a \ b$ (查询是否同集)。

输出

对Q操作输出Yes/No。

范围

 $1 \leq n, m \leq 10^5$

输入	输出
4 5	Yes
M 1 2	No
M 3 4	Yes
Q 1 2	
Q 1 3	
Q 3 4	

思路 并查集: $p[x]$ 存父节点(初始 $p[x]=x$)。 $find(x)$: if $p[x] \neq x$: $p[x]=find(p[x])$; return $p[x]$ ——路径压缩。
 $union(x,y)$: $p[find(x)]=find(y)$ 。所有操作均摊 $O(\alpha(n)) \approx O(1)$ 。这是数据结构课程上最简洁又最高效的设计之一。

技巧 并查集是竞赛中使用频率最高的数据结构。路径压缩($find$ 中的 $p[x]=find(p[x])$)是将树扁平化的关键。按秩合并: 若两个集合秩相同则根秩+1。Python递归 $find$ 可能超深度——用while迭代或 `sys.setrecursionlimit`。并查集在Kruskal、连通性、动态图问题中是核心工具。

C++

```
#include <cstdio>
const int N = 100007;
int p[N];
int n;

int find(int x)
{
    while (p[x] != x) {
        p[x] = p[p[x]];
        x = p[x];
    }
    return x;
}

int main()
{
    int m;
    scanf("%d", &n, &m);
    for (int i = 1; i <= n; i++) p[i] = i;
    while (m--) {
        char op;
        int a, b;
        scanf(" %c%d%d", &op, &a, &b);
        if (op == 'M') p[find(a)] = find(b);
        else puts(find(a) == find(b) ? "Yes" : "No");
    }
    return 0;
}
```

Python

```
import sys
sys.setrecursionlimit(200000)
data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
p = list(range(n+1))
def find(x):
    while p[x] != x:
        p[x] = p[p[x]]
        x = p[x]
    return x
idx = 2
out = []
for _ in range(m):
    op, a, b = data[idx], int(data[idx+1]), int(data[idx+2])
    idx += 3
    if op == 'M': p[find(a)] = find(b)
    else: out.append('Yes' if find(a) == find(b) else 'No')
print('\n'.join(out))
```

"上一题的扩展——除了查询连通性，还要知道每个连通块的大小。"小华说，"在并查集中额外维护一个size数组——初始size[i]=1。合并时size[新根]+=size[被合并的根]。查询某个节点所在连通块大小只需size[find(x)]。如果在合并时重复合并同一集合，size不变——避免重复累加。这是并查集的自然扩展。"

输入 第一行n,m。然后m行: C a b(合并,若已连通忽略) / Q1 a b(查询连通) / Q2 a(查询a所在连通块大小)。

输出 按要求输出。

范围 $1 \leq n, m \leq 10^5$

输入	输出
5 5	Yes
C 1 2	2
Q1 1 2	3
Q2 1	
C 2 3	
Q2 1	

思路 维护size的并查集：初始化size[i]=1。union时：if a_root!=b_root: p[a_root]=b_root; size[b_root]+=size[a_root]。注意必须先判连通再合并——避免重复size。find不变。

技巧 并查集可以维护更多信息：size、距离到根(带权并查集)、最小值/最大值等。关键是理解'合并时如何更新附加信息'。食物链(240)是带权并查集的经典应用——每个节点维护与父节点的关系。

C++

```
#include <cstdio>
const int N = 100007;
int p[N], sz[N];
int n;

int find(int x)
{
    while (p[x] != x) {
        p[x] = p[p[x]];
        x = p[x];
    }
    return x;
}

int main()
{
    int m;
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; i++) { p[i] = i; sz[i] = 1; }
    while (m--) {
        char op[4];
        scanf("%s", op);
        if (op[0] == 'C') {
            int a, b;
            scanf("%d%d", &a, &b);
            int ra = find(a), rb = find(b);
            if (ra != rb) { p[ra] = rb; sz[rb] += sz[ra]; }
        } else if (op[1] == '1') {
            int a, b;
            scanf("%d%d", &a, &b);
            puts(find(a) == find(b) ? "Yes" : "No");
        } else {
            int a;
            scanf("%d", &a);
            printf("%d\n", sz[find(a)]);
        }
    }
    return 0;
}
```

Python

```
import sys
sys.setrecursionlimit(200000)
data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
p = list(range(n+1))
sz = [1]*(n+1)
def find(x):
    while p[x] != x:
        p[x] = p[p[x]]
        x = p[x]
    return x
idx = 2
out = []
for _ in range(m):
    op = data[idx]
    if op == 'C':
        a, b = int(data[idx+1]), int(data[idx+2])
        idx += 3
        ra, rb = find(a), find(b)
        if ra != rb: p[ra] = rb; sz[r
b] += sz[ra]
    elif op == 'Q1':
        a, b = int(data[idx+1]), int(data[idx+2])
        idx += 3
        out.append('Yes' if find(a) == find(b) else 'No')
    else:
        a = int(data[idx+1])
        idx += 2
        out.append(str(sz[find(a)]))
print('\n'.join(out))
```

NQ163 食物链 AcWing 240

"动物王国三种关系：A吃B、B吃C、C吃A——循环克制。"这是带权并查集的巅峰应用！"每个节点维护到根的距离 $d[x]$ ： $d[x]\%3=0$ 与根同类， $=1$ 吃根， $=2$ 被根吃。合并集合时根据两个节点的关系计算出它们根之间的距离关系。路径压缩时同步更新 $d[x]$ 。这是竞赛中最精彩的并查集应用！"

输入 第一行N,K。然后K行：D X Y (D=1同类,2表示X吃Y)。

输出 假话数量。

范围 $1 \leq N \leq 50000, 0 \leq K \leq 100000$

输入

```
100 7
1 101 1
```

输出

```
3
```

```
2 1 2
2 2 3
2 3 3
1 1 3
2 3 1
1 5 5
```

思路 带权并查集： $p[x]$ 存父， $d[x]$ 存与父的关系(0同类/1吃父/2被父吃)。 $find$ 时路径压缩： $root=find(p[x]); d[x]+=d[p[x]]; p[x]=root$ 。 合并X和Y： 根据 $D=1/2$ 计算X根和Y根的距离关系。 判断假话： X/Y超范围、已连通但与关系不符。

技巧 食物链是并查集+模3运算的智力巅峰。 $d[x]\%3$ 是与根的关系： 0同类、1吃根、2被根吃。 路径压缩时 $d[x]+=d[p[x]]$ 保证距离累加正确。 理解这道题后， 所有带权并查集问题都不在话下。 这是OI/ACM金牌选手的基本功。

C++

```

#include <iostream>

using namespace std;

const int N = 50007; // Andy风格

int n, m;
int p[N], d[N]; // p集合 d距离初始化为0

int find(int x)
{
    if (p[x] != x)
    {
        int t = find(p[x]);
        d[x] += d[p[x]];
        p[x] = t;
    }
    return p[x];
}

int main()
{
    scanf("%d%d", &n, &m);

    for (int i = 1; i <= n; i++) p[i] = i; // 初始化p集合

    int res = 0;
    while (m --)
    {
        int t, x, y;
        scanf("%d%d%d", &t, &x, &y);

        if (x > n || y > n) res ++;
        else
        {
            int px = find(x), py = find(y); // 两个集合的父节点
            if (t == 1)
            {
                if (px == py && (d[x] - d[y]) % 3) res ++;
                else if (px != py)
                {
                    p[px] = py;
                    d[px] = d[y] - d[x];
                }
            }
            else
            {
                if (px == py && (d[x] - d[y] - 1) % 3) res ++;
                else if (px != py)
                {
                    p[px] = py;
                    d[px] = d[y] + 1 - d[x];
                }
            }
        }
    }

    printf("%d\n", res);
    return 0;
}

```

Python

```

import sys
sys.setrecursionlimit(200000)
data = sys.stdin.read().split()
n, k = int(data[0]), int(data[1])
p = list(range(n + 1))
d = [0] * (n + 1)

def find(x):
    if p[x] != x:
        root = find(p[x])
        d[x] += d[p[x]]
        p[x] = root
    return p[x]

ans = 0
idx = 2
for _ in range(k):
    t = int(data[idx]); x = int(data[idx+1]); y = int(data[idx+2])
    idx += 3
    if x > n or y > n:
        ans += 1; continue
    rx, ry = find(x), find(y)
    if t == 1:
        if rx == ry and (d[x] - d[y]) % 3 != 0: ans += 1
        elif rx != ry:
            p[rx] = ry
            d[rx] = d[y] - d[x]
    else:
        if rx == ry and (d[x] - d[y] - 1) % 3 != 0: ans += 1
        elif rx != ry:
            p[rx] = ry
            d[rx] = d[y] - d[x] + 1
print(ans)

```

NQ164 快速幂 AcWing 875

"计算 $a^b \bmod p$ —— b 可能大到 10^9 。"小华在黑板上写下算法，"朴素循环 $O(b)$ 超时。快速幂：利用 $a^b = a^{(b/2)} \times a^{(b/2)}$ ——分治+二进制分解。while(b): if(b&1) res=res*a%p; a=a*a%p; b>>=1。核心：将指数按二进制位拆开计算——时间 $O(\log b)$ 。这个模板出现在RSA加密、矩阵快速幂、模逆元等场景中。"

输入 第一行 n 。然后 n 行每行 $a \ b \ p$ 。

输出 每行 $a^b \bmod p$ 。

范围 $1 \leq n \leq 10^5, 1 \leq a, b, p \leq 2 \times 10^9$

输入	输出
2	3
2 3 5	2
3 2 7	

思路 快速幂while循环： $a^b \bmod p$ 。while $b > 0$: if $b \& 1$: $res = res * a \% p$; $a = a * a \% p$; $b \gg = 1$ 。每步 b 减半 $\rightarrow O(\log b)$ 。Python: `pow(a,b,p)`一行内置(支持三参数取模)——底层用快速幂实现。但手写理解原理是必须的。

技巧 Python `pow(a,b,p)`三步参数： $a^b \bmod p$ ——内置最高效。快速幂是数论算法的基础——之后模逆元、快速幂+矩阵乘法(矩阵快速幂)、米勒-拉宾素性检测都依赖于此。位运算 $b \& 1$ 判断奇偶比 $b \% 2$ 更快。

C++

```
#include <iostream>
#include <algorithm>
using namespace std;
typedef long long LL;
int qmi(int a, int b, int p)
{
    int res = 1; //初始值为1
    while (b)
    {
        if (b & 1) //b的最末尾为1
            res = (LL)res * a % p; //必须用long long防止溢出
        b >>= 1; //去掉b最低位
        a = (LL)a * a % p;
    }
    return res;
}

int main()
{
    int n;
    scanf("%d", &n); //数据很大, 需要用LL读入
    while (n--)
    {
        int a, b, p;
        scanf("%d%d%d", &a, &b, &p); //读入a b p

        printf("%d\n", qmi(a, b, p)); //计算快速幂
    }
    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
n = int(data[0])
out = []
idx = 1
for _ in range(n):
    a, b, p = int(data[idx]), int(data[idx+1]), int(data[idx+2])
    idx += 3
    out.append(str(pow(a, b, p)))
print('\n'.join(out))
```

NQ165 筛质数 AcWing 868

"找出1到n中的所有质数。"小华说, "埃氏筛法(Eratosthenes): 从2开始, 依次划掉2的倍数、3的倍数..... 未被划掉的就是质数。优化: 每个数只被它最小的质因子划掉——这就是线性筛。线性筛保证每个合数只被标记一次, $O(n)$ 。理解线性筛是理解数论算法的关键——最小质因子是数论中最核心的概念之一。"

输入 一个整数n。

输出 1到n中质数的个数。

范围 $1 \leq n \leq 10^6$

输入

8

输出

4

思路 埃氏筛: bool数组标记合数, for i from 2: if !st[i]: primes.push(i); for j=i到n步长i: st[j]=true。 $O(n \log \log n)$ 。线性筛: 每个数只被最小质因子筛——for p in primes: if i*p>n break; st[i*p]=true; if i%p==0 break。 $O(n)$ 。

技巧 筛法不仅是求质数——还可以同时记录最小质因子、欧拉函数、约数个数等。线性筛复杂度 $O(n)$ 所以可以处理 $n=10^7$ 。Python的筛法比C++慢几倍但 $n=10^6$ 仍可。关键优化：只用 $\leq \sqrt{n}$ 的质数来筛。

C++

```
// Andy 2021.06.04
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100000007;

int primes[N], cnt;
bool st[N];
void get_primes(int n) // 线性筛质数
{
    for (int i = 2; i <= n; i++)
    {
        if (!st[i]) primes[cnt++] = i;
        for (int j = 0; primes[j] <= n / i; j++)
        {
            st[primes[j] * i] = true;
            if (i % primes[j] == 0) break;
        }
    }
}

int main()
{
    int n; cin >> n;
    get_primes(n);
    cout << cnt << endl;
    return 0;
}
```

Python

```
import sys
n = int(sys.stdin.read().split()[0])
st = [False] * (n + 1)
primes = []
for i in range(2, n + 1):
    if not st[i]:
        primes.append(i)
        for p in primes:
            if p * i > n: break
            st[p * i] = True
            if i % p == 0: break
print(len(primes))
```

NQ166 货仓选址 AcWing 104

"最后一个问题——在数轴上选一个位置建仓库，使到N个商店的总距离最小。"小华笑道，"答案是中位数！将商店坐标排序，最优位置=中位数位置。证明：如果把仓库往任意方向移dx，离一半商店近了dx但离另一半远了dx——中位数处达到平衡。这展示了数学思维在算法优化中的力量——有时候最优解不需要代码搜索，数学定理直接给出答案。"

输入 第一行N。第二行N个整数。

输出 最小总距离。

范围 $1 \leq N \leq 100000$

输入

4
6 2 9 1

输出

12

思路 排序取中位数：`sort(arr); median=arr[n//2]; ans=sum(abs(a-median))`。 $O(n \log n)$ 。进阶：如果N是偶数，任何在中间两个数之间的位置都是最优的。中位数距离和是最小绝对偏差问题的标准答案——不需要搜索，理解定理就完成了。

技巧 中位数vs平均数：最小化绝对偏差和用中位数，最小化平方偏差和用平均数。这是统计学的基础结论。Python一行：`median=sorted(arr)[n//2]; print(sum(abs(x-median) for x in arr))`。16章到此结束——从A+B到货仓选址，你已经完成了算法竞赛入门的所有基础训练。

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007;

int n;
int a[N];

int main()
{
    cin >> n;
    for (int i = 0; i < n; i++) cin >> a[i];
    sort(a, a+n);
    int res = 0;
    // 中位数a[n/2], 求距离之和的最小值
    for (int i = 0; i < n; i++) res += abs(a[i] - a[n/2]);
    cout << res << endl;
    return 0;
}
```

Python

```
import sys
data = sys.stdin.read().split()
n = int(data[0])
a = list(map(int, data[1:1+n]))
a.sort()
mid = a[n // 2]
ans = sum(abs(x - mid) for x in a)
print(ans)
```

 全书总结

- 16章·161题·C++/Python双语：从A+B到货仓选址——完整覆盖算法竞赛入门的全部知识点
- 四大阶段：语法基础(L1-5)→编程进阶(L6-8)→核心算法(L9-13)→算法进阶(L14-16)
- 两语言并行：C++让你理解底层，Python让你快速验证——两种视角互补
- 厦大校训：自强不息，止于至善。16章的征程结束，但算法之路刚刚开始。